大模型协作记录（AI Collaboration Log）

> 竞赛要求：提交**体现大模型协作过程的交互记录，含提示词、模型回答与自我纠错轨迹**（`COMP_GUIDE_2026.txt:807-808`）；报告正文中需有专门章节 「大模型协作记录：提示词、模型回答、自我纠错轨迹、智能体工作流设计（如有）」（`COMP_GUIDE_2026.txt:857`）。
>
> 本文件的目的是把**真实发生过的事情**按可核查的格式归档。三条自我约束：
>
> 1. **每一条都带仓库内锚点**（`docs/DEV_STATUS.md:行号`、源文件行号或产物文件名）。 没有锚点的事件不写进本记录 —— 因为无法验证的"协作故事"没有价值。
> 2. **用户提示词按原话引用**，并标注出处（本仓库里用户原话最权威的三个落点见 §1.2）。
> 3. **记录失败。**本记录里至少 8 条是"我先做错了、然后被实测推翻"。 只写"AI 一次就做对了"的记录，在评审眼里等于没写。

---

1. 协作方式总览

1.1 人与模型的角色分工

角色	承担	具体表现
人（队长）	需求、拍板、物理世界、最终验收	提需求原话；在多个方案里选一个（`m08918` 五项全选"推荐"）；接线与上电；示波器/信号源操作；决定"先做哪一项"
模型（AI 智能体）	设计、实现、取证、自检、归档	读写仓库、跑仿真与实现、跑真机脚本、写文档、自己找自己的 bug
确定性工具	判定（不依赖模型）	`sim/logs/SUMMARY.md`、`impl_timing.rpt`、`*_check.py` 的退出码

最重要的一条约定：凡是"结论"都要落到**不依赖模型立场**的判定上 —— 测试台的 `TEST PASSED/FAILED`、实现的 `All user specified timing constraints are met.`、 `tools/pinplan_check.py` 的 `RESULT: PASS`、真机脚本的 `PASS n/m`。 模型的自述只用于解释，不用于结论。

1.2 用户提示词的三个权威落点（可直接核对）

落点	内容
`docs/10_host_requirements.md:188` 起「附录 A：需求原话（用户 m00001）」	上位机软件的第一版完整需求原话
`_synth_work/prompt_host.md`（8 行）	界面优化六条需求的**全文原话**，与 `docs/12_host_ui_requirements.md` 的 R1–R6 逐条对应；第 7–8 行是后续追加的放大态需求（对应批次 4a/4b）
`_synth_work/prompt_disp.md`（3 行）	7 寸屏本机界面的需求原话 —— ⚠️ **这一条线尚未实现**，本记录只在 §4 当作"下一阶段"提及，**不计入已完成工作**

⇒ 这三份文件本身就是"提示词"的物证：它们是用户输入被原样落盘的产物，不是我事后重写的。

1.3 一个典型回合的流程（本记录的固定行文格式）

① 用户原话（带消息锚点）
② 我的错误假设 / 第一版方案
③ 暴露它的数字或现象（来自实测或仿真，不是来自我的推理）
④ 修法 + 验证（改了什么、用什么证明改对了）
⑤ 可复用结论（已沉淀到 skill/ 的哪一项）

---

2. 自我纠错轨迹（86 条真实记录）

记录 1：测试台"空转"——激励根本没接到被测逻辑上

- ① **触发**：多轮仿真"看起来正常"但覆盖率异常。用户提示词原文见 `docs/DEV_STATUS.md:465-493`（§1.11）。
- ② **错误假设**：我默认 `tb_*` 里的激励已经接到了 DUT 端口上，于是把注意力放在 DUT 逻辑上。
- ③ **暴露它的现象**：写了一个**只找"被计算但从未被读"的线**的静态检查（`tools/scan_dangling.py`），它列出测试台内部的中间信号 —— 激励驱动的信号 **从未出现在任何 DUT 端口连接里**。也就是说那个测试台一直在自己跟自己玩。
- ④ **修法 + 验证**：修测试台连接，重跑；同一批测试台从"看似全绿"变成真正跑通。 此后 `tools/scan_dangling.py` / `tools/scan_unused.py` 成为固定检查项， 与回归并列跑（见 `skill/vivado-report-autocollect.md` §3.4）。
- ⑤ **可复用结论**：**"测试通过"必须先证明"测试真的在测东西"**。 静态检查（死信号）能抓住断言永远抓不到的东西。 ⇒ 沉淀进 `skill/testbench-blindspot.md`（"先怀疑工具与测试台，再怀疑逻辑"）。

记录 2：工具真 bug 污染了实验数据 —— 该先怀疑工具

- ① **触发**：真机读回的寄存器值与写入值不符，一度被当作"RTL 有 bug"。
- ② **错误假设**：先怀疑 RTL。
- ③ **暴露它的数字**：`docs/DEV_STATUS.md:3549-3552`（§1.68 D）—— 调试脚本 `0x051A` 的位域被**截断成 4 位**；并且"哨兵写 `0x00FF` "这个动作 在 RTL 里被解释成 **粗调 ×8**。两条都是**上位机脚本自己的 bug**。
- ④ **修法 + 验证**：修脚本位宽与哨兵值，重读，RTL 行为与手册一致。
- ⑤ **可复用结论**：**"工具真 bug 污染实验数据"是高概率事件**。 修法顺序：先证伪工具 → 再证伪测试台 → 最后怀疑被设计对象。 ⇒ 沉淀进 `skill/testbench-blindspot.md`。

记录 3：集成路径 Z 脉宽被窗截断 —— 单元测试台全部"假通过"

- ① **触发**：Z 输出的脉宽与理论值不符。
- ② **错误假设**：单元测试台（`tb_abz_jitter` 等）已覆盖 Z 宽度，于是从别处找原因。
- ③ **暴露它的数字**：`docs/DEV_STATUS.md:2200-2223`（§1.44）—— **单元测试台量到 2 个时钟拍，集成路径实测 2042 拍，理论值 2048 拍**。 根因是集成路径里 Z 被 `z_window` 窗**截断**（`z_window = (cycle_cnt[5:0] == 0)`， `rtl/enc_top.v:643-644`），而单元测试台**没有这个窗**，于是单元台全部"假通过"。
- ④ **修法 + 验证**：把结论标注为"**只在窗常开下成立 / 集成路径 v7 实为 40 ns，v8 起才成立**"， 并同步改掉 5 处受影响文档（`docs/DEV_STATUS.md` §1.26 的 `z_width=4 ⇒ 3.9 µs` 表、 §1.33 ② 的 `tb_abz_jitter` "Z 宽度 3.84 µs"、 `docs/06_board_test_guide.md:293`、 `docs/05_vivado_sim_impl_report.md:153` 与 13

项表第 13 条）。 ★ 并诚实记下：`docs/DEV_STATUS.md` §1.42 那次代码审计**没有发现这条截断**， 因为它只比对代码与注释，没有跨"单元台 vs 集成台"比对。

- ⑤ **可复用结论**：**单元测试全绿 ≠ 集成正确**； 凡是被上层信号门控的时序参数，必须在**集成台**上再量一次。 ⇒ 沉淀进 `skill/testbench-blindspot.md`。

记录 4：ABZ 边沿抖动量化 + "过速静默且无告警"的监视盲区

- ① **触发**：用户提示词原文要求量化 AB 抖动并解释过速行为 （`docs/DEV_STATUS.md:1614-1681`，§1.33；触发消息 `m02801@` / `m00598`）。
- ② **错误假设**：我起初只打算"加大迟滞 `hyst`"来压抖动（这是规范里的常规建议）。
- ③ **暴露它的数字**：量化后发现 —— 规范里"加大 hyst"的建议**只在"角度干净、抖动纯属量化"时才成立**；本工程设计里角度本身有噪声，加大 hyst **无益**。 同时发现"过速（UVW 频率告警阈值以上）时输出静默且没有任何告警位"这一监视盲区。
- ④ **修法 + 验证**：把量化依据写进文档，并**明确推翻**自己此前引用的常规建议 （`docs/DEV_STATUS.md` §1.40 ③ 修正 §1.39 的结论）； 新增 `uvw_freq_warn` 类告警位 （帧 off10 的 `io_flags` bit3）进遥测帧，使盲区可观测。
- ⑤ **可复用结论**：**引用手册建议前，先验证该建议的前提在本设计里是否成立。**

记录 5：位域解码口径写错 ⇒ 20 轮统计全部失真

- ① **触发**：一段连续 20 轮的真机统计结果。
- ② **错误假设**：把这 20 轮当作设计质量的证据。
- ③ **暴露它的数字**：`docs/DEV_STATUS.md:1362`、`:1376-1393` —— 原文写着"**我的解码器当时写错了**"：位域解码口径错了，所以那 20 轮的统计**全部失真**， 不能作为结论。
- ④ **修法 + 验证**：修解码器、重跑 20 轮，重出统计。
- ⑤ **可复用结论**：**统计结果的可信度上限 = 解码口径的正确性。** 凡是"批量统计得出一个漂亮结论"，先拿 1 帧**逐位手工对照**一次。

记录 6：时序约束口径 —— 负余量是"诚实建模"的结果，不是设计错误

- ① **触发**：实现报告出现 `Timing constraints are not met.`
- ② **错误假设**：第一反应是"设计没收敛，得降频"（并把"降到 25 MHz"写成了回退项）。
- ③ **暴露它的数字**：`docs/DEV_STATUS.md:846-885`（§1.19）—— ADC 源同步输入路径按数据手册诚实建模：`set_input_delay -max 14.000/-min 9.500`， 给定源时钟 16.78 ns、总预算 19.78–23.78 ns，得到 **WNS −3.271 ns / 26 个失败端点 全部是 ADC 输入路径。** ⇒ 这个负值是"把真实最坏情况写进约束"的必然结果， **不是综合器失误。**
- ④ **修法 + 验证**：不掩盖：报告里写明"负值里约 3 ns 是相位口径错误，其余是真实建模余量"， 并给出实证回退顺序（`constr/enc_top.xdc:243-259` 五步）： ① 缩短排线 / 串 22~33 Ω；② 捕获改 `clk_i` 下降沿；③ 降采样率；④ 最后才 `set_false_path` （必须写明"ADC 输入路径未做时序分析"）。 ★ 最终由**上板实测**给出结论：示波器量 `tPD_meas = 3–4 ns`（手册 2…6 ns 内）， 建立余量 ≈ **+12.3 ns** ⇒ **①②③④均确认不需要执行**， 同相捕获 + `-max 14.000/-min 9.500` 已收敛（`sys_clk` **WNS +0.959 / 0 失败端点**）。
- ⑤ **可复用结论**：**"未达标"要分清是"设计没做到"还是"约束写得诚实"**， 并且**用一句可量测的上板判据**（这里是 `tPD_meas ≤ 4 ns`）把争论终结。 ⇒ 沉淀进 `skill/adc-source-sync-constraints.md`。

记录 7：环境类假失败三连 —— 残留进程 / 假成功退出 / 挂死

- ① **触发**：同一脚本、同一 RTL，有时通过有时"失败"。
- ② **错误假设**：先怀疑 RTL 改坏了。
- ③ **暴露它的现象**（三条，全部有记录）： 1. `docs/DEV_STATUS.md:26-29`：首轮回归报 `ERROR: [XSIM 43-3356] Could not open file xsim.dir/tb_enc_top_sim/xsim.type for writing` —— 根因是**上一轮被中断的 xsim 进程仍占用该目录**（PID 29468 / 28132）； 杀进程后单跑立即 PASS（497 s）。 ⇒ 由此给**每个测试台分配独立工作目录**（`_sim_work/<tb>/`）。 2. `docs/DEV_STATUS.md:2278`： `vivado` 在打印 `Bitgen Completed Successfully` 之后 **不再退出**（17 分钟无新输出）⇒ 按 `.bit` 与五份 `impl_*.rpt` 的**时间戳与哈希** 确认写完后手工终止 —— 脚本"几乎成功"其实就是成功。 3. `docs/DEV_STATUS.md:2534` / `:2537` / `:3186`：受限环境下 `opt_design` 展开 debug core **挂死**（`rundef.js(122,5) Access denied`，两个 OOC run 子进程 0% CPU 死等 11 分钟）； 另有 `Get-PSDrive` 报 0 GB 而实际 29 GB 可用的假信息。 ⇒ ILA/生产位流构建改用全权限跑，同一脚本一次通过。
- ④ **修法 + 验证**：把这三条写进工具文档，并把"**用产物时间戳与哈希判断构建是否真的完成**" 固化成流程（见 `skill/vivado-report-autocollect.md` §3.2）。
- ⑤ **可复用结论**：**脚本 `exit 1` 不等于构建失败；脚本 `exit 0` 也不等于成功。** 判据是产物。

记录 8：我自己的预测被自己的实测证伪（诚实记录，结论悬置）

- ① **触发**：对 `angle_raw` 毛刺的来源给出过一个几何假设（"Gauss 格点"）。
- ② **错误假设**： `docs/DEV_STATUS.md:2701-2710` 的预测 —— 毛刺与某个格点几何关系一一对应。
- ③ **暴露它的数字**： `docs/DEV_STATUS.md:2747-2753` 、 `:2828-2831` —— 用 4 版 `ang_glitch` 触发窗口 + 环路再注入对比 + v6 复算， **"我上一节的预测被证伪（诚实记录，不许糊弄）"**。
- ④ **修法 + 验证**：改为**三轮修正后把结论悬置**，明确列出仍需的激励 （当时没有可用的外部激励，链路错误位在板上恒 0，是平凡结果； 要真正激励须用户改接线后复测）。
- ⑤ **可复用结论**：**结论悬置是一种合法产出。** 写下"我还需要什么证据才能定论"，比编一个自洽的故事有价值。 ⇒ 沉淀进 `skill/adc-source-sync-constraints.md` （"判据要可量测"） 与 `skill/deterministic-measurement.md` （"证据不足时结论悬置"）。

记录 9：子代理分片审计 —— 23 个文件 / 6 个子代理 / 26 处真缺陷

- ① **触发**：用户提示词原文（`m03503`）要求做一次全面自查，并亲自给了分片清单（`docs/DEV_STATUS.md:3175-3199`，§1.66）。
- ② **我的做法**：把 23 个文件切成 6 片，**每片一个独立子代理**， 子代理只负责"找问题并列证据"，**不允许改代码**；我再逐条复核、只修真的。
- ③ **暴露的数字**：6 片合起来报了一批疑点，逐条复核后确认 **26 处真缺陷**， 包括： `encoder` 丢帧率口径错、 `link` 命令 0.502 s/条且丢命令、 `regs` 读失败被当作 0 覆盖、 `disconnect_port` 丢尾巴、 当时叫 `signal.py` 的 `evaluate` 崩溃（该文件后来改名 `alarm.py`）。
- ④ **修法 + 验证**：逐条修复并重跑；报告 `docs/11_host_selfcheck_s1_s5.md` ， 逐条证据 `docs/DEV_STATUS.md` §1.66；真机 S1–S5 全套复跑保持全绿。
- ⑤ **可复用结论**：**分片 + 独立评审 + 逐条复核**是"AI 自查"唯一有效的形态：
- 分片让每片的上下文足够小，评审才认真；
- "只找问题、不改代码"避免审计者顺手改坏；
- 复核环节必须有，因为子代理会报**误报**（本次报了不止 26 条）。 ⇒ 沉淀进 `skill/deterministic-measurement.md` 与 `skill/testbench-blindspot.md` （"谁有权下结论"与"怎么证明检查真的在检查"）。

记录 10：打包版启动即崩 —— `host/signal.py` 遮蔽标准库 `signal`

- ① **触发**：用户原话（`m07281` / `m07427` / `m07468`）：源码跑得好好的， **打包成 exe 之后启动就崩。**用户提示词原文见 `docs/DEV_STATUS.md:4331-4334`（§1.77）。
- ② **错误假设**：先怀疑 PyInstaller 打包参数、缺依赖。
- ③ **暴露它的原因**：`host/signal.py` 这个文件**遮蔽了 Python 标准库的 `signal` 模块** —— 源码模式下启动路径不同所以没暴露，打包后 `signal` 被优先解析到本仓库文件， 标准库功能失效 ⇒ 启动即崩。
- ④ **修法 + 验证**：文件改名 `host/signal.py` → `host/alarm.py`， 并同步所有 import；重新打包 + 目标机双击验证通过（S6 验收）。 ★ 这个改名在本次会话里还能在 `git status` 里看到： `RM host/signal.py -> host/alarm.py`。
- ⑤ **可复用结论**：**不要用标准库模块名当自己的文件名**（`signal` / `types` / `queue` / `select` 都是高危名）。这个坑只在"打包 + 干净环境"下暴露 ⇒ 必须真的在目标机验证。 ⇒ 沉淀进 `skill/repo-reproducibility.md` §3.2（单一入口与环境一致性）。

记录 11：把"厂商手册移出仓库"做成了假动作 —— `.gitignore` 对已跟踪文件无效

- ① **触发**：用户原话（`m00444`）：**"先做1，再做2"** —— ① 加 `LICENSE`(MIT) **+ 把厂商手册移出仓库**；② 仓库内文件名去中文。
- ② **错误假设**：我把厂商 PDF / 引脚表 `git mv` 进 `_refs/vendor/`、再往 `.gitignore` 加一行 `_refs/`，就认为"已经移出仓库、不随仓库分发"，并把这句结论写进了 `NOTICE`、`README.md` 与 `docs/DEV_STATUS.md`（第 8 条）。 ★ 我甚至**在 README 自查清单里写了正确的判据 `git ls-files "*.pdf"`，但从没跑过它。**
- ③ **暴露它的原因**：用户后来让"一起 `git add -A`"（`m01265`），我在清点暂存内容时才发现 `git ls-files _refs` **列出了 6 个厂商文件。**根因：**`.gitignore` 只影响"未被跟踪"的文件**， 这些文件自最初那个基线提交（`9e9aebc`，2026-10-09）起就一直被 git 跟踪， 所以"文件不在仓库里"这句话当时是**假的。**
- ④ **修法 + 验证**：`git rm -r --cached _refs/vendor`（**保留本地文件**，不删工作区）⇒ `git ls-files _refs` 现在是**空输出**、`git check-ignore -v` 命中 `.gitignore:31`、 两个校核脚本（`tools/pinplan_check.py`、`tools/pinout_verify.py`）仍 **RESULT: PASS**（本地资料还在，功能不受影响）。 ⚠️ **只剩一半没解决**：文件**仍在历史提交 `9e9aebc` 里**（`git ls-tree -r 9e9aebc` 能列出）， 所以**克隆仓库依然会下载到厂商手册。**本仓库**没有任何 remote**、共 103 个提交， 重写历史安全但会改掉其后全部哈希 ⇒ 我把"**推到 Gitee/GitHub 之前必须重写历史**" 写成 `NOTICE` §1.1 的显式前置条件与可照抄命令，**没有擅自执行。 ★ 2026-10-15 当晚用户拍板"重写历史"，于是已执行**：先备份 `git bundle create _refs/backup_before_filter.bundle --all`（72 MB， `.gitignore` 排除， 只留本机不随仓库分发）、再 `git filter-branch --index-filter "git rm -r --cached --ignore-unmatch _refs （厂商数据手册）.pdf 'HSTAD9238模块说明书.pdf' AD9238V11标准版原理图.pdf Davinci_Lite_1V0.pdf '双AD9238标准版引脚映射表.xls' '达芬奇Lite开发板IO引脚分配表.xlsx'" --prune-empty --tag-name-filter cat -- --all`（exit 0， **103 个提交全部重写**）。复核：`git rev-list --objects main | grep -iE 'datasheet|HSTAD9238|AD9238V11|Davinci_Lite|_refs/vendor|引脚映射表|IO引脚分配表'` = **空输出**、 ★ 判据的特征串要**足够精确**：写宽了会命中仓库里其它含 `datasheet` 字样的自建文档， 判据就永远不可能是空 —— **这条一开始我就是写错的， 跑判据时才发现**（判据本身也要被检查）。 新基线 `8f194eb`（旧 `9e9aebc`）、新 tip `6a995e9`（旧 `de59811`）、 `git ls-files` 非 ASCII = 0、`.git` **135 MB → 70 MB。**遗留一个纯本机的现象：链接工作树 `_v11_verify`（detached HEAD 停在重写前的旧提交） 让那些 blob 仍留在**本机**对象库里 —— 但这**不影响克隆结果**（克隆只取 `main` 可达对象）， 已写进 `NOTICE` §1.1 并给出 `git worktree remove --force _v11_verify` + `git gc` 的收尾办法。 重写还有一个**操作教训**：我一开始额外建了 `backup-before-filter` 分支再执行 `--all`， 结果 filter-branch **把这个备份分支本身也一起**

重写了（`--all` 连它的 ref 都算进去了）， 备份形同虚设；**真正的备份必须是仓库外的 bundle 或外部副本**， 不能是同仓库里的一个分支/ref。

- **⑤ 可复用结论**：**"移出仓库" ≠ "加 `.gitignore`"**，必须 `git rm --cached`（保留本地文件）才真的从索引移除； `.gitignore` 是给"还没被跟踪的东西"用的。 更普适的一条：**写了判据就要立刻跑判据** —— 这条错误的生命周期完全取决于我什么时候第一次 执行 `git ls-files`。⇒ 沉淀进 `skill/repo-reproducibility.md` §3.4（合规判据清单）。

记录 12：手工维护的两份清单之间没有校验 —— 汉字字模漏了两个字

- **① 触发**：用户原话（`m03752`）：**"先做②，然后做③，之后做④"** —— ② = 满足 `docs/13_display_ui_requirements.md` 的 C4（主界面中文 + 数据等宽数字）。
- **② 错误假设**：我以为"字模表"和"版面字表"只要**看起来**一致就行，于是凭人眼抄了一份 汉字清单写进 `tools/make_cn_glyphs.py`。这份清单和版面实际用到哪些字，是**两份独立的 手工维护清单**，中间没有任何校验。结果连着漏了两个字： 第一次是 `诊` （"触摸诊断"用到），补上后又漏了 `速` （"转速"用到，U+901F）。
- **③ 暴露它的原因**：我写了一个守卫 `layout uses N CJK char(s) with no glyph: ...` —— 它 **import `tools/gen_text_ram.py`、扫版面里所有字符串**，再把汉字集合与字模表求差。 第一次跑就报 `诊`，改完再跑报 `速`。**如果只靠"我再仔细看一眼"，这两次都会漏到板上**， 而屏上的表现只是一个安静的错字，别处完全发现不了。
- **④ 修法 + 验证**：守卫留在 `tools/make_cn_glyphs.py` 里（每次重生成字库都跑全量覆盖检查）， 最终 `layout CJK chars 66, all covered by 66 glyphs`。另在 `disp/tb_font_rom.v` 加了 **"任意两字不得同形"** 的判据 —— 这是"某个汉字被搬成了另一个汉字"的**唯一机器判据** （同样只在屏上表现为错字）。扩充后 110 字全部通过：非空、两两不同形、ASCII 填充位为零、 两档尺寸一致放大。
- **⑤ 可复用结论**：**凡是"两份手工清单必须保持一致"的地方，都要有一条从源数据出发的自动校验， 而且必须扫全量、不能抽查。**这条和当天另一次错误是同一类：我一度用 `Join-Path $root '..\..\' $f`（三参数、传参失败）调 `xvlog`，结果 `xvlog` **没收到任何文件、 当然零诊断**，我差点把"零 ERROR/WARNING"当成通过 —— **判据报"通过"之前，必须先证明它拿到了输入**（那次是靠"analyzing module ..."这一行才发现的）。 ⇒ 沉淀进 `skill/` 的"自查条目"惯例：任何新写的校验，都要先用一个**已知失败**的输入验一次， 确认它真的会红。

记录 13：注释里写了一个没验证过的断言 —— "二维数组会被综合成 BRAM"

- **① 触发**：做 C4 中文点阵时扩字库， `tools/gen_font_rom.py` 把字库存储体写成 `reg [15:0] mem [0:109] [0:15]` （"向量组成的二维数组"），并顺手在生成器的注释里写下 **"二维数组 -> 综合器直接推出 BRAM，像素路径上没有任何乘法/加法/偏移"**。
- **② 错误假设**：我把"Vivado 能推断 2D 存储体"当成了常识，**没有去综合日志里核对。** 真相是综合器直接拒绝： `WARNING: [Synth 8-7186] Applying attribute ram_style = "block" is ignored, object 'mem[0][0]' is not inferred as ram due to incorrect usage`。
- **③ 为什么差点溜过去**：这次的字库有 **110 x 16 x 16 = 112,640 bit**，全部被综合成 **常量逻辑（LUT）**。但 —— 仿真全绿、冒烟判据全过、屏上显示完全正确。 唯一的异常在两处**只有对着数字才会注意**的地方：Slice LUT 8,578 -> 9,053， 以及 WNS 0.701 ns -> 0.097 ns。**是资源账和时序账把问题喊出来的，不是功能判据。**
- **④ 修法 + 验证**：一维化。lab 的 `cell_h = 16`、val 的 `cell_h = 64` 都是 2 的幂， 所以 `{ch_idx, row_in_cell}` **拼接即乘法**： `wire [10:0] addr = {ch_idx, row_in_cell};` （lab） / `[11:0]` （val）， 存储体改成 `mem [0:1759]` / `mem [0:2815]`。重跑综合， `u_lab_font/bits_reg` 与 `u_val_font/bits_reg` 都变成

`implemented as a Block RAM`， `ram_style ignored` 警告归零；LUT 9,053 -> **8,494**（比加汉字之前的 8,578 还低）， 字库改用 2 个 BRAM Tile（4 -> 6），WNS 0.097 -> **0.299**。

- ⑤ **可复用结论**： 1. **资源推断必须读综合日志，不能靠经验假设** —— 尤其"综合器会帮我优化"这类期待。每次改动存储体 / 移位寄存器 / 大常量表之后，都要在日志里确认它被实现成了什么。 2. **注释里的断言必须是真的。**我把一个未验证的猜测写成了事实，后果比不写注释更糟： 它让后面读代码的人（包括我自己）以为这件事已经被核对过。 ⇒ 沉淀：注释只写**已验证的事实**，写不了就不写，或者标注成"待验证"。 3. **功能判据全绿不等于设计正确。**这一版三项判据（回归 7/7、冒烟逐字符比对、 1:1 版面预览）全部通过，缺陷却真实存在 —— 每轮实现收尾必须**同时看资源与时序四项数字** （LUT / BRAM / WNS / WHS），并把它们和上一版并排比对。

记录 14：以为要加第二个写口，其实第一个写口一直空着

- ① **触发**：做 ④（P8 底栏 `中断电平 / 翻转 / 复位次数` 三个数）时，第一反应是 "数值层格高 64 px，第 9 格行 = y576..639 已经越过 600 px 屏高，所以这三个数 **必须**写到文字层，那就得给文字层**再开一个运行期写口**"。于是打开 `tools/gen_text_ram.py`，准备改 RAM 的生成结构。
- ② **错误假设**：把"需要一块能运行期写的文字层 RAM"直接等同于 "需要**新增**一个写口 / 把 RAM 改成双写口"。
- ③ **为什么没先发现**：没有做**接口占用审计。** `tools/gen_text_ram.py` 生成的 每一页文字平面本来就是"一个写口 + 一个读口"，而 `disp/disp_engine.v` 里 `p1_lab_ram / p6_lab_ram / p8_lab_ram` 三个写口**全都接着** `1'b0` （标签/斜杠是 `initial` 静态预置的，从来没人写过）。这一行在文件里一直写着， 我读过很多遍 `disp_engine.v`，但每次看的都是读地址和字形通路，没去看写口那一行。
- ④ **修法 + 验证**： `p8_lab_ram` 的写口从 `1'b0` 改接新增的 `disp/band_view.v`； `tools/gen_text_ram.py` 的 RAM 生成结构**一行没动**，只改了 P8 底栏的静态版面 （把 `中断电平 / 翻转 / 复位次数` 拆成"标签 + 斜杠 + 留空列"，让运行期数值有地方落）。 `disp/tb_band_view.v` 把 `band_view` 的写口接到**真的** `p8_lab_ram` 逐列回读， 一轮里验证"值进了空列"且"标签没被踩掉"；回归 9/9。
- ⑤ **可复用结论**：**动手改一个结构之前，先把"现有接口的占用情况"数一遍。** 资源台账（LUT / BRAM / DSP 各占多少）这个项目一直在维护，但**接口台账** （哪个端口接了信号、哪个还悬空）从没系统看过。这次真正的代价不在 LUT， 而在于差点去动一个本来不必动的地方：改 `gen_text_ram.py` 的 RAM 生成逻辑会连带 影响三个页面、六个平面，而实际上只需要接一根线。

记录 15：测试台自己把被测信号搅浑了 —— 一个只在特定帧相位成立的"通过"

- ① **触发**：给第 2 条（导航高亮跟随页号）补回归断言。因为要遍历 P1/P6/P8 三页， 把 `disp/tb_disp_engine.v` 的仿真长度从 2 帧加到 4 帧；顺手补了一条"抽头还活着" 的检查（改 `t_angle`，抽头校验和 `tap_xor` 应当随之变化）。
- ② **现象**：nav 断言报 X（`[FAIL] nav highlight on P1: item0=XfXXXX (expect 8fc4ec)`）， 而同一条**早就存在、一直是绿**的 tap-alive 检查也翻红： `[FAIL] tap-alive block ignored the tap change (e08b2d -> e08b2d)`。 一次"只是把帧数改大"的改动，同时打红了一条新检查和一个老检查。
- ③ **根因一（激励源与检查器抢同一根线）**： `disp/tb_disp_engine.v:89-99` 有一个 `always @(posedge clk) if (frame_st)` 的后台生成器，每帧都在给 `t_angle / t_pos_full / t_sin / t_cos / t_iostate` 递推赋值 —— 它当初是为了 "让画面看起来是活的"写的。tap-alive 检查往 `t_angle` 写值，随即被它覆盖。 层次化 dump 坐实了这一点： `q_angle` 的第 7 位**确实**翻转了，但 `tap_xor` 一次变了多位的组合（因为生成器同时改了别的位），检查因此判定"抽头没响应"。 这条检查过去能过，只是因为采样恰好落在生成器更新之前：2 帧时相位凑巧，4 帧时不凑巧。
- ④ **根因二（仿真里的假 X）**： nav 采出来是 X，而 X 不是设计问题。 `tools/gen_text_ram.py` 生成 RAM 时**只** `initial` **化真正放了字的格子** （`if mem[r][c] != 0:`），空格子保持 X；空格子的 char code 一路

流过 `font_rom_lab -> lab_sr -> glyph_bit -> with_txt -> compose`，于是任何一个 "本该是背景"的像素都可能变成 X。硬件上未写的 BRAM 字会初始化为 0（= 空格）， 屏上没问题 —— 但这个 X 让**背景像素不可测**，也把真正的 X 藏在噪声里。

- ⑤ **修法 + 验证**： `tools/gen_text_ram.py` 在写字符之前先按 `cols*rows` 把整块 `mem` 显式清 0；`disp/tb_disp_engine.v` 增加 `tap_freeze`，把它接进生成器的 `if (frame_st && !tap_freeze)`，并让 tap-alive 检查改成"冻结 → 置值 → 等满一帧 → 采样 → 再置值 → 再等满一帧 → 采样"的完整时序；nav 断言改成按当前页号逐帧校验 （P1 高亮总览、P6 高亮设置、P8 不高亮任何一项）。
- ⑤ **验证**： `python tools/run_sim_disp.py` → **PASS 9/9**， `tb_disp_engine` 从红转绿（10s，errors=0）， `tb_band_view` 仍为绿。**一次改动同时修掉了 "一条假绿的老检查"和"一条 X 导致的新检查"。**
- ⑥ **可复用结论**：**只在某个特定时序相位成立的"通过"，不是通过。** 把仿真长度从 2 帧改成 4 帧，本意只是想多跑两页，结果把一条一直存在的竞态暴露出来。 这也说明"改一下测试台的运行长度/执行顺序"本身就是一种**审计手段**： 如果一个断言对运行长度敏感，它本来就不可信。 另外，**激励源和检查器必须能互斥**：任何"后台让信号动起来"的 always 块， 都要有一个开关，能在断言期间把它冻住。

记录 16：扩了检查范围，才第一次发现检查器自己跑不起来

- ① **触发**：做第 1 条 —— 给 `disp/` 补 lint 覆盖。把 `tools/lint_all.ps1` 的目标 从 `rtl\*.v + sim\tb_*.v` 扩到再加 `disp\*.v + disp\sim\*.v`。
- ② **现象**：脚本一行不跑，而且**换了三次错**：先报"字符串缺少终止符" （ParserError，位置指向 `Write-Host (" 合计: ERROR {0} / WARNING {1}" -f ...)` —— 那一行本身完全正确）； 补回 UTF-8 BOM 后变成 `NotSupportedException / DotSourceNotSupported`； 去掉 `[CmdletBinding()]` 之后 又撞上 `.NET 静态方法在受限语言模式下被拒`。
- ③ **根因（三个坑叠在一起，都不是显示代码的问题）**：
- **BOM**： `tools/lint_all.ps1` 是全仓库唯一一个**没有 UTF-8 BOM** 的 `.ps1` （其余 7 个都有）。Windows PowerShell 5.1 读无 BOM 的 `.ps1` 会按 ANSI/GBK 解码，文件里的中文注释被误解码，某个汉字的第二字节恰好是 `0x22` （ `"` ）， 于是"字符串缺少终止符"。**报错行号会指向一个完全正确的行。**
- **`[CmdletBinding()]`**： 加上它脚本就成了"高级脚本"，在受限语言模式下被调用时 PowerShell 会尝试把它点源进调用者作用域，而跨语言模式点源是禁止的。
- **`.NET` 静态方法**： `[System.IO.Path]::GetFileNameWithoutExtension()` 被拒（ `MethodInvocationNotSupportedInConstrainedLanguage` ）； 同样被拒的还有 文件里原本就有的 `New-Object System.Text.UTF8Encoding($true)` （ `CannotCreateTypeConstrainedLanguage` ）。
- ④ **修法**：补 BOM；去掉 `[CmdletBinding()]` （本脚本只用到一个 `[switch]` 参数， 普通 `param` 完全等价）； `New-Object` 包 `try/catch`，降级为"中文显示乱码但不中止"； `GetFileNameWithoutExtension` 换成纯字符串替换 `$rel -replace '\.[^.]*$',''`。
- ④**b 同一个坑在第二个脚本上又踩了一次**：修完 lint 去重建位流， `tools/run_vivado.ps1` 报同样的 `DotSourceNotSupported`。它 `param` 里写的是 `[Parameter(Mandatory = $true)][string] $Tcl` —— **只要出现 `[Parameter()]`， 脚本同样会被当成高级脚本**；去掉它、改成手工判空 `if (-not $Tcl) { throw ... }` 就正常了。顺带发现它在末尾还用了 `[string]::IsNullOrWhiteSpace($LogFile)`， 同样是受限模式下的禁区，换成 `-not $LogFile`。 也就是说这不是某一个脚本的问题，而是**整个 `tools/` 目录对受限语言模式不设防。**
- ④**c 而且 BOM 是"编辑一次丢一次"**：改完 `run_vivado.ps1` 之后 BOM 又没了 —— 仓库里所有会重写文本文件的环节（格式化、追加章节、生成文档，以及本次的编辑动作） 都可能把 BOM 丢掉，而丢掉之后故障现象是"报错指向一个正确的行"。 于是新增 `tools/fix_ps1_bom.py`： 检查 `tools/*.ps1` 的 BOM， `--fix` 可补齐， 退出码 1 表示有缺失，可以直接挂进提交前检查。 **把一个隐形故障变成一条可自动检查的规则，比在下一次踩坑时重新排查一遍便宜得多。**

- ⑤ **验证**：脚本跑通，**66 个文件全部 `[ok]`，ERROR 0 / WARNING 0**， 且输出里第一次出现了 `disp/` 的文件（`band_view.v / disp_engine.v / text_ram.v / tb_band_view.v / tb_disp_engine.v`）。**这次"ERROR 0"才第一次真的包含显示代码。**
- ⑥ **可复用结论**：**一个检查器报"通过"，只有在你能证明它会对该范围内的错误报 "失败"时才算数。** 这个脚本过去报的 `ERROR 0 / WARNING 0` 从来没有包含过 `disp/` 下的 9 个模块 —— 而它"跑不起来"的表现只是退出码非 0，很容易被上层忽略。 扩展检查器的**第一步应该是塞一个已知错误进去看它报不报**， 而不是直接接受它给出的绿。这一条和第 1 条的目标正好互为镜像： 我们想补的是"被检查的范围"，先暴露出来的却是"检查器本身"。

记录 17：规范写着"不设计按键兜底"，我差点在触摸没上过板的时候真删了

- ① **触发**：第 3 条（触摸命中判定）做完。按 `docs/13_display_ui_requirements.md` 的 C3「触摸修好了就纯触屏，不设计按键兜底」，下一步应当把 ③ 的旧入口 —— **长按板载 KEY1 一秒 = 改倍频档** —— 整段删掉，只留"点 P1「倍频分频」卡"。
- ② **删之前核了一遍证据链，发现不对等**：触摸的**几何判定**只在 `disp/tb_hit_test.v` 里验过（51 条断言，覆盖边界、不连发、越界坐标）， **没有任何一次上板记录**；而被删的旧入口是上过板、验过的（U-9 归档）。 把验过的删掉、换成没验过的，等于拿一次"合规"换掉一次"可用"。 更具体地说：如果实机上触摸的 Y 轴是反的（这种事在触摸 IC 上很常见）， 删掉兜底之后这一版就连"改倍频"这件事都做不成了。
- ③ **处理：不删，改成双入口，并把偏差和它的解除条件一起写进代码。** `disp/enc_top_disp.v` 的 4.7 节开头写明：本版触发有两条，①点卡片（主入口）、 ②长按 KEY1（上板兜底）；并写清"等触摸在实机上确认命中之后，删掉 ② （约 25 行：`lp_cnt` / `lp_armed` / 那个 `else if` 分支）就兑现 C3"。 同一句也写进测试单 `docs/15_display_touch_bringup.md`，让它出现在上板核对表里。
- ④ **可复用结论**：**规范与现状冲突时，正确的产物不是二选一，而是一条 "带解除条件的偏差记录"。** 默默违反规范，下一个人会以为 C3 已经兑现； 默默执行规范，这一版上板时会把已经验过的能力弄丢。 把"为什么暂时不兑现 + 兑现的确切条件"同时写进代码注释和测试单，两者才都不丢。

记录 18：越界坐标会被截断成"合法"坐标

- ① **触发**：给 `disp/hit_test.v` 写坐标判定之前，先回头核了一遍上游能给什么值。
- ② **事实**： `disp/touch_ctrl.v` 从触摸 IC 读回来的是 **16 位** `tch_x0` / `tch_y0`， 而显示坐标只有 12 位（`pix_x` 最大 1023、 `pix_y` 最大 599）。 如果直接取低 12 位去比， `x0 = 2048` 会变成 `0`、 `y0 = 4096` 也会变成 `0` —— **一个明显非法的坐标被截断成屏幕左上角，正好落在导航栏第一项上。** 于是"噪声/误码产生的一个坏值"会表现成"用户点了总览"。
- ③ **修法**：判定之前先过一道闸，不满足就报"没碰任何控件"： `wire xy_ok = (tch_x0 < 16'd1024) && (tch_y0 < 16'd600);` （`hit_test.v` 里所有矩形判定都与它相与，落空时输出控件编号 31 = `HIT_NONE`。事件本身照报 —— **"有接触但落在屏外"和"根本没有接触"是两件事**， 合并成一件会让上板排障时少一个判据。）
- ④ **验证**： `disp/tb_hit_test.v` 第 7 节专门喂坏值，五个全部要求 `id=31`： `(1024,100)` `(2048,100)` `(500,600)` `(500,4096)` `(65535,65535)`； 并且与"屏内但不在任何控件上"的 `(1023,599)` 分开断言，两组不能互相掩盖。
- ⑤ **可复用结论**：**上下游位宽不同时，先问"上游的合法值域是不是下游的值域"。** 截断在 Verilog 里是**静默**的：不报警、不报错、综合后时序全绿， 只在某个坏值到来的那一拍，表现得像是用户点了屏幕左上角。

记录 19：注释里引了一份不存在的 SPEC 章节

- ① **触发**：P6 要显示 9 个设置项，v13 的抽头只够 3 个，得扩 `disp_tap`。 扩之前按 `rtl/enc_top.v` 里那句"字段定义见 SPEC §1.14"去翻 `rtl/SPEC_interface_contract.md`，准备照着它改契约。
- ② **事实**：那份文档里**根本没有 §1.14** —— 章节只有 0.x、1.1–1.13、2、3、4； 而且**从头到尾没写过 `disp_tap` 一个字**。也就是说：134 位的位序契约只存在于 `rtl/enc_top.v` 的一段注释里，而那段注释指向一个不存在的章节。
- ③ **为什么一直没人发现**：这条引用只在"有人真的要改它"的时候才被用上， 而 v13 以来没人改过。写注释的人以为那边有文档，读注释的人以为那边有文档 —— **两边都以为对方在维护，于是谁也没去看。**
- ④ **修法**：这次不只改引用，而是把契约搬到它该在的地方。 `rtl/SPEC_interface_contract.md` 新增 §2.2「实验性只读抽头 disp_tap」， 把 183 位逐字段列全（含 v14 六项），并写三条硬规矩：① `ENC_DISP_TAP` 永远不许出现在生产构建脚本里（唯一合法使用者 `tools/synth_disp.tcl`）； ②不定义宏时端口与赋值都不存在，生产构建与 v13 逐字一致； ③只读快照，右边只许放**已在寄存器里的既有信号**，绝不放 `gap_num` 一类组合中间量。 同时把 §2 拆成 §2.1「生产端口」与 §2.2「抽头」，让"生产接口"和"实验接口"不再是同一节。
- ⑤ **可复用结论**：**文档引用是可验证的事实，不是装饰。** 一条"见 §X"如果从没被 打开过，它和一句空注释没有区别；而它造成的最坏后果不是"信息缺失"，是 **"以为信息存在"** —— 于是没人去补。写引用的那一刻就翻开确认一次， 否则宁可不写这句引用。

记录 20：那块 RAM 的写口一直是空的，但屏上的数字恰好是对的

- ① **触发**：给 P6 做"按页字段表实时回读"（屏上 9 个设置项要显示寄存器当前值）。
- ② **事实**： `disp/disp_engine.v` 里 `p6_val_ram` 的写口接的是 `1'b0`。 P6 那 9 个数字来自 `tools/gen_text_ram.py` 的 `initial` 静态预置， 而静态预置写的**正好是编码器复位默认值**（`rtl/reg_file.v:255-278`）： inter=0→256、div=0→1、pairs=4、hyst=0、ablimit=0、zwidth=4、zphase=0、uvwph=0、pol 全 0。 于是上电后不改任何设置时，屏上显示的**恰好就是真值。**
- ③ **为什么长期没被发现**：判据一直是"数字看着对不对"，而 **"对"和"通路存在"是两件互相独立的事。**写口空着不影响复位后第一眼的正确性， 只影响"改过设置之后"——而那正是没人专门去验的场景。上一版的注释甚至把 "为什么不接这个写口"写得明明白白，读的人（包括我）就默认了"所以 P6 的显示是没问题的"。
- ④ **修法**：新增 `disp/p6_view.v`（按 P6 自己的字段表重写数值层）， 写口从 `1'b0` 改成 `p6v_we & (page == 2'd1)`； `disp_tap` 补 6 个字段； 测试台 `disp/tb_p6_view.v` 例化**真正那块** `p6_val_ram` 逐格回读， 四轮 45 条断言（含"写口覆盖面恰好等于 9 个字段矩形"这一条）。
- ⑤ **副产物：新口径立刻顶出一处旧的不一致。** 静态预置里写的是 `X 64`， 而 `rtl/reg_file.v:261` 的复位默认值是 `r_inter0 <= 8'h00`，旁边注释写着 `INTER = 0 -> inter = 256`。换句话说，那个"一直看着对"的 64 从来就不是真值 —— 测试台第一轮就把它钉成了 `X256`。
- ⑥ **可复用结论**：**凡是"没有写口也显示得出来"的数据，都要追问一句它从哪来。** "静态预置值"恰好等于"复位默认值"时，一条根本不存在的通路会被长期伪装成"通了"； 而它暴露的时机不是上电，是**第一次有人改设置** —— 也就是最不适合排障的时刻。

记录 21：页面编号按"我做的顺序"编，导航按"用户看的顺序"排 —— 于是多了一张表

- **现象**： `disp/disp_engine.v` 里曾经有这么一段，专门用来算"当前页该高亮哪个导航项"：

```
verilog wire [2:0] nav_page = (page == 2'd0) ? 3'd0 : (page == 2'd1) ? 3'd5 : 3'd7;
```

它是 ⑤ 那一轮修"导航高亮不跟页号"时加的，当时看着很自然。 - **根因**：页面编号（`page`）是按**我实现的顺序**编的 —— 0=P1 总览、1=P6 设置、2=P8 诊断； 而左侧导航的六项是按**用户看的顺序**排的 —— 0 首

页、1 波形、2 XY、3 时序、4 状态、5 设置。 两套顺序互不相同，于是"页号 → 导航项"只能靠一张手写映射表硬接。 - **它是怎么被顶出来的**：这一轮要加 P2 波形页。P2 的入口按 spec §4.4 是**导航第 1 项**， 而导航第 1 项在任何旧编号里都不存在对应页面。补一个 `3'd3`？ 那张表就变成了 `(page==0)?0:(page==1)?5:(page==3)?1:7` —— 一张纯粹记录"历史实现顺序"的表。 - **修法**：把页面编号改成 spec §4.4 的页面顺序（0=P1 总览、1=P2 波形、2=P3 XY、3=P4 时序、 4=P5 状态、5=P6 设置、6=P7 标定、7=P8 系统信息）。于是"导航第 k 项 == 页号 k"， 那张表**整段删掉**， `nav_sel` 直接比较 `nav_idx == page`。 多花的代价只有一件事：改一下 `page` 的位宽（2→3）和几处字面量。 - **可复用结论**：**能用编号顺序表达的关系，不要用查表表达。** 查表把"顺序"这个信息从结构里搬进了数据里 —— 顺序本身不会出错，抄进表里的顺序会。而且查表的分支数会随页数增长，顺序不会。 判据很简单：**如果一张表的每一行都是"旧编号 → 新编号"，那它映射的不是业务，是我的历史。**

记录 22：单缓冲的波形会在屏幕中间接缝 —— 而"写得比扫得快"救不了它

- **现象**（设计阶段就抓到，没写进 RTL）：扫描轨迹采样器每秒要往一块存储里写 944 个"列 → 行区间"，光栅扫描器同时要按自己的顺序把这块存储读出来画。第一版我写成了单缓冲： 采样器写 `mem[wr_ptr]`，光栅读 `mem[{disp_buf, pix_x[9:0] - 80}]`，同一个地址空间。
- **为什么必然撕裂**：设每列花 `SWEEP_DIV ≈ 128` 拍，采样器在 `第 x*128` 拍左右重写第 x 列； 而光栅到达"第 r 行第 x 列"是在第 `r*1024 + x` 拍。
- 第 0 行： `x < x*128`，光栅**先到**，读到的是上一帧的内容；
- 第 199 行： `199*1024 + x = 203776 + x > x*128`，光栅**后到**，读到的是这一帧的内容。 同一帧里左边几行是旧数据、下面几行是新数据，接缝还会随帧移动。屏幕上看起来就是波形被横着切开。
- **"写得比扫得快"不是解法**：一帧 853,440 拍，一次完整扫描只要 944*128 ≈ 121,600 拍， 采样器整体上快得多。但问题不在吞吐，在顺序* —— 对同一个地址， 读者在写者到达它之前和之后各读到一次，就一定会撕裂。
- **修法**：双缓冲。两块 944 列的存储面，采样器只写"写面"，光栅只读"显示面"， 换面只发生在帧首（那时没有像素在取数）。地址最高位用来选面： `{wr_buf, wr_ptr[9:0]}`。 代价是 944 字 × 32 bit × 2 = 59 Kbit ≈ 2 个 RAMB36（ `disp/wave_ram.v` 按 2048×32 实例化）。
- **意外的副产物（正面的）**：因为读者和写者被彻底隔开，测试台可以**直接往"显示面"写期望数据、 再把 `disp_buf` 强置成那一面**，而不用去关掉被测逻辑。测试因此变得更容易写、也更接近真机行为。
- **可复用结论**：**只要同一地址在"读者到达它之前"和"之后"都会被写一次，就必须假设它会撕裂。** 这类结构的必要条件是把"进行中的写"和"正在被读的那一份"物理分开； 常见的错误是把"写得更快"当成隔离。

记录 23：双缓冲的两个指针在复位时是同一个 —— 我把"还没换过面"当成了"已经分开"

- **现象**： `disp/xy_plot.v` 的测试台第一次跑就报 `[FAIL] 复位后不进清零相（没有任何一次写）: got 9493 want 0` —— 复位之后、第一次帧首之前，描点器已经往点阵里写了 9493 次。
- **根因**：模块是"一帧两相"（擦整面 → 打点），两相都由帧首启动。但第三支分支（打点）不受任何状态约束： `clr_phase` 复位成 0 时， `else if (div == SAMPLE_DIV-1)` 那一支每 4 拍就置一次位。 第一版更糟：我把 `clr_phase` 复位成 1，于是复位后先花 65536 拍擦面、再往**同一个面**上打点。
- **为什么必然出错**：双缓冲的不变量是"写面 ≠ 显示面"，而这条不变量在复位那一拍**必然被违反**： `wr_buf` 和 `disp_buf` 的复位值只能是同一个（没有"上一帧"可继承），而把两个指针分开的唯一动作是换面，换面只发生在帧首。 所以"复位后立刻开始写"和"写面不等于显示面"这两句话不可能同时成立。
- **修法**：加一个显式的生命周期状态 `reg started`（复位 0、帧首置 1），在 `!started` 分支里**一次都不写。** 代价是上电后第一帧屏上是空点阵（17 ms）； 而点阵存储的 `initial` 铺的就是 0，所以那一帧是"空"而不是"错"。

- **可复用结论**：**凡是有"两份、换着用"的结构，复位状态一定是"两份还没被区分开"的那一刻。** "什么时候允许开始写"必须是一个显式状态，不能是"复位之后"。

记录 24：端口连接引入的隐式网线，声明位置取决于那一条例化语句

- **现象**：`xvlog` 直接报 `ERROR: [VRFC 10-3380] identifier 'xy_disp_buf' is used before its declaration [disp/disp_engine.v:481]`， 测试台一行都没跑。
- **根因**：`xy_raddr = {xy_disp_buf, xy_by, xy_bx}` 写在 `xy_plot` 例化**之前**。 Verilog-2001 允许"端口连接里出现的名字自动成为一条隐式网线"， 但那条隐式声明的**生效位置是例化语句本身**，不是模块开头。
- **修法**：把 `xy_we / xy_waddr / xy_wdata / xy_disp_buf` 四个 `wire` 显式声明到 `xy_raddr` 之前。
- **可复用结论**：**隐式声明不是"在整个模块里都算数"，它算数的地方是那一条例化语句。** 同一份代码里既有显式声明又有隐式声明时，顺序会悄悄变成语义。

记录 25：五条失败里，两条是我自己算错的

- **现象**：`disp/tb_p3_page.v` 第一版跑出 5 条 `[FAIL]`，混在同一份清单里、长得一模一样：
- 3 条关于"擦面拍了多少拍、第一次擦的是哪一面"（真错，见记录 23）；
- 2 条关于坐标映射：我断言 `{by,bx}` 应该等于 `16'h6096`，而 96 = 0x60，两个都是 96 的拼接只能是 `16'h6060`； 我又断言 `c = -2560 → by = 86`，而 `by = 96 - (c>>>8) = 96 - (-10) = 106`。
- **为什么这两条看起来一模一样**：断言失败只告诉你"被测值和期望值不同"， 它**不告诉你该怀疑哪一边**。这一轮里两条是 RTL 错、两条是我算错、一条是声明顺序错，三种原因共用同一种输出格式。
- **怎么分辨**：算错的那两条都能在一行纸上重新推一遍（96 = 0x60；`96 - (-10) = 106`）， 而且它们落在"纯组合映射"这一组；真错的那三条都跟**时间 / 状态**有关（复位后第几拍开始写、擦面持续多少拍）。所以先按"这条断言涉不涉及时间"分组，再看哪一组更需要怀疑。
- **修法**：改掉两个常数；并把非对称样本 `(2560, -5120)` 换进去，让 `bx` 和 `by` 取到 106 / 116 两个不同的值 —— 原来用 `(2560, -2560)` 时两者都是 106，一个常数写错也不会暴露。
- **可复用结论**：**断言失败时，先问"这条断言自己算对了吗"，再去看被测件。** 一个检验另一个的前提是它自己可信；而"期望值"往往是我手算的，是整个测试台里唯一没被任何东西检验过的部分。 顺手把期望值取成互不相同的数，能让"哪个数错了"本身变得可读。

记录 26：把坐标停在一点上采样，测不出管线级数

P2 的波形页和 P3 的李萨如页各写了一张测试台，里面都有一条"列对齐"断言：预置 位图的第 100 列，然后在屏上找它落在哪一列，期望 x = 180（P2）/ 420（P3）。 两张台子都是绿的 —— 绿了整整两轮。

写 P4 的时候被迫把"补几级寄存"重新推了一遍：数据通路的读地址用的是**未延迟** 的 `pix_x`，而合成链里的坐标是延迟两拍的 `px2`，所以"数据比坐标早一格"，补 **一级**正好对齐。按这条推导回头看 P2/P3，它们各补了**两级**。

于是写了一张只用连续光栅的测试台（`disp/tb_pipe_align.v`）： `pix_x` 每拍 +1 走满一行，只在 `de_o && dbg_y == 目标行` 时读 `lcd_rgb`。结果 P2 的落点是 **181**、P3 是 **421** —— 各右移 1 px，与推导一致。

为什么原来那两张台子测不出来：`smp()` 把 `pix_x` 停在一点上、等 5 拍再读。 坐标停住之后 `wave_q / wave_d1 / wave_d2` 全部收敛到同一个值，**几级寄存都 一样**。它们实际测的是"位图第 100 列 ↔ 屏上 x=180"这个**组合映射**，而这个 映射本来就是对的 —— 错的是"它在第几拍被算出来"。级数是**时间**属性，停在 一点上采样就把时间维消掉了。

判据：任何"停住输入再观察输出"的检查，都测不出与时间有关的缺陷。要测时序， 输入就必须在动。

记录 27：栅格化只判了行，没判列

P4 时序页里，高/低轨与"箱首竖直边沿"原本只用 `lg_rband` 判定，而它只看行 （`cy` 是否落在轨迹带内）不看列。于是第 0 行的高轨从 `x = 212` 就开始画 —— 那是外框的左沿，一路画进了左边的标签栏，而且把标签色块也盖住了。

这个错是"移动光栅"测试台顺带暴露的：本来只想量高轨的起点（期望 500），结果 拿到 212，等于把框左沿的坐标也一起读了出来。修法是把轨迹单独门控一次 （`lg_tband = lg_rband && (cx >= LG_TX0)`），标签色块继续用 `lg_rband`。

判据：给"某一片区域"写渲染条件时，行条件和列条件要分别写出来、并各自检查 一遍。只判行不判列，在几何上就等价于"这一行整条都要画"。

记录 28：声明了、用上了、但没有驱动源 —— 一页的像素全成了 X

`disp/tb_p5_page.v` 第一次跑，P5 页每一个采样点都是 `got XXXXXX`；而"跑满帧 / 查帧率 / 查色板"这类检查一条都没红。在测试台里层次引用打印了内部信号，得到 `val_ch=z`（高阻）—— 一个"声明了、也参与了选择、但没有任何驱动源"的 wire。

`disp/disp_engine.v` 的 val 平面选择器写成了四档

```
wire [5:0] val_ch_raw = (page == 3'd5) ? val_ch_p6 :
                        (page == 3'd7) ? val_ch_p8 :
                        (page == 3'd4) ? val_ch_p5 : val_ch_p1;
```

`val_ch_p5` 也声明了，但 **`p5_val_ram` 从来没有例化** —— 少写了一条 `.raddr/.rdata` 连接。Verilog 里没人驱动的 wire 是 z，z 进字形通路 （`font_rom_val -> val_bit`）就变 X，而 `with_txt = glyph_bit ? txt_col : ...` 的条件含 X ⇒ 整页像素都是 X。

判据：mux 的分支数等于"页数"时，必须逐页核对每个分支都有驱动源。 "整页采样都是 X"这种症状，第一件事是层次引用打印那个 mux 的输出 —— z 是它的指纹。

记录 29：测试台自己把"正在跑的那一轮"丢掉了

小参数那块 `p5_view` 的 8 个字段里，只有**最后一个**（`BAD 000`）渲对了 （`90.0`），前两个渲成 `0.0`，条形阈值也全是 0。

根因：`pv_frame()` 只等 2 个 negedge 就发下一次 `start`，而一轮渲染要约 450 拍 （8 个字段 x（40 拍除法 + 10 拍双 dabble + 5 拍写））。`p5_view` **只在 S_IDLE 认** `start` ⇒ 第二个脉冲被丢掉，而**正在跑的那一轮**继续读 `snap`：已经渲完的 字段用的是旧值，还没渲到的用新值。`字段 0/1 = 0.0、字段 6 = 90.0` 就是这个 "新旧混合"的指纹。修法：`pv_frame()` 先等 `!busy`。

判据：由"帧首脉冲"驱动的 FSM，测试台必须自己保证上一轮已经空闲。 把 `start` 当成"可以随时拍一下"的信号，是把握手当电平用。

记录 30：顶层那块渲染器从来没跑 —— 因为测试台不跑真光栅

force 之前，顶层 `disp_engine` 里那块 `p5_view` 的阈值一直是复位值 0， 百分比列一个字都没有。根因是 `disp/disp_engine.v` 的

```
wire last_px  = de && (pix_y == 12'd599) && (pix_x == 12'd1023);
wire vf_start = last_px_d;
```

它只认**真实光栅的最后一个可见像素。**测试台为了省时间只驱动若干采样窗口， 从不到 (1023, 599) ⇒ 顶层所有"帧驱动"的逻辑（`val_format` / `touch_view` / `p5_view` / `wave_trace` / `xy_plot`）一次都没启动。

处理办法是**分工**：渲染逻辑用小参数的那一块单独验（`p5_view #(.WIN_FRAMES(3))` 配真的 `p5_lab_ram`），顶层的像素映射用 force 阈值 + 预置显示面来验 —— 两个测试台各自只测一件事。

判据：依赖"整帧驱动"信号的断言，如果测试台不跑真光栅，那它测的其实是 "测试台有没有跑真光栅"，而不是被测逻辑。

记录 31：少算两拍流水 —— 整条曲线右移一箱，而"箱箱相同"的测试数据看不出来

P7 的曲线 dump 相要把累积 RAM 里 128 个箱的 `{hi, lo}` 换算成屏上行号再写进显示 RAM。原码是

```
if (dp != 8'd128) acc_raddr <= dp[6:0];
aft_waddr <= dp[6:0] - 7'd1;
aft_wdata <= {hi_y, lo_y};
```

它的隐含假设是"这一拍读到的 `acc_rdata` 就是第 `dp-1` 箱"。但 RAM 的读是**两拍**： 第 `dp` 拍发地址、第 `dp+1` 拍才把数据打进 `rdata`（`rdata <= mem[raddr]` 是非阻塞）， 所以第 `dp` 拍手上拿到的是**第 `dp-2` 箱**的数据。写地址因此必须整体后移两拍： `aft_waddr <= dp - 2`，循环终点也由 128 挪到 129（最后一拍写成第 127 箱）。

指纹出现在测试台里：我给第 40 箱预置了 `{hi=0x9000, lo=0x8000}`（换算后应是 `{160, 128}`），结果 `aft_ram` 的**第 40 箱**是 `{0, 239}` —— 那是"空箱 `{hi=0, lo=0xFFFF}`"换算出来的值。**写进第 40 箱的，是第 38 箱的数据。**

判据：凡"发地址 / 取数据 / 写出去"跨拍的流水，必须把三者的拍数在注释里写成 一个等式（这里是 `数据(d) = mem[d-2]`）；只要测试数据在箱与箱之间完全相同， **这类右移就永远测不出来 —— 预置数据必须逐箱不同。**

记录 32：`@(posedge clk)` 之后立刻读层次信号，读到的是跳变前的旧值

同一轮里还有两条断言全红：

```
[FAIL] acc_ram 箱 5 写出 = {hi=30, lo=10} : got 0000ffff want 001e000a
```

而 `[dbg]` 监视器明明白白打印了 `acc WR addr=5 data=001e000a`（写确实发生了、 地址数据都对）。差别只在**读的时机**：测试台的任务写成

```
pv_ang = ang; pv_amp = amp;
repeat (n) @(posedge clk);      // 任务在这里返回
... 立刻 chk32(u_acc.mem[5], ...)
```

非阻塞赋值在 NBA 区生效，而 `@(posedge clk)` 唤醒的是 active 区的过程块 —— 任务返回时，那一拍的 NBA 更新**还没执行**， `mem[5]` 还是旧值。修法就是每个 采样任务末尾加一句 `#1`。

判据：测试台里"等沿 → 立刻读寄存器/RAM"是一个固定坑。驱动信号的沿（时钟） 与"值被更新"的时刻之间隔着一个 NBA 区，要么 `#1`，要么改用 `@(negedge clk)` 采样。 （这与记录 25"断言红了先问断言自己是不是对的"是同一类问题的两种表现：一次是期望值 算错，一次是**读取时刻**选错。）

记录 33：版本相关的开关靠猜，连撞三次才去读 `-help`

为了回答"WNS 能不能在不改 `rtl/` 的前提下推到 +1 ns"，我新写了一个只动综合开关的实验 脚本，然后连撞三次墙：

```
synth_design ... -retiming on -directive PerformanceOptimized
ERROR: [Common 17-165] Too many positional options when parsing 'PerformanceOptimized'

synth_design ... -retiming on
ERROR: [Common 17-165] Too many positional options when parsing 'on'

place_design -directive AggressiveExplore
ERROR: [Constraints 18-641] Directive 'AggressiveExplore' is not a recognized directive
```

第一次的报错点名的是 `PerformanceOptimized`，于是我先去掉了 `-directive` —— 方向完全错了： **真正的原因是 `-retiming` 在这个版本里是个不带值的开关**，`on` 才是那个多出来的位置参数 （解析器报的是它最后遇见的那个）。最后把 `synth_design -help` 整个落到 `_disp_work/synth_help.txt` 再读，一行就看清了：

```
-retiming          (Optional) For non-Versal devices who do not have retiming ...
-global_retiming   Values: auto, on, off.
-directive <arg>   Valid values are: ... PerformanceOptimized ...
```

而 `place_design` 的合法指令名根本不用查文档 —— 仓库里 `tools/impl_disp_opt.tcl:68-71` 就是一份**已经跑通过的**名字表（`ExtraTimingOpt` / `AggressiveExplore` / `Explore`）。

判据：① 工具链的开关拼写属于"版本事实"，不是常识；动手前先把 `-help` 落盘再读， 一次就够。② 报错里点名的那个词往往只是"最后一个位置参数"，不一定是原因本身 —— 先问"多出来的是谁"。③ 同一个仓库里已经有同类脚本跑通时，先抄它的参数，再谈自己拼。

记录 34：4:1 缩略图按"覆盖"抽样，把 16x16 汉字整批漏掉

P7 版面的第一次"看一眼"，缩略图里 ASCII 标签（`OFS-S`、`GAIN-C`）都在，汉字一个也看不见。 我当时的第一个怀疑是字库：以为 `tools/make_cn_glyphs.py` 生成的字形是空的。真实原因在 采样那一行：

```
img[dbg_y >> 2][dbg_x >> 2] <= (lcd_rgb === C_TEXT);
```

非阻塞赋值让同一格里的多次写在 NBA 区按顺序生效、后者覆盖前者，等价于"每个 4x4 块里 只保留 `x%4==3 && y%4==3` 的那一个像素"。16x16 汉字（含 `disp/font_rom.v` 这一版用 SimHei 15 px 光栅化的结果）笔画只有 1~2 px 宽，正好整根落在被丢掉的 12 个采样点上； 而 ASCII 标签笔画更密，总有像素撞进那个亚网格。

修法有两步：① 冒烟件里给 `PAGE7` 加两个 **1:1 窗口**（`WA_*` / `WB_*`）直接看图， "汉字有没有、写得对不对"由它回答；② 与像素无关的"格坐标版式"另写 `_disp_work/show_p7_layout.py`，直接 import

`tools/gen_text_ram.py` 打印 lab 平面的 行列占用 —— 它比任何缩略图都权威。

判据：缩小的采样必须做"或"（累积），不能做"覆盖"；反过来说，"缩略图里看不见"永远 不能推断成"光栅里没有" —— 先怀疑采样，再怀疑数据。（与记录 25"断言红了先问断言自己 是不是对的"同源：先怀疑自己的量具，再怀疑被量的东西。）

记录 35：测试台里写死的"卡片中间一点"，网格一移就把 FAIL 指向了错误的模块

改版面（三列网格 32/352/672 -> 96/416/736）之后的第一轮回归是 14/16，其中一条是：

```
[FAIL] 抬起后再按未触发（stb=1 id=31，期望 id=12）
```

`id=31` 是 `HIT_NONE`。读这句话的第一反应是"命中判定坏了"——因为期望值 `id=12` 指的是 **卡片 11**，而测试台用的坐标是 `(400,400)`：旧网格里卡片 11 占 x352..639，400 稳稳在 里面；新网格里卡片 11 占 x416..703，而 **400 落进了卡片 1 与新卡片 1 之间那条 16 px 内边距**（`x∈[384,416)`），那里本来就不该命中。

也就是说：**被测逻辑是对的，坏掉的是"这个坐标代表卡片 11"这条隐含前提。** 测试台把 "卡片内部的一点"写成了一个看起来在中间的数字，而不是从卡片几何推出来，于是几何一改， 失败信息里的 `id=31` 会把人引去查命中判定，而不是查坐标表。

修法不只是把 `400` 改成 `480`，还顺手把这条前提本身变成了断言（`disp/tb_hit_test.v` 新增第 4 节"卡片左沿与导航列之间的 16 px 内边距必须是空命中"）：

```
chk_miss(88,  100);   // 内边距里
chk_miss(95,  319);   // 内边距右下角
chk_tap (79,  100, 5'd0);   // 导航栏最后 1 px 仍然是导航
chk_tap (96,  100, 5'd8);   // 卡片左沿第一列就是卡片
```

判据：测试台里的坐标要么由几何常量推出来，要么同时断言"它落在哪个矩形内"。 一个"看起来在中间"的魔数，在布局变动时产生的失败信息会把排查引向错误的模块。

记录 36：预览件照 spec 手抄了一份几何，于是它画的 P1 比真板子"干净"

发现这轮瑕疵的经过值得记下来。`tools/preview_pages.py` 一直按 spec §4.3 硬编码画卡片：`300×264 @ 92/404/716`，而且 `card_rows = 3 if page in ("p6", "p8") else 2`。但 RTL 早就 不是这样了 —— `disp/disp_engine.v:798-802` 有一条明确的注释：**卡片框只在 P1 画**（P6 用 spec §4.4 的"3 列 x 4 项网格"，项在格行 3/15/27，与 y=64/336 的卡片框整段错开）。更糟的 是绘制顺序：预览件先画卡片、后画导航栏（"导航在卡片之上"），而 `bg_sel` 里 `in_card` **优先于** `in_nav`（卡片压导航）。两个错误叠在一起的效果是：`_disp_work/page_p1.png` 里导航栏干干净净、卡片规规矩矩 —— **真板子上卡片正压在导航栏右半上。**

真正把这个缺陷钉住的不是这张图，而是**算数**：改网格时顺手算了 P1 字段的右沿（`disp/val_format.v:104-109 f_wid = 10/7/8/7/8/8`、字段 0 是 10 个数值格 = 320 px）， 发现它比卡片宽 288 px 多出 32 px；再用真光栅（`_disp_work/smoke_p1.log` 的缩略图里 笔画落在 x164..412、跨过卡片右沿 x319）确认了一次。

判据：量具（预览件、抽样窗口、检查脚本）的几何要么从 DUT 的常量/数据里取， 要么与 DUT 在同一次提交里一起改 —— 凭文档手抄一份，等于给自己留一个"看起来没问题"。 并且"看图"和"算数"要互相校验：这次的越界是算出来的，图反而骗人（与记录 34 同源， 那一条骗人的是采样规则，这一条骗人的是几何来源）。

记录 37：同一份"两处契约"，我按像素写法扫了陈旧值，它在格子写法下活了下来

U-20 把三列网格从 `32/352/672` 挪到 `96/416/736` 之后，我扫过一遍测试单里的陈旧字面量， 用的词表是 `32…79|x = 32|352|672|404|716|x 92|x 96|416|736` —— 全是**像素**写法， 命中两处，都处理了，于是我认为扫干净了。

烧完位流、准备照着测试单做上板验收时重读了 P6 那一节，才发现第 171 行的注②还在说：

> ……标签在 `tools/gen_text_ram.py` 的 `P6_LAB`（格行 3/15/27、格列 4/24/44），
> 数值列在 `disp/p6_view.v` 的字段表（格行 1/4/7、格列 2/12/22）……

`4/24/44` 与 `2/12/22` 是**旧值**。这条注释正是"两处契约"的说明（标签与数字分属两个模块）， 而它描述契约时用的是**格子**词表、不是像素词表，所以我的扫描词表里一个都没有它们。

危害不在于"注释旧了"，而在于这条注释的**用途**：它教用户在看到"数字整体偏一格"时怎么判断 是哪一处出错。上板验收的人拿着它去对数，会拿一张已经不存在的列号表去对照。

判据：一份契约在文档里可能以多种词表出现（像素 / 格子 / 模块内的 `f_col` 常量名）， 改契约时的陈旧值扫描必须按"别名全集"逐个搜，而不是只搜自己最先想到的那一种写法。 更省事的做法是把扫描绑到契约的定义处**（这里是 `tools/gen_text_ram.py` 的 `P6_LAB` 与 `disp/p6_view.v` 的字段表），从那里把新旧值都取出来，再去文档里找旧值。** 顺带记一条： `_disp_work/verify_u20.py` 当时只断言了新值**在**（`+0.456`、`1,540,785`、 `10,268`）和某一处旧措辞**不在**（`（0.060）（**编码器侧**）`），并没有做"旧值不许在 活文档里存活"的普查 —— 验收脚本的默认动作应该是后者，因为它防范的是**没想到的** 那一种改写。

记录 38：坏量具比没有量具更贵 —— 它差点让我去改一处本来是对的代码

`disp/tb_p2_page.v` 报出两条 FAIL 之后，我把其中一条读成"P2 的文字层没画出来"， 于是开始怀疑刚接上的 `lab_ch_p2` 平面。为了一次看清整行，我在测试台里加了一段 ASCII 点阵打印。

那段打印有两个毛病。第一个很小：`$write(mapbuf[cc] ? "#" : ".")` 在 Verilog 里输出的是 **十进制码**，不是字符 —— 满屏都是 `46`（`.` 的码），我一开始还以为是"整页没墨"。 换成 `%c` 之后更糟：**整页仍然一个墨点都没有**，而同一时刻、同一像素直接探针打出来的是

```
[p2dbg] ch_raw=18 bits=ffc0 bit=1 txtpg=1 glyph=1 valbit=0 rgb=1f2d3d
```

`rgb=1f2d3d` 就是 `C_TEXT`。**两个量具在同一时刻给出相反结论，这时候必须假定新加的那个是坏的**， 而不是去改 DUT 让它跟坏量具一致。两个诊断块连同它们声明的 `integer rr, cc; reg [63:0] mapbuf;` 一起删掉了 —— 留着比删掉危险。

真正定位那两条 FAIL 的，是一个便宜得多的办法：日志里的中文标签被 GBK 控制台糊成了乱码， 但 `[ OK ]` / `[FAIL]` 是 ASCII。把它们按出现顺序连行号抽出来，一眼就看清：

```
489  [ OK ] de=0 期间一列都不写 : 0
491  [ OK ] de=0 期间列指针不动 : 0
493  [FAIL] de 回 1 后第 8 个可见拍写一列 : got 0 want 1
495  [ OK ] de 回 1 后列指针走到 1 : 1
497  [FAIL] de 门控后写的是第 0 列 : got 2 want 0
```

两条 FAIL 全在**我自己新加的第 7 节**里；那两条关于文字的断言（375 行、377 行）**本来就是 `[ OK ]`**。 我差一点就按"文字没显示"去改一条正确的通路。

判据：诊断件与直接探针冲突时，先怀疑诊断件；定位失败断言时不要依赖会乱码的标签， 按 `[ OK ]` / `[FAIL]` 的出现顺序编号。量具本身也要有量具 —— 一个"整页没墨"的结论， 必须能被"某一个像素有墨"的直接观测推翻。

记录 39：我自己新写的测试里有一个 off-by-one，而它伪装成了 DUT 的缺陷

给 `disp/wave_trace.v` 加 `de` 门控之后，我在 `disp/tb_p2_page.v` 第 7 节加了三条断言， 头两条就红了： `de 回 1 后第 8 个可见拍写一列 : got 0 want 1`。

被测逻辑其实是对的。错的是我的观察者：

```
always @(posedge clk) if (tr_we) n_wr <= n_wr + 1;   // 寄存的计数器
```

`we` 只持续一拍，而这个计数器是在**写那一拍之后**才更新的。我在使能之后只等了 8 拍就去采样， 看到的是"第 8 拍写事件之前"的值；要看到那一拍的结果，必须 `repeat (8 + 1) @(posedge clk); #1;`。 第三条 `got 2 want 0` 也是同一个根因（列指针多走了一格）。

判据：任何"寄存的观察者"报告事件的时间都比事件本身晚一拍，等待拍数必须写成 "使能拍 + 1"；断言的文字里要写明它在第几拍采样，否则下一个人只会看到"差一格"而不知道差在哪。

记录 40：12 位的中间量把地址的高 6 位吃掉 —— 而它伪装成"影子内存坏了"

P2 四枚徽标（Vpp / f / Δt / ΔV）的写口长这样：

```
waddr <= {BADGE_ROW, (f_col(fld) + {9'd0, p})};
```

我读它的方式是"行号 + 列号"，工具读它的方式是：内层 `{9'd0, p}` 是 **12 位**， 拼接结果是 18 位，而 `waddr` 只有 12 位 ⇒ **高 6 位被截掉，而高 6 位正是 `BADGE_ROW = 36`**。 四枚数值全部被写到第 **0** 行、列 40..63。改成先按 6 位算好列号再拼：

```
wire [5:0] wc = f_col(fld) + {3'd0, p};
... waddr <= {BADGE_ROW, wc};
```

真值表能对、波形能跑、综合无警告 —— **这类错误只能靠"读回来的东西不符合预期"暴露。** 它暴露时的样子极具误导性：上一轮我在测试台里用一块影子内存抓写口再回读， `always @(posedge clk) if (we) sh[waddr] <= wdata;` 里 `$display` 明确打出 `a=43 d=8`， 而同一运行里读 `sh[2347]`（即 `{36,43}`）却是 **0**，于是我把三版判据全花在"影子内存为什么不灵"上 （清数组、换 `integer` 位选、去 `-NoNewline` 改文件……）。**影子内存没错， `sh[2347]` 当然是 0 —— 真写落在地址 43。** 最后把判据从"抓下来再回读"改成"**每来一次写就与期望串比一格**"， 这个 bug 立刻现形。

判据：凡是把拼接/相加的结果赋给更窄的左手边，宽度由左手边决定 —— 写之前先问一句 "这个和在目标位宽里放得下吗"。另外： `读回 0` 的第一嫌疑是"写根本没落在你以为的地址上"， 不是"存储坏了" —— 要证伪它，打印写口的地址，而不只是数据。为了看清一行而放弃 一层间接（影子内存）换来的直接判据，值。

记录 41：同一类截断在量具里又犯一次；"帧内统计"的窗口边界必须写清靠哪一拍切

修完上面的地址错误，测试台从 96 errors 掉到 6 errors。这 6 条逼出了两件事。

第一件是**窗口语义。** `disp/p2_badge.v` 的统计区间是"两次 `start` 之间"：`start` 那一拍 先冻结再清零；而 min/max 的累积**不受 `chg` 门控**（每拍都吃当前样本）。于是每个用例的测量帧里 都混着**上一个用例幅度突变**的残影 —— 用例 ④ 是静止样本，却测到 `a = 29..99`，那正是上一用例 从 18101 掉到 0 的那一跳留下的。修法在测试台侧：加 `reg chk_en`（只有测量帧的写才比对）， 并加任务 `flush_case()` 先空转一帧把残影冲掉：

```
task flush_case(input signed [15:0] aa, ab);
    begin chk_en = 1'b0; run_samples(aa, ab, 9); pulse_start; end
endtask
```

第二件更没面子：**同一类位宽截断，第二天在测试台里又犯了一次。**分组判据写成

```
if ((wcol >= F3) && (wcol < F3 + 6'd6))
```

`F3 = 58`，于是 `58 + 6 = 64` 在 **6 位里回卷成 0**，第四枚永远匹配不上， 每一次写都被判成 `[FAIL] badge write outside the four fields: row 36 col 58..63` （48 条），而窗口里明明写着 `col ..63`。改成 `wcol <= F3 + 6'd5` 才对。

判据：任何"帧内统计"都必须写明窗口是靠哪一个事件（哪一拍）切的边界， 不能让"上一个用例刚跑完"充当边界；刺激在两个用例之间变化时，第一帧必须作废。 以及 —— 位宽截断不认人：它前一天吃掉的是 DUT 的地址高位，第二天吃掉的是量具的比较常量。 写常量比较之前，把那个和在心里按目标位宽算一遍。

记录 42：烧录被"另一个 hw_server"挡住 —— 而它在沙箱里既看不见、也杀不掉，只有 netstat 找得到

U-22 位流（1,627,057 B @ 2026-10-15 23:24:01）下板时，`connect_hw_server` 报：

```
ERROR: [Labtoolstcl 44-494] There is no active target available for server at localhost.
Targets(s) ", jsn-JTAG-HS1-210512180081" may be locked by another hw_server.
```

我按 22:44 那次成功烧录的命令逐字重跑了两次，都死在这里，于是在错误的方向上找了一圈环境问题 （仓库里记过的 `cs_server` 往 `%TEMP%` 解包 `VCRUNTIME140.dll` 是**另一个坑**，这次 cs_server 起得来，日志里明明有 `Launching cs_server...`）。真正有用的动作只有一个：**去看谁占着 3121 端口。**

```
netstat -ano | Select-String ':3121'
→  TCP  127.0.0.1:3121  ...  LISTENING  15668      （外加两条 ESTABLISHED 的旧客户端 5088 / 29164）
```

而在这之前，`Get-Process` 只列得出 **13 个进程**、里面没有任何 `hw_server`； `tasklist /FI "PID eq 15668"` 回的是**拒绝访问**； `Get-PnpDevice -PresentOnly` 甚至数出 **0 个设备**（一台正常机器不可能）。 也就是说：**沙箱把"别人的进程"和"设备枚举"一起挡在外面了，而 netstat 是唯一漏出来的缝。** `taskkill /F /PID 15668` 必须带一次性 `danger-full-access` 才成功；杀掉后 3121 立刻空出来。

但故事没完：清了残留服务器之后，Vivado 这次自己 `Launching hw_server...`，结果从"目标被锁" 变成"**一个目标都枚举不到**"：

```
ERROR: [Labtoolstcl 44-199] No matching targets found on connected servers: localhost
```

⇒ 结论要分开记：**① 22:44 那次之所以能成功，是因为当时已经有一个更早启动、且跑在沙箱之外的 hw_server 占着 3121**（对照两次日志：成功那次**没有** `Launching hw_server/cs_server` 行，而失败 的这几次都有）——真正的 USB 访问发生在那个进程里，不在我的进程里；**② 把持线的旧服务器杀掉之后， USB 侧不会自动恢复**，需要物理拔插 JTAG-HS1 重新枚举，新起的 hw_server 才认得到线。

判据：烧录前先 `netstat -ano | Select-String ':3121'`。若 3121 已有人 LISTENING，而它又不是你 这一次要用的服务器，先弄清它是谁 —— `hw_server` 会活过 Vivado 进程（常驻），下一次烧录会连上 这个旧服务器**而不是新起一个，于是"坏掉的旧状态"会被继承下来。在沙箱里查不到进程时，别再用 `Get-Process` / `tasklist` 反复确认，直接上 netstat + `taskkill`（带一次性 `danger-full-access`）； 硬件侧的最后手段永远是拔插那根线。**

记录 43：量具连翻两次车 —— 两次都是"量具坏了而不是被测件坏了"

U-23 要给 `disp/disp_engine.v` 新加的绘制层（状态栏三入口、P6 危险区、模态对话框）写像素级判据， 先写了一个 `count_text(x0,x1,y0,y1)` task 数"某个格行里有多少个文字像素"。它报的 FAIL 看起来 非常像 DUT 的缺陷：**危险区标签、对话框标题、寄存器号"全都没画出来"。**

转机是**拿它去数一段一定存在的文字**：P1 导航的 `总览`（tb 自己的 [5] 段早就在 y 65..78 找到过 `min_x_nav=16`）—— `count_text` 数出 **0**。逐层探针（`u_dut.lab_bits / glyph_bit / with_txt / lcd_rgb`）随即证明 DUT 完全正常：page=5 时 `(96..115, y=53)` 上确实有一簇 `1f2d3d`（= C_TEXT）。 task 自己打印的进出也露了馅： `enter y0=544 dbg=(0,544)` 而 `exit n=1 dbg=(0,0)` —— 只数到一个像素 就跳出了。⇒ **task + `wait(de_o && ...)` / `while (de_o && ...)` 这个写法在本 tb 里不可靠**； 同一份 tb 里已被 [5]/[6] 验证过的是"自由运行 always 每拍累加 + 帧边界前后清零"。

第二次换量具又撞在同一条规律上：改成 `scan_ph` 相位 + `wait (frame_done)` 划窗口，连续两次 wait 之间**只隔一个时钟**（`[scan] after wait1 t=204499910000` → `after wait2 t=204499930000`， 一个时钟 = 20000 ps），相位窗口根本落不到整帧上 ⇒ 又是三轮假 FAIL。最终写法：**用 tb 自己掌握的 状态（`dlg_tb`）门控计数、用 `n_frame` 变化判帧边界**（不信 `wait(frame_done)` 的相位）。

还有一处判据本身不讲理：关闭态本想用"格行 15 上没有文字"判"对话框没露头"，但对话框标题/寄存器号用的正是 P6 自己放标签与数值的格行 15/17/21 —— 那 420 px 是页面自己的字、**合法存在**（改成 `[info]`）； 关闭态只能判**像素**（遮罩 `5A6B7A` 与红框 `D9453D` 不该出现）。

判据：新量具先拿"已知一定存在"的内容校准一遍；要划时间窗口就用自己掌握的状态， 不要用被测件的握手信号（它的相位不由你决定）。

记录 44：几何层不认识页号 —— 一个"看起来能加进矩形表"的按钮，两种排序都是错的

给 P6 底部危险区加"清除锁存故障"按钮时，第一反应是把它的矩形（x 192..640、y 528..592）加进 `disp/hit_test.v` 的矩形表。结果是**永远点不中**： `in_card` 在优先级里最高，而 P1 的两张 row1 卡片 （col0 x 96..384、col1 x 416..704、y 336..592）把那块面积**大面积重叠**掉了，只剩两张卡片之间 那 32 px 的缝（x 384..416）能落到危险区上。

而我当时差点把"排到卡片之前"当成修法 —— 那会把 P1 卡片 11/12 的那部分面积**抢走**，更糟： `disp/disp_engine.v` 的 `in_card = (page == 3'd0) & ...` 是**按页门控**的（只有 P1 有卡片）， 而 `disp/hit_test.v` 从设计上就**不认识页号**（文件头写明"只做几何"）。同一块屏幕坐标在不同页 属于不同控件，几何层要判归属就必须知道页号 —— 两条路都不通。

最终把归属**上移到策略层**： `hit_test` 里**不登记**这个按钮， `disp/enc_top_disp.v` 用原始坐标 + 页号判 —— `wire tap_dz_clr = hit_stb && (page == 3'd5) && (hit_x >= 12'd192) && (hit_x < 12'd640) && (hit_y >= 12'd528) && (hit_y < 12'd592);`（`hit_x` / `hit_y` 本来就在端口上，此前注释写着 "留作调试、本层用不到"，

现在正好用上）。`disp/tb_hit_test.v` 加了两条**反向判据**（`(300,560) → 5'd11`、`(500,560) → 5'd12`）把这个几何事实钉住：谁要是将来又把危险区矩形塞回 `hit_test`，测试台立刻报出来。

同轮另外两个自找的坑，一并记：

- **新加端口不接就是 X**：`disp_engine` 新增 `input wire [1:0] ui_dlg` 之后，例化它的测试台 没接 ⇒ `dlg_on = (ui_dlg != 2'd0)` 求值成 X，三元 `compose = dlg_on ? ... : no_dlg` 把整屏染成 X，`tb_disp_engine` 报出"最左文字像素 = 999999"。当时按 grep 结果补了 6 个文件（`tb_disp_engine` / `tb_p2_page` / `tb_p3_page` / `tb_p5_page` / `tb_p7_page` / `tb_pipe_align`），跑完 17/17 全绿就 以为收工了 —— **漏了第 7 个例化点 `disp/sim/smoke_p1.v`**：它不在 `run_sim_disp` 的 17 个台里， 所以回归全绿也照不出问题，直到按流程补跑 `run_smoke_disp.py` 才看到 p1 光栅文字检查 `FAIL: screen text ' ' != written text '1525.88'`（整屏 X，解码器读不出字形 ⇒ 全是空格）。 补上 `.ui_dlg(2'd0)` 后立刻 `PASS`。**加端口要搜的是"所有例化点"，不是"回归清单里的例化点"； 用 `disp_engine u_` 这类模式搜，而不是按测试台文件名的清单搜。**
- **又用 PowerShell 批量回写源码**：`Get-Content -Raw` + `.Replace()` + `Set-Content -NoNewline` 把这 6 个测试台的行尾整体改掉（`git diff --stat` 显示 663 行删除）。这次当场发现并 `git checkout --` 还原，改用 edit 工具逐处插入。**源码文件只用 edit/write 工具改** —— 同一条在 U-22 已经踩过一次，这是第二次。

判据：屏幕坐标的归属必须先问"哪一页"；往几何层加矩形前，先问"它和已有矩形重叠吗、 重叠部分在别的页属于谁"。

记录 45：同一个"位宽不认人"，第三次 —— 这次是 time 字面量

U-24 给 `disp/tb_disp_engine.v` 加了 `[8]` 段（P1/P4 标题带按钮 + P2/P3 页内 Tab 的像素判据）， 帧数变多，于是把超时保护从 `#1_200_000_000`（约 70 帧）抬到 `#2_400_000_000`（约 140 帧）。

结果整个仿真 **8 秒就结束**，日志里只剩一行 `[FAIL] TIMEOUT`，前面 `[1]..[7]` 段的打印 **一条都没有** —— 第一眼像"编译/加载失败"或"测试台崩了"。真正的证据在 xelab 的输出里：

```
WARNING: [XSIM 43-3368] "C:/Users/zhaoy/Desktop/FPGA/disp/tb_disp_engine.v" Line 735.
Negative delay (-1894967296000.000000) treated as 0.
$finish called at time : 0 fs
```

`2_400_000_000` 超过 32 位有符号上限 `2_147_483_647`，字面量回绕成负数，xsim 把负延迟 **当成 0** ⇒ 不等待、直接走到 `$finish`，于是"超时"在第一帧就触发了。 改成 `#1_900_000_000` 即可，并在注释里写下这条教训。

同一个坑在本项目里已经是第三次：U-21 的 `waddr <= {BADGE_ROW, (f_col(fld) + {9'd0, p})}` 把 18 位拼接截成 12 位、把行号丢了（数值被写到第 0 行）； U-22 的 `F3 + 6'd6` 在 6 位里 回卷成 0（第四枚徽标永远匹配不上）； 这次是 `time` 字面量。三次都不是"逻辑想错了"， 而是**写常量时没问一句"这个值放得下吗"**。

判据：任何常量、任何拼接、任何延时字面量，落笔前先算它的位宽上限； 仿真"秒退"先看 xelab/xsim 的 WARNING，不要先怀疑自己的测试台逻辑。

记录 46：两套几何各写各的 —— 框底线从字上穿了过去

U-25（批次③「标注精修」）给 P5 补表格框： `p5_frame` 的外框取 `x = 398..901`、`y = 126..321`（321 = 条形区底 `P5_BAR_Y1` + 1）。 写完先跑 `tools/run_sim_disp.py`： **17 / 17 全过** —— 因为测试台是照着同一份 常量写的断言，(500, 321) 上确实有一条 `C_GRID` 线，量具与被测件"同源"， 自洽得很。

真正发现问题的时刻是 `python tools/preview_pages.py` 之后把图点开看： `_disp_work/page_p5.png` 里框底那条横线**正从「111 PCT」这几个字上穿过去。** 原因是屏幕上有**两套各写各的几何**：

- 文字层（`tools/gen_text_ram.py` 的 `P5_LAB`）：第 20 行 `y = 320..335`， 退化状态「BAD 000 PCT」/「111 PCT」就落在那一行、 `x = 384..495`；
- 绘制层（`disp/disp_engine.v`）：条形槽最后一格到 `y = 303`，我"顺手"把框底 取成了 321。

两者单看都对，叠起来才错。改成 `P5_FRAME_Y1 = P5_BAR_Y1 - 15 = 305` （303 + 2 px 边距）之后，框只包住"六行表"，退化状态那一行落成框外脚注； 测试台的断言也跟着改成「(500, 305) 有线、(600, 306) 必须无线」—— **即"框底不许切到脚注那一行"这条规则，现在有判据钉着。**

教训：显示工程里"文字格位"与"像素几何"是两套契约，谁都不许单方面推断对方。 凡是新画一条线，先问一句"它在纵向会不会撞上某一行的字"，然后开图确认。 回归全绿不能替代看图 —— 同源的量具量不出跨层冲突。

附：量具算错，被测件是对的（同一次排错）

同一轮， `disp/tb_p7_page.v` 新增第 3b 节，用四个例子钉住 P7 的百分比格式。 其中一例：门限取默认值（`0x170A_3D71` = (0.60)²、 `0x5C28_F5C3` = (1.20)²）， `amp_sq = 0x0100_0000` ⇒ `mag16 = 512`、 `diff = 11795`、 `11795 * 1000 / 35383 = 318` ⇒ 期望屏上是「**- 31.8**」。

xsim 报了 4 条 FAIL：

```
[FAIL] lab cell (28,12) = 0 want 11
[FAIL] lab cell (28,13) = 11 want 9
[FAIL] lab cell (28,14) = 9 want 16
[FAIL] lab cell (28,16) = 16 want 8
```

第一反应是"百分比通路算错了"。但把 `0, 11, 9, 16` 翻成字符（`ASCII_CHARS` 里 `'0' = 8`、空格 = 0、 `'-' = 2`）：**空格、'3'、'1'、'8'** —— 也就是 「`- 31.8`」， **正是设计意图。**是我把 318 的百位/十位/个位在断言里写反了 （`pct_val` 是"值 × 10"的十分位定点： `p1..p3` 依次是 $10^3/10^2/10^1$、 `p5` 是 10^0， 所以个位其实落在第 5 格而不是第 3 格）。改断言，不动 RTL。

教训：`[FAIL]` 只说明"实际值 ≠ 我写的期望值"，不说明"实际值是错的"。 被测件与断言冲突时，先把实际值翻成人能读的形态（这里是字符），再判断谁错 —— 本轮 Q1 记录 11、记录 28 都是同一个方向：先怀疑量具。

记录 47：三条"看起来 PASS 其实没测到"的断言，两个"画了却看不见"的像素

U-26（批次④ 交互能力）新写了 `disp/ui_panel.v`（倍频编辑面板）与 `disp/tb_ui_panel.v`。台子首跑 **29 个 errors**，根因很朴素： `ui_panel` 把 `page` 当成"0 表示在 P1"，而策略层传进去的是 `page == 3'd0`（1 表示在 P1） —— 极性反了，而且开面板那一支**根本没检查 `page`** ⇒ 面板压根没开。

真正值得记下来的是当时那些 **PASS**：第 1 节的"开面板后吸附到档 5"与 "待应用值 = 64"**两条都过了** —— 因为复位值 `pnl_idx <= 3'd5` 、 `pnl_inter <= 10'd64` 恰好等于期望值。**"面板开没开"那一条 FAIL 得正是时候**： 它先证明了"这块状态是活的"，后面"值对不对"才有意义。 教训写进了台子注释：**每个状态块的第一条断言，必须先证明"这块状态真的被激活了"， 再断言值。**

同一轮还有两条同类： * 三个命令按钮的底色断言取 `y = 552`，而按钮标签占 lab 格行 34（`y = 544..559`） ⇒ 读到的是标签笔画 `1f2d3d` （`[FAIL] ... @ (528,552) = 1f2d3d, want fff3cd`）。 改到 `y = 535` （按钮内、文字行之上），描边断言留在 `y = 552`。 * 面板关着时的反向判据里有一条"`(190,496)` 不能是白" ——

而 P1 上那里本来就是 白卡。**反向判据只禁"签名色"（强调色边框/按下态/符号/选中档/轨道/待应用黄/ 压暗底），不许钉页面自己的底色。**

像素路径上还有两个"画了却看不见"： * `[-]` / `[+]` 按钮第一版只画了框、**根本没有符号。**16 px 字格的格心落在 `183.5 / 199.5`，而 64 px 按钮的中心是 `192 / 832` —— 差 8 px，字符放不正 ⇒ 改成**图形画的横杠/十字**（32×6 与 6×32），并且"符号是符号、不是整块按钮" 也用两侧留白的断言钉住。 * 长按进度环第一版画在按钮的 1 px 外框上，而按下态的描边是**同色**的 `C_ACC` 且优先级更高 ⇒ **环被自己的描边完全盖住。**改成内环（62×62、周长 244 px、 每级 4 px），并把环的优先级提到描边之前。

教训一：回归全绿只说明"量具与件一致"，不说明"人看得见"。本轮 29 + 8 + 5 个 errors 里只有两个是真缺陷（没符号、环被盖住），其余全是断言自己站错了位置。 教训二：像素链上的"画了"不等于"看得见" —— 同色同位置的元素会互相吃掉， 必须用一条独立颜色的断言去钉它。

记录 48：注入"看起来完全成功"，被测件却一次都没看见

批次④（㉒）收口后补写 `disp/tb_enc_disp_panel.v` —— 全工程**第一个例化整片 `disp/enc_top_disp.v` 的台子**（此前 19 个台子都只例化子模块，这一层接线从未被 精化过；㉒ 唯一那个真 bug「`page` 极性」就出在这里）。为了绕开 I2C 与固定坐标的 从机模型，触点用**层次 force** 喂快照。

首跑 13 个 errors，报"点 P1 卡片 2 后面板没打开"。诊断打印显示：

```
dbg: tg=11 down=1 | dut.num=001 dut.tog=1 dut.x0=560 | hit.num=001 hit.rst_n=1
```

即"注入完全到达命中层"：`tch_num = 1`、`tch_toggle` 在翻、坐标 560 就在 `hit_test` 的输入端口上、复位已释放。可 `hit_stb` **一次都没出现过** （加锁存计数后读到 `stb_cnt = 0`，而 `tch_tg_d` 一直是 0）。

根因在**时间步**而不是逻辑：`force` 是在 `@(posedge sys_clk)` 之后立刻执行的 （task 从上一拍被唤醒），于是力和被测件自己的 always 块**抢同一个时间步** —— `hit_test` 在那一拍就把新 `tch_toggle` 记进 `tch_tg_d` 了，"新样本"这件事当拍 被消费掉，下一拍 `new_sample` 已经是 0。**连续两次等待 posedge 也补不回来： 事件已经不存在了。**

修法一行：`@(negedge sys_clk);` 之后再 force，把力施加在**两个时钟沿之间**， `hit_test` 下一拍才会看到"toggle 变了、坐标也换了"。修后 `stb_cnt = 10`、 面板正常打开，全链（触点 → `hit_test` → 策略 → `ui_panel` → `ui_edit` → 层内 UART → `enc_top` 的 reg_file → `disp_tap` 回读）一次跑通。

同一段里还有一个"编码陷阱"：上电回读 `tap_inter = 0`，而 `disp/ui_edit.v` 的约定是 **0 表示 ×256**（`rtl/abz_gen.v:118` 的口径）—— 也就是**最高档。** 台子第一版按 `[+]` 改值，被档位夹住不动， `pnl_pend` 永远是 0，后面的 APPLY / BACK / CANCEL 断言**连锁假失败**（又一组"看起来是件坏了、其实是量具 站错位置"）。改成按 `[-]` 后：档 7 → 档 6， `pnl_inter = 128`，两阶段提交 一次走通。

教训一：层次 force 必须避开时钟沿 —— 与 always 块抢同一时间步的注入， 可能"看起来生效"（DUT 边界读得到）却永远不被状态机看见。 教训二：凡是"先读回、再在此基础上改"的测试，第一步必须先确认读回值的语义 （本轮 0 = 最高档），否则改值方向的选错会把后续断言全部带偏。

记录 49：一条"永远为真"的断言，和一份抄了两遍的字符表

㉓ 字库扩容（lab 平面字符码 7→8 bit）本身很顺：加 `%` `×` `÷` 三个符号、8 个汉字、 把 `idx_w` 从 7 改成 8，生成器打印 `47 ASCII + 89 cjk = 136 codes`，测试台改完位宽 全绿。真正花时间的两个坑都不在"改宽度"上。

坑一：同义反复的反向断言。

㉒ 给 P1 加了模态编辑面板，面板上那两行按钮文字（`倍频分频` / `PRESET`、 `CANCEL` / `APPLY` / `回到`）是画在**文字层**里的 —— 也就是 P1 静态 lab 平面的 格行 21 与 34。 `disp/tb_disp_engine.v` 第 [9] 段当时断言"面板收起后这些字消失"， 它统计的像素数写在：

```
if (de_o && pnl_on_tb) begin                // 计数被"面板开着"门控住
    ...
    sc_pnl_t <= sc_pnl_t + 1;
end
...
pnl_on_tb = 1'b0;                           // 反向测试：先把面板关掉
sc_pnl_t  = 0;                              // 再清零、重数一帧
if (sc_pnl_t == 0) $display("[ OK ] 面板收起后文字消失");
```

量具本身被"面板开着"这个前提门控住了，而它要判的恰恰是"面板关着时有没有字"： 关掉面板之后 `pnl_on_tb = 0`，计数条件永不成立， `sc_pnl_t` 必然是 0。 **这条断言从写下的那一刻起就不可能失败。**

本轮在 `tools/preview_pages.py` 出的 P1 预览图上第一眼看见的就是它：卡片 2 底下 浮着 `倍频分频` 和 `CANCEL` `APPLY` `回到`。回看 RTL， `disp/disp_engine.v` 的 `glyph_page = glyph_bit & (~dlg_cell | val_bit) & ~glyph_pnl_sup;` 只排除了 P6 对话框的格位 —— P1 自己那两行标签在面板关着时照样被页面层画出来， **这个缺陷是 ㉒ 自己引入的，而 ㉒ 的测试台正对着它、却整段看不见。**

三处一起修：① `glyph_page` 追加 `& ~(lab_bit & (page == 3'd0) & ((lrow2 == 6'd21) || (lrow2 == 6'd34)))`； ② 测试台拆掉 `pnl_on_tb` 门控，改成**无条件统计**，正反两条分别判 `> 0` / `== 0`； ③ 预览工具（它只画静态层、不模拟模态）在 P1 上把那两行抹掉。

坑二：同一份契约抄在两个文件里。

`ASCII_CHARS` 在 `tools/gen_font_rom.py:528` 与 `tools/gen_text_ram.py:46` 各有一份 （字形表与标签表是两个生成器）。本轮只改了前者，第二个生成器立刻抛：

```
char '%' (in '%') is not in the font charset
```

报错信息诚实，所以这一坑只花了十分钟 —— 但它揭示的结构性问题是： **"字库"这个契约 没有单一真相源，两处靠人肉同步。** 现在两处都在注释里写明了"改这里要同步那一处"。

同一轮还有一个小号的同类坑： `disp/tb_val_format.v` 的期望串里不能直接写 `×` `÷` —— Verilog 字符串字面量会把它编成两个字节，逐字节比对必然失败；期望串里只能用 单字节替身（`×`→`*`、`÷`→`d`）。**语言层的编码细节也是契约的一部分。**

教训一：断言不能活在被测条件里面。 量具要先无条件地量（开着也量、关着也量）， 再分正反两侧判 —— 凡是"只在某个前提下统计、然后拿这个统计去判该前提"， 都是同义反复，写一百条也测不出东西。 **教训二：与 RTL 并行维护的"第二个渲染器"（预览工具）有独立价值。** 它不共享 `disp_engine` 的中间信号，正因为如此，它能看见自引用断言看不见的东西 —— 本轮的真缺陷是预览图先发现、RTL 后确认，顺序不能反。

记录 50：一块闲置的写口，和一张看不见模态的预览图

㉔ 把规范 §6.5 那句「↩ 回到 ×8」从纸面落到屏上：面板 BACK 按钮的标签要显示 `ui_panel` 记住的"上一次值"。改动本身不大（一个新模块 + 一根端口），但它踩中的 两个问题都不是"写代码"层面的。

坑一：屏幕上的"上一次值"没有地方可写。

P1 的静态文字层（`disp/text_ram.v` 的 `p1_lab_ram`）是 `initial` 一次成型的： 里面那行 `回到` 是**死字**。要显示运行期的值，就得找一块**运行期可写**的 字符 RAM —— 而 lab 平面一共八块，P2/P5/P7/P8 的写口分别被 `p2_badge` / `p5_view` / `p7_view` / `band_view` 占着。真正让这一步变简单的 是先去查了写口，而不是先写模块：

```
p1_lab_ram u_p1_lab_ram (
    .clk(clk), .we(1'b0), .waddr(12'd0), .wdata(8'd0),   // ← 从 v22 起就一直是 0
    .raddr(lab_raddr), .rdata(lab_ch_p1));
```

`we` 恒 0 —— P1 的 lab 平面是**唯一没有主人的一块**，所以新模块可以直接接它的 写口，不需要任何仲裁、不需要改 `disp_engine` 的取字时序。**"哪块 RAM 的写口是 空的"比"怎么写这个模块"更值得先查清楚。**

坑二：预览工具画的是静态层，模态内容在它眼里永远缺席。

`tools/preview_pages.py` 出的八张页图是查汉字、查版面的主要人眼工具。它在 ㉓ 做了一件对当时正确的事：把 P1 的格行 21/34（面板标题与按钮标签）从图上**抹掉**， 理由是"这两行是模态行，静态图上不该出现，否则看着像常显"。

后果是：本轮新加的标签带**在图上必然不存在** —— `undo_lab` 是运行期的， 静态层里只有上电兜底的两个字。抹掉是对的，但它把"模态内容"整类都抹掉了， 而 ㉔ 恰恰就是模态内容。

修法不是把抹掉撤掉（那会重新引入"看着像常显"的骗人图），而是**多出一张**： `panel=True` 走一遍 `render()`，出 `_disp_work/page_p1_panel.png`。做这张图时 又踩了两个"看着像真的、其实是假的"细节：

1. 面板是**不透明**的（`disp_engine.v` 的 `with_pnl` 那条 mux：面板矩形内一律取 面板色，页面层文字不透出来）。第一版只在面板矩形上画地板再叠文字层，结果 P1 那串 `12345678` 压在滑条上 —— RTL 里绝不会这样。
2. "落在面板矩形里"要按**格矩形**判交，不能按格号一刀切：文字层格是 16x16、 数值层格是 32x64，两者与面板矩形的交叠格不同。按格号切会留下数值层碎片。

坑三（小号）：注释比设计跑得快。

`tools/gen_text_ram.py` 的注释写的是「↩回到×N÷M …（pnl_undo_inter / pnl_undo_div）覆写」，实现里却只有 `×N`、没有 `÷M`。原因是分频根本不需要显示： 面板从头到尾都不改分频（`disp/ui_panel.v:316 req_div_code <= tap_div;` 原样带回）， 两次写入的分频必然相同， `÷M` 只是噪声。注释是先写的、实现是后想的， 收尾时把注释改回"只写倍频"并写清理由。同类小坑还有一处：注释里的格范围 `45..53` 与实现的 `46..52` 不一致（`undo_lab` 的 `LAB_C0 = 6'd46` / 7 格）。

教训一：改"运行期内容"之前，先查清哪块存储器还有空闲写口。 这一步定下的 不只是模块边界，还决定了要不要仲裁、要不要改取字时序 —— 先查端口，再写代码。 **教训二：只抹掉模态的预览图，等于宣布"模态功能不会有人眼证据"。** 静态图与 模态图要**各出一张**，否则要么图骗人（模态字看着像常显），要么功能看不见 （新标签带在图上不存在）。**教训三：预览器是"第二个渲染器"，它自己也会画错。** 不透明区域、格与矩形的交叠，这两件事在 RTL 里是显式 mux 与显式格判，在预览器里 必须**照着抄**，不能想当然。

记录 51：把"圆"先钉成一张表

㉕ 把长按进度从矩形环换成真圆环（spec §6.5「配环形成进度反馈」）。这一轮最值得记的 不是 RTL，而是**顺序**：先写生成器、把几何钉成整数表并自检，再写 RTL —— 于是 RTL 只剩 "查表 + 象限装配"，没有乘法器、没有 CORDIC、也没给像素流水线加级。U-26 当时把"真圆环" 记为偏离，理由是"要上乘法器/CORDIC，为 0.6 s 的进度条不值当"；这一轮发现那个理由 其实取决于**先做哪一步**：把平方与角度搬到生成期，这个问题就消失了。

坑一：生成期的自检不是形式，写错了会一路带到 RTL。

`tools/gen_ring_lut.py` 里那条 `环带像素 == 4 x 带内表项数` 的等式一开始是错的：我把 折叠表的"重数"想成了"只有折叠极值处才是 4 重"，算出 183 项并把它写进判据。实际 `fold(i) = i[5] ? i[4:0] : (i[4:0] ^ 5'h1F)` 是**处处 2:1**，每个表项服务 4 个像素 ⇒ 带内 175 项、环带 700 像素。要是这条等式没写，RTL 会拿着一个错表跑得很正常。

坑二：生成的 Verilog 里的语法错，要等工具来报。

`initial` 块第一版把一行四项写成 `mem[0] = 5'h00, 5'h00, 5'h00, 5'h00;`（Verilog 没有 这种多重赋值形式）。生成器自己不检查，`disp/ring_lut.v` 看上去也像模像样 —— 直到 xvlog/综合才报语法错。生成物要**当被生成物一样检查**：本仓库的做法是把"表内容"的判据 放进生成期 + 一个专门的 tb，而不是靠肉眼看文件。

坑三（小号）：`cover` 是 SystemVerilog 关键字。

`disp/tb_ring_lut.v` 是 `.v`，但 xvlog 默认按 SV 解析 ⇒ 拿 `cover` 当变量名直接语法错。 改成 `cov`。

坑四（这一轮真正值得写下来的）：判据会跟着实现一起失效。

`disp/tb_disp_engine.v` 第 [9] 段原来只有一处进度环采样点 `(200,526)`：矩形环时代它 落在环上；换成真圆环（环带 26..30 px）之后，它距圆心约 33 px、**落在环外**。几何改了、 采样点没改 ⇒ 测试会以"颜色不对"失败，看起来像 RTL 错了，实际是判据过期。修法不是去调 颜色，而是按圆心/半径把采样点重算成四个方向（12/3/6/9 点，各自落在扇区 0/15/31/47） 再加一条"环内（r ≈ 17.7）不画"的反向判据。

教训一：量具先行 —— 把模糊需求先钉成整数表。 "画个圆环"这种需求在 RTL 里怎么写都能 "像"，只有把半径、扇区数、取整口径先说清并写进生成期自检，"对了"才有意义。表定下来 以后 RTL 只剩装配，连"要不要乘法器"这个问题都不存在了。 **教训二：预览器要么与硬件同源，要么就会骗人。** `tools/preview_pages.py` 新出的环图没有 "自己再写一个 atan2 就算完"：它跑完用 `_disp_work/_ring_xcheck.py` 与 `disp/ring_lut.v` 的 1024 项**逐像素对表**（700 带内像素、valid 0 处不一致、扇区 0 处不一致）。"第二个 渲染器"必须能证明自己与硬件是同一个模型。 **教训三：改几何时，把"跟着几何走的判据"列一遍。** 采样点、期望色、魔数常量都是几何的 从属物；这一轮的 `(200,526)` 就是漏网的那一个。改完几何先 grep 采样坐标，再跑回归。

记录 52：一个乘法器的位宽，和一份没有跟着改的预览器

㉖ 把 P5 的表格按 spec §4.4 重排成 6 行 × 5 列、行高 30、列宽 188。这一轮没有新算法 （除法用比较器、相位用 16:1 mux），却连着踩了两个"老坑"：一个是 Verilog 的位宽规则， 另一个是本仓库自己刚记过的教训（预览图必须与硬件同源）**在同一轮内又演了一遍。**

坑一：`perm * 10'd188` 的结果宽度是 10 位，不是 16 位。

条形图长度按"满量程 188 px"算，写成 `bar_t <= (perm * 10'd188) >> 10;`（perm 是千分比、 10 位）。仿真里满程只有 64 px，第一反应是"阈值算错了"。实际是 Verilog 的规则： 赋值目标/上下文的位宽**不会**向内传播， `a * b` 的结果宽度取两个操作数里**较大**的那个 （这里 10 位），于是 188×1000 = 188000 先被截成 10 位。改成 `(perm * 16'd188) >> 10` 才是 183 px。教训是：**乘法里不要依赖上下文宽度，把常数写成目标宽度。** （这与 U-26 记的"位宽陷阱"同源，但触发点不同：那次是加法进位，这次是乘法结果宽度。）

坑二：17 位的字面量在 16 位端口上被静默截断。

`disp/p5_view.v` 里从上一版抄来的"无数据哨兵" `99999` 是 17 位，lint 报 `WARNING: [VRFC 10-8497] literal value 'd99999 truncated to fit in 16 bits`。计数饱和点 本来就该取 16 位满量程（double-dabble 只迭代 16 次）⇒ 改成 `16'd65535`。 lint 的这类警告值得当错误看：截断后仿真里"看起来也对"，只是数值范围悄悄变小。

坑三（这一轮真正该记住的）：预览器是第二个渲染器，几何一改它就会变成第二张骗人的图。

`tools/preview_pages.py` 上一轮已经有 P5 的预览（16 px 格网版）。这一轮 RTL 改成虚拟格网 （30/188）之后，预览器**没改**，于是它画出来的 P5 与硬件不再是一回事 —— 如果拿它 去核对新几何，只会"确认"一个错的版面。这正是 U-26 记录 50 写过的坑（预览图只画静态层、 看不见模态内容），换了个形式又出现一次。修法是给预览器补上同一套映射 （`p5_cell_org()` / `draw_p5_lab()` / `draw_p5_chart()`，常量照抄 `disp/disp_engine.v`）， 并在文件头写明"这里的常量与 RTL 是同一份契约"。

坑四（小号）：几何的从属物散落在测台里。

`disp/tb_p5_page.v` 里有 24 处几何魔数（`BAR_X0=400 / BAR_X1=900 / BAR_Y0=128` 等）。 几何一改，这些判据全部作废 —— 但它们**不会报"过期"**，只会以"颜色不对"的样子失败。 这一轮的做法是按新几何把采样点重算一遍，并在注释里标清每个采样点对应哪条线。

教训一：位宽要显式。 Verilog 里"上下文定宽"是个诱人的假设，乘法/加法都不遵守它； 凡是 188、1000 这种"看起来小、乘起来大"的常数，把操作数宽度写出来。 **教训二：第二个渲染器要么同源，要么就会骗人。** 预览器、生成器、测台都算"第二个渲染器"： 它们的正确性来自"与硬件是同一个模型"，而不是"看起来像"。每轮改几何都要问一次 "还有谁在重算这个几何"。 **教训三：几何变更要有清单。** 常量、判据采样点、期望色、生成器的文字表、预览器 —— 这一轮的清单是五项，漏掉任何一项都会在下一次核对时才发现。

记录 53：灵敏度表藏在一个函数体里，和一次不成立的变异实验

㉗ 把「正常 / 警示 / 故障」状态词上屏：新模块 `disp/stat_chip.v` 每帧往六张卡片的 底部写 24 格（字形 + 空格 + 两个汉字），判据全部来自 `io_flags` 与 `stat` 的位。 单元测试（15 个用例）绿了，端到端段（`disp/tb_disp_engine.v` 的 [11] 段）一开就是 **7 条红**，而且红得极有规律：**每张卡的字形格装着上一张卡的字形**（卡 k 装卡 k−1）， 文字格却全对。

排查是怎么收敛的。 `chip_st` 探针是对的（`12'h9a5`），说明判据与状态本身没问题； `check_card` 逐格读 `p1_lab_ram.mem[{row, col+k}]`，只有 k=0 错。第一反应是"写丢了" （RAM 写口被 `undo_lab` 抢走）或"寄存器晚一拍"，于是给写口加了逐笔台账： 每帧 24 笔全部发出、地址正确，**但字形格那一笔携带的数据本身就是旧状态** （`[wr] #1 idx=0 chip=0 cellk=0 addr=(15,6) data=49 cst=0` ——地址是第 0 张卡， 数据却是第 1 张卡的状态）。所以不是"没写进去"，是"算出来的就是旧的"。

主因：连续赋值的灵敏度表只由实参决定，与函数体里读了什么无关。

```
function [7:0] cell_at; input [1:0] k;   // 函数体里直接读模块级 cst
    case (k) 2'd0: cell_at = (cst == ST_OK) ? CH_OK : ...; endcase
endfunction
assign wdata = cell_at(cellk);           // 灵敏度表里只有 cellk
```

`cellk` 每张卡才换一次（3 → 0），而那一拍 `cst` 也刚更新 —— 两个进程在同一个时间步里 抢先后，xsim 里 `cellk` 先算，于是字形格整整一张卡都停在上一张卡的状态上。文字格走的是 `word_at(s, ...)`（状态作为**参数**传进去），所以完全正确 —— 这个"只有一格错、其他全对" 的现象极具误导性，一眼看像写口/时序问题。

修法是把状态也变成实参：`function [7:0] cell_at; input [1:0] k; input [1:0] s;`， 调用 `cell_at(cellk, cst)`。**Verilog 里函数的参数不只是给值，它还是灵敏度表的来源。**

同一条路径上还有第二个真缺陷：寄存输出 + 优先权 mux = 每帧静默丢一笔写。 `we/waddr/wdata` 原本是寄存器：决定写的拍上 `hold=0`，下一拍 `hold` 可能变 1，而 disp_engine 的写口 mux（`p1_waddr = ul_we ? ul_waddr : sc_waddr`）此时已把写口让给 `undo_lab`，可 `sc_we` 还是 1 —— 这一笔（地址 + 数据）被无声吃掉，FSM 的 `idx` 却照常 推进。两个写者都从 `vf_start` 启动，所以丢的固定是第 0 格。改成组合输出

（`assign we = (st == S_WR) && !hold;`）后，"在写 ∧ 未 hold"成了同一拍的一件事。 **教训：跨模块的写口仲裁，写信号必须与它依赖的优先权信号同拍。**

元教训（这一轮最该记住的）：变异实验本身也可能是错的。 为了拿到"修复是必要的"这一侧的证据，我做了变异——把 `cell_at` 的函数体改回读模块级 `cst`，**但保留了 2 个实参的调用。**结果单元测试与端到端**都通过**，差点得出 "这个修复没什么用"的结论。原因很简单：实参里有 `cst` 这件事本身就已经在修灵敏度表， 所以那次变异根本不是复现。改成与被复原的代码**逐字同形**的真·原始形态（1 个实参 + 函数体读模块级）之后：单元测试 **27 errors**（含专门补的 ⑮ 灵敏度守卫）、 端到端 **7 errors** ⇒ 修复这才有了正反两侧的证据。 **教训：变异必须逐字复现被修的那一行；只要你的"坏版本"和原版有任何一处不同， 它证明的就不是原版的问题。**

附带：同一个单元测试第一版曾经是绿的。 那个 14 用例的版本用的是同一个缺陷代码， 却通过了——因为 xsim 那次把 `cst` 排在前面算。**竞态"曾经绿过"不是证据**： 判据必须能在反例上变红（这一轮补的 ⑮ 就是干这个的），否则绿灯只说明这次调度运气好。

其它小坑（同一轮）： `cell` 是 Verilog 保留字（`VRFC 10-4982 syntax error near 'cell'`）； `idx` 的声明位置（`VRFC 10-3380 identifier 'idx' is used before its declaration`）； 列序用 `chip[1:0]` 会把卡 4/5/6 的列搅成 2/0/1（单元测试第一次跑就抓到， `[FAIL] 卡 6 第 0 格 got 50 want 48`）； 测试台超时不能用 ns 硬编码（126 帧 = 2.16e9 ns

> xsim 的 `2^31-1` ns 量程）⇒ 改按帧计数。

另一条：既有的字库测试抓到了新字形的一个真缺陷，而它自己也没覆盖新码位。 第一版的 `×` 点阵与字母 `X` **逐点相同**， `disp/tb_font_rom.v` 的"两两互不相同"判据 直接报 `[FAIL] glyph 41 is identical to glyph 50`；改成加粗的叉才拉开区别。 更值得记的是后半段：那个测试台的 `NA`（ASCII 字数）写死着 48，不改它的话， 新加的 48/49/50 三个字形**根本没被判到**，而且它对汉字的位序还整体错位 3 —— 也就是说"新增字模"这件事，只有把判据的覆盖范围一起推过去才算真的验了。 （顺带把打包字面量的宽度按 UTF-8 字节数 61 显式声明，不再靠截断。）

教训一：连续赋值里不要调"读模块级变量"的函数。 要读什么就让它进实参。 **教训二：变异必须逐字（否则它证明的是另一个程序）。 教训三：竞态的判据要能变红。** 只看到绿灯不能收工。 **教训四：加字形/字模时，先问判据的覆盖范围推过去了没有。**

记录 54：一条被自己盖住的像素断言，和一个失去意义的差值判据

㉘ 给状态字模补 spec §5.2 的**第三重冗余**「反转视频（白字反色块）」： 某张卡的状态 只要不是"正常"，就把它的迷你状态条那 4 个格（64 × 16 px）整块反转成"状态色底 + 白字"。 RTL 只动一层 mux（`disp/disp_engine.v` 的 `with_txt`）：

```
wire scbar_inv = scbar_any && (scbar_sel != 2'd0);
wire [23:0] with_txt = scbar_inv ? (glyph_page ? C_CARD : scbar_bg)
                                 : (glyph_page ? txt_col : with_dz);
```

白字条件刻意用 `glyph_page`（而不是 `lab_bit`）： 它与文字层**实际取位同源**，所以反转态 的白字与正常态的黑字逐像素同位置。这条选择后来直接决定了判据怎么写 —— 见下面第二段。

第一次跑红在一条看起来最"稳"的断言上。 新加的判据里，六张卡各取"空格那一格"的 一个像素（那格没有字形，像素只可能是块色或卡片白底，与字模改动解耦）。轮 A、轮 B 全过，轮 C 报：

```
[FAIL] 轮 C 卡 6（故障）状态条反色块 @ (760,584) = eaf3fb, want d9453d
```

`eaf3fb` 是页面底色（`C_BG`），不是"没反转"，而是**那个像素在轮 C 根本不属于卡片**： 轮 C 把模态面板打开了，而面板矩形是 `PNL_X0..X1 = 96..927`、`PNL_Y0..Y1 = 320..591`（`disp/disp_engine.v:255-258`）—— 它把**整排第二排卡片**都盖住了。同一轮里第一排的 断言（`120,248`、`760,248`，状态条在 y 240..255）全过，因为它们在面板外面。

修法不是放宽判据，而是把断言挪到"当下可见"的地方：面板开着时只查第一排（卡 1、卡 3）， 第二排的可见性判据留在面板关着的轮 A / 轮 B 里查。**教训：像素断言是"某个模态下的 断言"，写之前先问一句"这个坐标在这个模态里还可见吗"。** 这跟记录 50 那次"预览图看不见 模态"是同一类错 —— 量具先行的纪律必须覆盖模态，不能只覆盖几何。

第二条：一个信号一旦"反转"，原来的差值判据就失去意义了。 U-31 留下的判据是"正常轮的墨量 ≠ 警示轮的墨量"（差分）：它只证明**屏上改了东西。** 加了反转之后，正常轮卡 1 是 273 px 黑字、警示轮变成 **0 px 黑字**（字变白了）—— "两者不等"仍然成立，可它已经不能说明"反色块出现了"：如果哪天状态条整块被盖住、 一个像素都没画，这个差分照样可能"不等"。于是判据被拆成两条**互相独立**的谓词：

- `C_TEXT` 墨量：正常轮 **> 0**（黑字在卡白底上）且警示轮 **== 0**（字变白了）；
- 块色墨量（`C_WARN` / `C_FAULT`）：正常轮 **== 0**（不反转）且警示轮 **> 0**（块在）。

实测轮 B 块色 762 px，而一个 64 × 16 的反转块是 1024 px，`1024 − 762 = 262` px 正是 白字墨量 —— 与轮 A 的 273 px 同量级（差值来自字形码不同：`● 正常` vs `▲ 警示`）。 **数字自己要对得上**：一个"看起来通过"的差值判据，换成一个能自洽的绝对判据，才经得起 下一次改动。

第三条：同一个数字在两个地方算，预览器必须用同一条 mux。 `tools/preview_pages.py` 新增 `invert_chip_bands()`：它只把**已经画成 `C_TEXT` 的字形 像素**改成白、其余像素填状态色 —— 与 RTL 那条 mux 的语义逐字对应（`glyph_page ? C_CARD : scbar_bg`）。同时新增一张 `page_p1_chip.png`（演示状态：卡 5 警示 / 卡 6 故障）， 因为静态遥测里六张卡都是"正常"、主图上永远看不到色块。第一版演示图出了个自相矛盾的 画面：**警示色块里写着「● 正常」**（静态 RAM 里那四格就是"正常"字样，而运行期是 `stat_chip` 每帧改写它们）。修法是演示状态时把那四格字符码也一起换成 `(49, 0, 136, 137)` / `(50, 0, 97, 98)`。**教训：预览图也是判据，它自相矛盾就等于没画。**

其它小坑（同一轮）： `disp/tb_disp_engine.v` 里原来只声明了 `C_FAULT`，本轮要用 `C_WARN` ⇒ 补 `localparam [23:0] C_WARN = 24'hE08B2D;`； `expect_not_px` 的失败文案原来 写死"（面板关着不该出现）"，现在这条 task 也被反色块判据复用 ⇒ 改成"（这个坐标不该 出现该色）"，免得下一个人被文案误导。反色块会压住卡片的左/下边框各 1 px —— 这是几何 使然（卡框 x 与格列起点重合、第二排卡底框 y 591 = 格行 36 的最后一行），**记为口径、 不改几何。**

教训一：像素断言先问"这个模态里还可见吗"。 教训二：判据被"反转"后要重写成能自洽的绝对谓词，别留着失去意义的差值。 教训三：预览器与 RTL 同源；预览图自相矛盾等于没画。

记录 55：一个恒为 0 的累加器，和一条"可关闭"的纪律

㉙ 补 spec §5.2 那一条的最后一块：四重冗余 = 字形 + 中文文字 + 反转视频 + **可关闭的 2 Hz 闪烁。**㉗ 落了前两重、㉘ 落了第三重，本轮落第四重。RTL 只多了一段相位寄存器 与一条门（`disp/disp_engine.v` 的 5.13 段）：

```
// 每 15 个帧首翻一次：半周期 15 x 17.0688 ms = 256.032 ms => 1.9529 Hz
always @(posedge clk or negedge rst_n)
    if (vf_start) begin
        if (blk_div == 4'd14) begin blk_div <= 4'd0; blk_ph <= ~blk_ph; end
        else                          blk_div <= blk_div + 4'd1;
    end
wire scbar_blk = scbar_inv && (ui_blink_en ? blk_ph : 1'b1);
```

频率只能是帧量化的。 一帧 = 853,440 px ÷ 50 MHz = 17.0688 ms；半周期取 15 帧 = 256.032 ms（1.9529 Hz，差 spec 的 2 Hz 约 2.4%）。14 帧会到 2.093 Hz（差 4.7%）， 16 帧 1.830 Hz（差 8.5%）—— 15 是 58.59 Hz 帧率下最接近 2 的整数帧。为什么不引 像素级计数器：`vf_start` 已经是一帧的边界，再引一次 853,440 这个魔数只会多一处 需要同步的常量。

"可关闭"不是可选项。 spec 把闪烁写成默认存在的第四重冗余，所以策略层 （`disp/enc_top_disp.v`）的 `reg scr_blink = 1'b1;` 复位默认**开**；"可关闭"是给现场 嫌闪的人用的。它是**显示侧偏好**：不进寄存器、不进 `disp_tap` （`rtl/` 一行未改）， 关掉以后前三重冗余仍然全在 —— 这正是"冗余"的设计意图。

第一次跑红了 3 条，而且红得很有欺骗性。

```
[FAIL] 闪烁相位 1（亮）：卡 1 状态条没有反色块（块色 0 px）
[FAIL] 闪烁相位 0（灭）：状态条没有黑字（该回到 ●/▲ 字形 + 中文）
[FAIL] 闪烁关闭 + 相位 0：反色块没常亮（块色 0 px）
```

同一段里"分频算术"（强推 `blk_div = 13/14` 看相位是否翻转）与 P6 复选框的四条 `expect_px` **全过** —— 也就是说 RTL 的相位发生器、复选框几何、标签格都是对的。 红的全是"块色墨量恒为 0"，这看起来像"反转没生效"，但根因在测试台： 这个台子的自动翻页是 `if (n_frame > 0 && !hold_page) t_page <= …`（每帧一页）， 而 `[11]` 段留下的那个每帧累加器按 `t_page == 3'd0` 门控。`[12]` 段我抄结构时 写了 `hold_page = 1'b0;`，于是 `wait_frames(1)` 的整帧里页面早就翻到 P2/P3， 累加器一个像素都没数到。`[11]` 段开头有 `hold_page = 1'b1;`。

★ "计数为 0"先怀疑门控，不要先怀疑 DUT。 累加器的门控条件（页号、模态、坐标） 本身就是判据的一部分；改了段就必须把门控条件一起想一遍。这是记录 50 / 54 里 "像素断言先问这个模态里还可见吗"的孪生兄弟：那次是坐标被模态盖住，这次是整帧 被翻页带走。修法是 `[12]` 开头 `hold_page = 1'b1; t_page = 3'd0;`，注释里写清理由。

同一个信号，两处的默认值刻意相反。 测试台 `reg t_blink = 1'b0;`（默认关）， RTL `reg scr_blink = 1'b1;`（默认开）。前者是为了让 ㉗/㉘ 的全部像素断言**确定** （开着闪烁，断言就会随相位飘 —— 那是假的稳定）；后者是因为 spec 把闪烁写成默认 冗余。两处的理由各自写在注释里，免得下一个人"统一"掉其中一个。

开关状态用几何表达，不用颜色。 P6 那颗按钮里的 16 × 16 复选框**实心 = 开、 空心 = 关**（`chk_zone` + `chk_edge` + `chk_mark`）。这与 §5.2 的"颜色绝不作为 唯一的状态编码"是同一条纪律：填/空在任何截图、打印、色弱条件下都能读。它也让 "这颗按钮现在是开还是关"变成一个可以逐像素写判据的东西（`expect_px(668,36)` / `expect_px(675,43)`）。

字库只追加、不插队。 §4.4 的 P6 需要「闪烁」两个字，`tools/make_cn_glyphs.py` 的 `CN_ORDER` 末尾追加即可（142 码、3.64 RAMB36）。插队会让屏上所有中文串位 （㉓ 扩容时正是这么定的规矩），所以哪怕"闪"字在字典里更靠前也不动位置。

其它小坑（同一轮）： `disp/disp_engine.v` 的 `ui_blink_en` 是新端口 ⇒ `disp/tb_disp_engine.v` 的例化必须补上 `.ui_blink_en(t_blink)`（漏了会让该输入悬空、 `ui_blink_en ? blk_ph : 1'b1` 变成 x）； P6 的复选框矩形（x 668..683）与文字层的 「闪烁」（格列 44/45 = x 704..735）**不许重叠**，取格列 41/42 恰好留出这段空隙。

教训一：计数为 0 先查门控（页号 / 模态 / 坐标），别先怪 DUT。 教训二："可关闭"要真有一个入口，且入口状态本身要能用几何读出来。 教训三：同一个信号在不同层可以有不同默认值，但理由必须写在注释里。

记录 56：一片白的面板，和一个停在 X 上的"镜子"

㉚ 把 spec §6.3 那条"连续滑条在 1024 px 上做 256 档 = 每档 <4 px，物理上不可用"的 最后一件补齐：**数字键盘兜底**（前三条 —— 预设档位步进器、长按加速、8 档吸附滑条 —— 在 U-22/U-26 已经做完）。本轮新增 `disp/seg7.v`（组合的单数位七段译码器）、 `disp/ui_panel.v` 的键盘状态机、 `disp/disp_engine.v` 的键盘绘制、 `disp/tb_seg7.v`， 以及 `disp/tb_ui_panel.v` 第 9 节与 `disp/tb_disp_engine.v` [13] 段的判据。

为什么要先写七段译码器。 `disp/disp_engine.v` 的 `glyph_pnl_sup` （:1564）把面板 矩形内的**字库字形全部抑制**，只放回 lab 行 21/34 的面板标题与按钮标签： 面板里的键位数字与输入值必须是**几何图形**。所以键上那 12 个数字、输入窗那 3 位 大数字，全部由一个 24 行的组合模块画（SMALL 20×32 厚 4 / BIG 48×80 厚 8， 掩码 `{a,b,c,d,e,f,g}`）。

先写"量具"，再写模块：七段的亮像素是可数的。 段矩形刻意做成**互不重叠** （g 的 x 范围 `[T, W-T)` 与 f/e、b/c 的 x 范围不相交），于是"字段内亮像素总数" = 点亮的段数 × 段面积，与段掩码是**两条独立的判据。** `disp/tb_seg7.v` 两者都断言， 第一次跑就靠第二条抓到真缺陷：

```
[FAIL] 大字段 0：亮像素 2146，期望 1536
[FAIL] 字段外 (63,…)/(…,-1) 四点全亮
```

根因是减法写在无符号域里：初版 `wire [11:0] dx = (cx >= X0) ? (cx - X0) : 12'd0;` 把"在字段左边"的像素**钳到 0**，于是它们落进 `seg_f` / `seg_e` 的 `dx < T` 而点亮。 修法是先判后减： `pre_field = (cx < X0) || (cy < Y0);` 再 `in_box = !pre_field && …`。 （同一版还把"字段左上角之前"的信号命名成 `before` —— SystemVerilog 的关键字， xvlog 直接报 `VRFC 10-8549 … keyword 'before' used in incorrect context`， 连带的 `VRFC 10-2989 'in_box' is not declared` 也是它引起的；改名 `pre_field` 即净。 `cell` 是保留字、 `idx` 必须先声明，这两条在记录 53 里已经写过一次。）

★ 真缺陷一："底"排在"图形"之前，图形就永远画不出来。 端到端第一次跑红 9 条， 全部是同一个形状：

```
[FAIL] 键盘入口按钮图标 @ (868,400) = ffffff, want 1f2d3d
[FAIL] 键 1 的数字（b 段） @ (140,342) = ffffff, want 1f2d3d
[FAIL] ← 键的箭头头部 @ (121,499) = ffffff, want 1f2d3d
...（描边、C_CARD 底、C_BG 的判据全过）
```

`ffffff` 就是 `C_CARD`： 面板上只剩一片白。原因在 `disp/disp_engine.v` 的 `pnl_col` 链里，我把 `kpb_any ? (kpb_down ? … : C_CARD) : kpb_dot ? C_TEXT :` 写成了**底在前、 图形在后**，而 `kpb_any` / `kp_box` / `kpv_any` 的矩形**包含**图标与数字的像素 ⇒ 后面的图形分支永远选不到。修法是把这一段重排成 **描边 → 图形 → 底**（并把理由 写在注释里）。这与 P5 条形图那条"fill > tick > track"的纪律同源： **底必须最后判，或者它的条件里显式排除图形。**

★ 真缺陷二：一个忘记驱动的"镜子"输出，会让整条通路停在 X。 `disp/ui_panel.v` 第一版把 `kp_on` 写成 `output reg` 却没在任何分支里赋值 ⇒ 端口一直是 **X**， 而下游 `disp/disp_engine.v` 用 `!pnl_kp_on` 门控步进器与滑条 ⇒ 那些几何也跟着 X。 `tb_ui_panel` 第 9 节第一条就报 `键盘入口前：键盘是关的 : got X want 0`。修法是把它变成**镜子**：`output wire kp_on; assign kp_on = kp_mode;` —— 状态只存一处 （`kp_mode` + 三位 BCD + `kp_sel`），其余全是镜子或门。这条与记录 53 的 "组合 `we` 才不会被面板抢拍"、`assign chip_st = …` 是同一条纪律： **同一个事实存两处，就一定会有一处忘记更新。**

★ 判据自己的毛病：`wait_frames(1)` 不是"恰好一帧"。 [13] 段第二次跑只剩 1 条 FAIL （`键盘模式：标题行一个像素都没进判定范围`），而同一个计数器在非键盘模式是绿的。 根因在测试台的 `wait_frames`：它是 `f0 = n_frame; while (n_frame == f0) @(posedge clk);`， 只等**帧号变化**，覆盖的是**半帧**（从上一次 `expect_px` 停住的 y 到下一帧同位置）。 键盘那一段前一个 `expect_px` 停在第 408 行，于是 y = 336 那一行根本没被扫到。 修法不是"多等一帧"，而是**换判据**：要判某个时刻的门信号，就在那个像素时刻**直读** （`wait (de_o && (dbg_x == 12'd136) && (dbg_y === 12'd336));` 再读 `u_dut.pnl_lab_cell`），并把不可靠的累加器与它的三个计数器全删掉。 ★ 教训：**"计数为 0"先怀疑门控与等待语义，再怀疑 DUT** —— 与记录 55 的"漏 `hold_page`"是同一条。

测试台自己的坐标也会错。 第 9 节第一次跑还有一条很能骗人的红： `输入 256：位数 3 : got 2 want 3`。原因是「6」被我手抄成 `tap_at(KP_C2, KP_Y2)`， 那是第 3 行第 3 列 —— 「9」，于是敲成 259 被上限拒绝（这**恰好证明**了"越界不接受" 是对的）。★ 教训：栅格坐标要从公式算（`(112 + 52·列 + 22, 328 + 52·行 + 22)`）， 不要手抄；手抄出来的"红"会让人去改本来正确的 RTL。

设计上的三处刻意选择（写进注释了）。 ① 越界**不接受**而不是截断：`kp_nv_ok = (kp_nv <= 12'd256)` 之外的那一次按键被丢弃， 位数不变 —— 与 §6.2"屏幕绝不允许显示用户想要的理想值"同一纪律。 ② 首位 0 不接受（否则能敲出 "0256"）。③ `CANCEL` 在键盘里 = **退一层**（先收键盘、 面板还在；再按一次才收面板），因为键盘是面板里的更内一层，一次按键收两层会让 "我只是想退出键盘"这个动作消失。 `↲` 只把值放进"待应用"，写寄存器**必须**按 APPLY —— 两阶段提交一个字没动。

教训一：色彩链里"底"永远最后判（或者显式排除图形）—— 否则画面上只剩一片白。 教训二：状态只存一处，其余一律是镜子 / 门；忘记驱动的"镜子"会把下游拖成 X。 教训三：`wait_frames` 只等帧号变化，不等于"恰好一帧"；要判瞬时信号就当场直读。 教训四：测试台里的坐标要从公式算 —— 手抄的坐标会把正确的 RTL 判成红。

记录 57　P8 不是「触摸诊断」，是「系统信息 / 自检」

这一轮做的事很小，但做了三个判断。

判断一：只做五项里的四项。 规范 `docs/13_display_ui_requirements.md:179` 把 P8 定义成「设备 ID、固件版本、链路统计、资源占用、自检结果」。前三项里 「设备 ID / 固件版本」在 `rtl/SPEC_interface_contract.md` 里**没有任何寄存器 来源**（全文没有版本/序列号寄存器），所以它们只能是显示侧自报的常量字符串； 「资源占用」是**位流期**的事实，运行期不可能测出来，也只能是常量。这两条 一旦不写进文档，下一个人就会去协议里找一个不存在的寄存器。第 ⑤ 项 「自检结果」要运行期判定 + 「通过 / 失败」两个字模，工作量与①~④相当， 所以这一轮把 ①~④ 落干净、把 ⑤ 明确写进"仍未做"，而不是塞一个假的通过。

判断二：换含义，不换坐标。 九个字段的行列坐标一个没动 （(1,3)(1,13)(1,23)(4,…)(7,…)），只换了含义与宽度。这样 `disp/disp_engine.v` 的 `p8_val_ram` 写口接线、 `tools/gen_text_ram.py` 的 `P8_VAL` 占位布局、 `disp/tb_touch_view.v` 的 `f_base()` 地址镜像**三处都不用改**， 只有镜像 `f_wid()` 跟着宽度走。上一轮数字键盘把面板标题格压住、 不得不加 `pnl_lab_cell` 抑制，就是"动了坐标"的代价；这次先算清代价再动。

判断三：静态常量要有守卫。 `RES_LUT_PCT / RES_BRAM_PCT / RES_DSP_PCT` 是写死在 `disp/touch_view.v` 里的三个百分比（来源 = 最近一次位流重建后的 `_disp_work/impl_util.rpt`）。写死的东西最容易变质，所以归

档的 `verify_u35.py` 会把这三个数与实现报告比对 —— 页面上的数字与自己宣称的来源必须一致。 另外把"每轮位流重建后回来核对这三行"写进了 RTL 的注释里。

这一轮的坑：加字必须同步字库台的计数。 版面加了 12 个汉字，`CN_ORDER` 只追加（顺序 = lab 字符码顺序，插队会让屏上所有中文串位），但 `disp/tb_font_rom.v` 的 `NC` / `NCH` 是按字面量声明的（91 → 104、142 → 155）， 忘了同步就会在字库台的"两两互不相同 / 非空"上先红 —— 这条纪律从 v23 起 一直在用，值只是换了数字。

另一个坑：控制台编码会把"通过"变成崩溃。 `python tools\run_smoke_disp.py` 这次不是仿真失败，而是**打印**失败：`UnicodeEncodeError: 'gbk' codec can't encode character`（xsim 日志里有替换字符，`print` 到 GBK 控制台就炸）。 绕法是 `cmd /c "set PYTHONIOENCODING=utf-8 && python … > 文件"`，再读文件。 这一轮的日志编码一共三种：xsim 与 `disp/logs/*.log` 是 UTF-8、 `_disp_work/lint_all_*.log` 是 UTF-16（`*>` 重定向的产物）、控制台是 GBK。 **判据是"日志里的那一行"，不是"进程退出码"** —— 这一条在 U-12 踩过一次， 这次是同一类问题的另一种形态。

教训三条： 1. 规范里那一页缺什么，先逐项问"这一项的数据**从哪来**"，来源不存在的项 要么做常量 + 守卫，要么明确不做，不能含糊地"意思一下"。 2. 改版面时把"动坐标"和"动含义"分开算代价：动含义往往能省掉三处镜像。 3. 一个"通过"的判据必须能说出它读的是哪一行； `print` 崩溃与仿真失败在 退出码上是同一种红，只有日志能区分。

记录 58　一级流水让判据读到了上一帧

这一轮做的事：把 P8 五项里的最后一项「自检结果」补上。规范 `docs/13_display_ui_requirements.md:179` 要求这一页有"自检结果"，而结论是 **运行期**的（IIC 计数每帧都在变），静态版面在 `disp/text_ram.v` 的 `initial` 里一次成型、改不了 ⇒ 新增第 6 个运行期写者 `disp/selftest_lab.v`：判据三项 （读次数 ≠ 0、无应答 = 0、超时 = 0），结论写在 P8 底栏格 36/37，静态标签 `自检结果` 在格 31..34，与 `band_view` 共用 `p8_lab_ram` 但**列不重叠**。

坑一：一级流水，判据读到了上一帧。 整机台第一次跑是 5 条 FAIL，逐条看 全是"结论停在上一组状态"：`[FAIL] P8 结论「通过」第 0 格（通）格 (36,36) = 161， 期望 159`。根因是两层叠加：① `q_tch_*` 在可见像素期（`de`）抓，结论在**下一次** `vf_start` 才写进 `p8_lab_ram`（差一帧 = 17.07 ms，画面上看不出来）；② `wait_frames(n)` 的道具是"等帧号变化"（= 半帧，㉚ 已经踩过一次），所以每换一次 `t_diag` 只等 1 帧时读到的必然是上一帧的结论。修法是把 `[14]` 段里 5 处 `wait_frames(1)` 全改成 `wait_frames(2)`，并把这两条因果写进注释。

坑二：假绿。 上面那一次跑里，用例 ③（NAK ≠ 0 ⇒ 失败）是"通过"的 —— 但它通过的理由是**上一个状态**（读次数 0 ⇒ 失败）恰好也是失败。也就是说， 判据在时序上不严的时候，"绿"不代表证据，只代表"这一次没抓到"。这与 U-12 那条"进程退出码不等于判据"是同一类的两种形态：**判据的价值取决于它读的是 哪一拍的哪一行。**修完之后每一组状态都换成了新的值，绿才是真的。

坑三：声明顺序，第四次。 整机台的驱动 reg `t_diag`（160 bit 的触诊位域） 在例化里被引用，所以必须在例化**之前**声明，否则 `ERROR: [VRFC 10-2938] 't_diag' is already implicitly declared on line 181`。 同一个坑前面三次是 `wr_n`（㉗）、`idx`（㉗，DUT 侧）、`t_kp_on`（㉚）。

坑四：表格行的锚点要用"最后一个"。 归档脚本往 `docs/15_display_touch_bringup.md` 里加现场记录时用了 `sub1(r"(\| 72 \|[^\n]*\n)", ...)` —— 命中的是**验证表**里的 `| 72 |` 行（§4.24 那张 4 列的表），于是 73..75 三行被插到了它的表尾，列数还对不上； 而它们该去的是第 8 节前那张**现场记录表**（5 列，末尾那一行也是 `| 72 |`）。 修法是取**最后一次**匹配（`re.finditer(...)[-1].end()`），再断言三行整体搬家、 字节数不变、每条记录号恰好 2 次。教训：一份文档里有多张同构表时， "命中恰好一次"只能证明锚点唯一，**证明不了插对了地方。**

承袭的两条纪律（本轮的收益）： - **组合写口 + `hold` 仲裁**：㉗（记录 53）的教训是"寄存输出会把'这一拍有没有 被别的写者占住'提前一拍猜"，被吃掉的写会静默消失。所以 `selftest_lab` 的 `we/waddr/wdata` 全组合，`hold` 期间既不写也不推进 `idx`；单元台的用例 ⑥ 专门钉这条（`hold` 拉高 5 拍，`we` 必须全程为 0，放手后两格仍全写完）。 - **共用写口只加一层 mux**：两个写者的列是 11/18/27 与 36/37，不重叠 ⇒ 不需要仲裁状态机；静态标签（格 31..34 = 155/156/157/158）的守卫同时是"新写者没有 越界"的证据。

这一轮的小坑：字库又加了 8 个汉字（自/检/结/果/通/过/失/败），仍然只追加在 `CN_ORDER` 末尾，`disp/tb_font_rom.v` 的 `NC/NCH` 同步成 112/163 —— 字面量的 纪律从 v23 起没变，只有数字在涨。

教训三条： 1. 有一级流水的地方，"等一帧"和"等一次帧号变化"不是一回事；判据读的是 **拍**，不是"大概过了一会儿"。 2. 见到"绿"要问一句：这个绿会不会是因为上一次的结果恰好相同？会，就要 把每一组状态都换成**新的期望值**再判。 3. 讲不清"我被谁仲裁、我什么时候真的写下去"的模块，不要用寄存输出； `we` 与 `hold` 必须同拍。 4. 往有多张同构表的文档里插行，锚点要用**最后一次**匹配，并且连"那张表有几列" 一起断言 —— 唯一性不等于位置正确。

记录 59　判据在旧值下也全绿，等于没有回归能力

这一轮做的事：把 spec §4.4 给 P6 写的「底部危险区 944×96」收口。⑲（U-23） 当时只做到 944×80（y 520..599），而且注释里写着"含触控外扩后 72 px"， **代码里的命中矩形其实一点没扩**（就是按钮画框本身）。本版：`DZ_Y0` 520 → 504 （96 px 靠收回第 4 项数值格下沿那 8 px 无墨迹空白，区 / 框 / 三角 / 按钮文字 四者中心仍在 y = 552 ⇒ 版面一个字没动）； `disp/enc_top_disp.v` 新增 `DZ_HIT_X0/X1/Y0/Y1` （184..647 / 512..591）= 画框 ±8 px，命中高度 80 px， 落在 spec §6.1「危险动作 72..96 px」内；整机台补 12 条像素判据、面板台补第 9 段 （四边"框内 4 px 受理 / 框外 4 px 不受理" + W1C 请求只发一条）。

坑一：注释里的"约束"也可能是错的。 `tools/gen_text_ram.py` 的 P6 注释从 ⑲ 起 就写着"档距必须是 64 的倍数、最小 128"。真约束只有一条：**下一项的标题不能落进 上一项数值格的字形里** ⇒ 档距 ≥ 69 px ⇒ 取 16 的倍数后最小 **80 px**（88 也合法）。 128 是"4 × 128 = 512 px 把 12 项铺满、让危险区贴底"的**版面选择**，不是约束。 教训：注释里出现的"必须"，要能当场把推导复述一遍；复述不出来就写"这是选择"。

坑二（本轮最值钱）：旧判据在旧几何下也全绿。 `disp/tb_disp_engine.v` 的 `[7]` 段原本有 6 条危险区判据，它们在 80 px 与 96 px 两种几何下**都能通过** —— 因为钉的都是区域内侧与按钮中点。于是 ⑲ 留下的 80 px 三轮没人拦。新加的 12 条 一律钉在"新旧几何的差"上：区上沿 (700,504) 是危险色而 (700,503) 不是；按钮 (192,520)/(192,583) 是、(192,519)/(192,584) 不是；三角 (140,532)/(140,572) 是、(140,531)/(140,573) 不是。**判据要钉在"改动会经过的地方"，否则改动就从旁边溜过去。**

坑三：同一份几何在两层各写一份裸数字。 危险区按钮的几何同时存在于 `disp/disp_engine.v`（画框 `DZ_BTN_*`）与 `disp/enc_top_disp.v`（命中矩形四个 裸数字 192/640/528/592）。画框改了命中不改，画面上按钮动了、点它却没反应 —— 而"注释说扩了"正好把这个缺口盖住。修法是用算式把两侧绑起来（`DZ_HIT_X0 = DZ_BTN_X0 - 8`），并在注释里写明"±8"是危险动作下限 72 px 的来源。

注一（新发现）：xsim 把 `$display` 里的中文写成 U+FFFD。 `disp/logs/tb_enc_disp_panel.log` 与 `disp/logs/tb_disp_engine.log` 各有约 2,049 个 替换字符（`ef bf bd`）⇒ 从这一轮起，**归档证据里引日志只能引 ASCII/数字部分**（用例编号 9a..9h、坐标、计数、`TEST PASSED` / `TEST FAILED`），中文标签不算证据。

承袭的一条纪律：往有多张同构表格的文档里插行，锚点要用**最后一次**匹配，并且 连"那张表有几列"一起断言（记录 58 的坑四）。本轮 `docs/15_display_touch_bringup.md` 的现场记录 76..78 与 §4.25 验证表都按这条走，验证脚本用"每个记录号恰好 2 次"兜住。

教训四条： 1. 判据要通过"新旧两版都跑一遍"的检验：如果改动前后它都绿，那它守的不是这段几何。 2. 同一个几何不要在两处各写一份数字；跨层引用要么用算式，要么写清"改这里必须 同时改那里"，并且有一条判据盯着。 3. 注释里写"必须"之前先把约束推导一遍 —— 否则一个错的"最小 128"会忠实地拦住 后面三轮的改进（⑲ 的 80 px 就是这么来的）。 4. 判据的锚点尽量选 ASCII/数字：日志与输出的编码不是证据的一部分，只有能被稳定 引用的字面量才是。

记录 60　拿一个字顶替另一个字，是一笔不会报错的欠账

这一轮做的事：把 spec §7 数据来源表（`docs/13_display_ui_requirements.md:305`） 给 ABZ/UVW 写的那句屏上要求落成字面 ——「时间轴画 A/B 会混叠，**必须按电角度 64/128 箱等效采样，并在图上注明**」。⑩（U-14）那轮确实在 P4 图上写了注记，但写的是 「128点等效采样」：当时字库是 44 ASCII + 83 汉字 = 127 码、lab 码宽 7 bit、 **只剩 1 个空位**，「按 / 重 / 建 / 箱」四个字一个都进不去，于是拿已有的「点」顶「箱」。本版补齐：`tools/make_cn_glyphs.py` 的 `CN_ORDER` 末尾追加「按重建箱」 ⇒ 汉字 112 → **116**（`disp/font_rom.v` = 51 ASCII + 116 汉字 = **167 码**、 140,672 bit = 17.17 KB = 3.82 RAMB36；上一版 163 码 / 139,648 bit / 3.79 RAMB36）， `tools/gen_text_ram.py` 的 P4 注记改成 `(27, 6, "按电角度重建128箱等效采样")` —— **14 格**、x 96..319、行 27（y 432..447，P4 框内唯一不压波形 / 轴标 / 刻度的 空行），`p4_lab_ram` 非空格 61 → 67。几何、其它七页、RTL 一个字节没动。

坑一（本轮最值钱）：顶替写法不会让任何判据变红。 用「点」顶「箱」这件事， ① 不编译报错；② 过得了字库覆盖检查 —— `check_covers_layout()` 只问"版面里的每个 汉字有没有字模"，而「点」本身有字模；③ 像素判据也照绿 —— 那一带确实有字。 三层防线全绿，于是一条与规范原话不符的注记在屏幕上活了十轮。**能抓住它的只有 "把规范原话与屏上字面并排逐字读"**（这一轮就是这么发现的），以及一条更根本的 纪律：**规范里出现引号内的字面口径时，屏上文字应当与引号内逐字一致**；不一致 就要么改字面、要么在文档里写明偏差与理由（像 ⑩ 那轮在注释里写的那样）—— 但"写在注释里"不等于"记在测试单里"，它不会在回归时被看见。

坑二：一条判据要同时管"是哪个字"和"有没有画出来"。 新加的 `disp/tb_disp_engine.v [15]` 段分两层：① 直接读 `u_dut.u_p4_lab_ram` 的 14 格字符码逐格比对 （`check_p4note`，期望码由 `_disp_work/_probe38.py` 按 `ASCII_CHARS + CHARS.index()` 复算 = 163 / 71 / 75 / 76 / 164 / 165 / 9 / 10 / 16 / 166 / 69 / 70 / 127 / 128）， ② 那一条带里 `C_TEXT` 的墨量 > 0（`p4n_px`，实测 2480 px）。只读 RAM 会被 "写对但没渲染"骗过；只看像素会把旧的 8 格注记也当成合格。两层合起来才等价于 "这一行字是对的"。

坑三：判据天然要"在旧值下会红"。 旧注记 8 格、新注记 14 格 ⇒ 逐格比对天生 对不上旧值；`NC 112 → 116`、`NCH 163 → 167` 也是同一道理（记录 59 的坑二）。 写判据时先问一句：**把代码改回上一版，这条判据会红吗？** 不会红就说明它没钉在 改动上。

注一（承袭）：xsim 把 `$display` 里的中文写成 U+FFFD。 本轮引日志只引 坐标 / 码值 / 计数 / `TEST PASSED`（记录 59 的注一）。写脚本时的期望值也一样： `dispatch` 到 doc 的字面量用 ASCII/数字锚，中文只作人眼可读的说明。

承袭的一条纪律：往有多张同构表格的文档里插行，锚点要用**最后一次**匹配， 并且连"那张表有几列"一起断言（记录 58 的坑四、记录 59 的承袭）。本轮 `rows` 锚点是 `| 78 |` 的**最后**一次出现 —— 这个行号在 `docs/15_display_touch_bringup.md` 里出现两次（§4.25 验证表 4 列 + 现场记录表 5 列），取第一个会把 5 列的行插进 4 列 的表格里。

教训四条： 1. 用"近义字 / 旧字"顶替规范里的字面，是一笔**不会报错的欠账**；发现它的唯一时机 是把规范原话抄下来逐字对屏上文字。 2. 一条文字类判据要成对：管"内容是谁"的读数 + 管"真的画出来了"的像素。 3. 判据写完先自问"改回上一版会不会红"；不会红就不是回归判据，只是现状记录。 4. 归档证据只引 ASCII/数字字面量（日志编码不是证据的一部分），中文说明归文档。

记录 61　规范里写着「屏上要写明」，而屏上一直没有那句话

这一轮做的事：spec §7 数据来源表（`docs/13_display_ui_requirements.md`）里还有两条 要求是写给屏幕的——`:300` 机械转速 `RPM = 60 × f_in / 极对数`（`0x0100`，默认 4）的 口径栏写着**标「估算值」**；`:302` 累计转数从「**本次上电**」起算、**不是掉电保存的 绝对值，屏上要写明。**屏上这两句话一直没有：P1 转速卡只有 `转速 / RPM`，累计转数卡 只有 `累计转数 / REV / Z计数`。本版补齐：`CN_ORDER` 末尾追加「估算上起」⇒ 汉字 116 → **120**（`disp/font_rom.v` = 51 ASCII + 120 汉字 = **171 码**、141,696 bit = 17.30 KB = 3.84 RAMB36），`tools/gen_text_ram.py` 的 `P1_LAB` 加 `(14, 46, "估算值")` 与 `(23, 26, "本次上电起算")`（3 + 6 格；行 14 夹在 `RPM` 单位与 卡片迷你状态条之间、行 23 在卡标题正下方），`p1_lab_ram` 非空格 114 → **123。** 几何常量、其它七页、值层数据、RTL 一个字节没动。

坑一（本轮最值钱）：同一张表里的要求会被人挑着做。 上一轮（U-38）把 §7 表格里 P4 那条注记做成了字面；这一轮才发现**同一张表里还有两条"屏上要写明"的要求**，十轮里没人动过。原因很具体：P4 那条要求连着"等效采样"这个功能，读表的人会顺手 看到；而这两条要求只是**表格里的一格文字**，看起来像"给文档读者的说明"而不是 "给屏幕的要求"。⇒ 教训是：跑完一个"字面化"任务之后，**要把同一张表从头到尾再逐行 读一遍**，把每一行的"屏上要求"列都问一次"这句话在屏上哪里？"。

坑二：字可以复用，码值不能动。 两句话总共只需要 **4** 个新字（估 / 算 / 上 / 起） 而不是 5 个 —— 「算」在「估算值」与「本次上电起算」里共用同一个字符码（168）；「值 / 本 / 次 / 电」早在表内（126 / 150 / 104 / 71）。但 `CN_ORDER` **只能追加**：码值 = `51 + CHARS.index(ch)`，插一个字进去会让后面所有码值整体漂移，P1/P6/P8 那些已经写进 测试台期望表的码值全部作废（㉞ 与 ㉟ 两轮都是靠"只追加"把改动关在 P1/P4 里）。

坑三：判据成对 —— 管"是哪几个字" + 管"真的画出来了"。 新加的 `disp/tb_disp_engine.v [16]` 段同样两层：① `check_p1notes` 逐格读 `u_p1_lab_ram` 的 3 + 6 = 9 格字符码，期望值由 `_disp_work/_probe39.py` 按 `ASCII_CHARS + CHARS.index()` 现算（`167 / 168 / 126` 与 `150 / 104 / 169 / 71 / 170 / 168`）； ② 两条带的 `C_TEXT` 墨量 > 0（实测「估算值」**624 px**、「本次上电起算」**988 px**）。 只读 RAM 会被"写对但没渲染"骗过；只看像素分不清那一带到底写了哪句话。

坑四：量具自己也会给出"看起来很合理的 0"。 写 `_disp_work/_cnt39.py` 数 `p1_lab_ram` 的非空格时，第一版正则照记忆写成 `8'hXX`，跑出来 **114 → 0**（两个版本 都数到 0）；真实文件里是 `mem[1] = 8'd33; // (0,1) 'P'` 这种**十进制**写法，改正则后 才是 114 → 123。⇒ 数到 0、数到"正好等于文件行数"这类**过于整齐的结果**时要先怀疑 量具，再怀疑被测对象（这与记录 59 的"判据在旧值下也全绿"是同一类错误的两个方向：一个是判据太松，一个是量具不对）。

注一（承袭）：xsim 把 `$display` 里的中文写成 U+FFFD。 本轮引日志只引坐标 / 码值 / 计数 / `TEST PASSED`（记录 59 的注一、记录 60 的注一）。

承袭的两条纪律：① 往有多张同构表格的文档里插行，锚点要用**最后一次**匹配，并且连 "那张表有几列"一起断言（记录 58 的坑四）。本轮 `rows` 锚点是 `| 81 |` 的最后一次出现。 ② 新判据写完先自问一句"**改回上一版它会不会红**"（记录 59、记录 60 的会红自检）—— 本轮 `[16]` 段的两层判据都满足：上一版 `p1_lab_ram` 里那两行位置根本没有字， 逐格读数与两处墨量都会红。

教训四条： 1. 一个"字面化"任务收尾时，把**同一张规范表格逐行重读**，逐行问"这行要求在屏上 哪里" —— 表格里的一格文字最容易被当成文档说明而漏掉。 2. 字库是资产：共用字让增量变小，但**顺序表只能追加**，任何插入都会让既有码值全部漂移。 3. 文字类判据必须成对：管内容的读数 + 管渲染的像素。 4. 工具给出的"过于整齐"的结果（0、全等、恰好等于行数）要先自查量具，再下结论。

记录 62　量具先红一次，把两个内缩逼了出来

这一轮做的事：spec §4.4 的 P5 是「6 行 × 5 列，行高 30、列宽 188，含占比条形图列」， 五列语义（序号 / **状态** / 次数 / 占比 / 条形图）是 v30 自定并记在 U-30.7 的。其中 **状态那一列一直是 ASCII 三位写法**（`U1 V0 W0` …），而界面是中文的。 `docs/DEV_STATUS.md:8846-8851`（U-25.7 ②）当年把它挂账，理由是"字库 44 ASCII + 83 汉字 = 127 码已满、lab 层码宽只有 7 bit（0..127），只剩 1 个空位"；㉓ 把 lab 码宽升到 8 bit 之后门槛消失，P5 的另外两条挂账也早已做掉。本版补齐： `CN_ORDER` 追加「高低」 ⇒ 汉字 120 → **122**（`disp/font_rom.v` = 51 ASCII + 122 汉字 = **173 码**、 142,208 bit = 17.36 KB = 3.86 RAMB36）， `tools/gen_text_ram.py` 的 `P5_LAB` 六行 1 → 高、0 → 低（U/V/W 与空格仍是 ASCII ⇒ **每行仍是 8 格**、 `p5_lab_ram` 仍是 84 个 非空格、虚拟格网与列位一个字没动）。RTL 里只有注释变化。

坑一（本轮最值钱）：判据的坐标要从 RTL 读，不能从"版面看起来在哪"推。 第一版 `chk_ink_band` 按"列带起点 + 16 px × 格"取横带（x 270..397），六行墨量各少 **26 / 26 / 26 / 30 / 30 / 30**。差值不是噪声，而是一个**可解释的常数**：正好等于每行 最后一格**最右 6 px** 的墨量（低 = 26 px、高 = 30 px）。顺着它回读 `disp/disp_engine.v` 才把 P5 的两个内缩读出来 —— `P5_INSET = 6`（列内左右各 6 px，11×16 + 12 = 188）与 `P5_VINSET = 7`（30 px 行带里 16 px 字形垂直居中）⇒ 真正的字格是 x **276..403。 如果当时把判据改成"墨量 > 0"，这个洞会一直留在这里**，而且没人会知道判据量的其实是 "列带"而不是"字格"。

坑二：容量类挂账要有一条"门槛解除后回扫"的纪律。 U-25.7 ② 挂账的原因是"字库满 + 码 7 bit"，㉓ 就把容量解掉了（lab 7 → 8 bit、新增 正常/警示/故障/回到 与三个符号）， 但**没人回头把当年因为容量做不了的三件事逐个捡起来**：其中"P5 汉字状态名"一直躺到 这一轮（另外两件 —— 百分比号、 `↩ 回到 ×8` —— 在 ㉓㉔ 里被顺手做了）。⇒ 记一条： **凡是因为"资源不够"而挂账的条目，解决资源的那一轮结束后要回头扫一遍名单。**

坑三：文字第一次换到"新渲染路径"上时，必须落像素。 汉字是**第一次进入 P5 的虚拟 格网**（30 px 行带 / 188 px 列带、RAM 地址 `{8+r, 11c + dx>>4}`、字形相位 `lab_bits[15 - dx[3:0]]`）—— ㊱ 之前这张表里全是 ASCII，那条渲染路径从来没跑过。而 "写进 RAM 了但没渲染出来"正是 v24 那个 7 bit 截断的表现（RAM 里码值一个不差，屏上 汉字全变成空格）⇒ 只回读 RAM 不够。本轮两层：① 六行各 8 格整块的 `C_TEXT` 墨量逐行 等于离线（`tools/gen_font_rom.py` 的同一份字形）算出的 **483 / 483 / 482 / 483 / 482 / 482**；② 18 个"新写法与旧 ASCII 取值相反"的像素（六行各 3 个）—— 这些点在旧版本上 必然红。

坑四（本轮新踩）：PowerShell 的 `>` 重定向会把 python 的 print 写成 UTF-16LE。 生成器输出（`python tools\make_cn_glyphs.py > x.txt`）落盘后 `read` 工具报 binary file —— 文件是 UTF-16LE。⇒ 读之前先按 UTF-16 解码（本轮落到 `_disp_work/_gen40_utf8.txt`）， 或者干脆走 `cmd /c "set PYTHONIOENCODING=utf-8 && python …"`。

坑五（本轮新踩，归档期）：验证脚本自己的期望值也会抄错 —— 它和判据一样是量具。 归档脚本 `_disp_work/verify_u40.py` 第一版有**三条期望写错**，全是它自己红出来才发现的： ① 判据窗口的终点我手抄成 `12'd404`，而 `disp/tb_p5_page.v` 里实测是 `12'd403`（x 276..403 含端点，276 + 8 格 − 1）； ② 断言"第 9 行 8 格只用 5 个不同的码"，忘了两个空格也占码位 （`code_of(' ') = 0` ⇒ 集合里其实是 6 个值，非空格才是 5 个）； ③ 逐格回读 `p5_lab_ram` 行 9 时写成 `cells.get(addr)`，而**生成器只写非空格**（RAM 初值为 0）⇒ 空格处取回 `None`，必须写 `cells.get(addr, 0)`。三条都不是被测件的问题。

注一（承袭）：xsim 把 `$display` 里的中文写成 U+FFFD。 本轮引日志只引行号 / 墨量 / 码值 / `TEST PASSED`（记录 59 的注一、记录 60/61 的注一）。

承袭的两条纪律：① 往有多张同构表格的文档里插行，锚点要用**最后一次**匹配，并且连 "那张表有几列"一起断言（记录 58 的坑四）。本轮 `rows` 锚点是 `| 84 |` 的最后一次出现。 ② 新判据写完先自问一句"**改回上一版它会不会红**"（记录 59 的"判据在旧值下也全绿"）—— 本轮 18 个证伪点全部落在新旧取值相反的像素上，改回旧版面它们必红。

教训五条： 1. 判据里的坐标必须从 RTL 的常量推（能读就现算）， "看着差不多"的横带会少量一格， 而差值恰好是一个常数时，它就是在告诉你"边界错了"，不是在告诉你"墨量差一点"。 2. 因"资源不够"而挂账的条

目，要有一次"资源解决后回头扫名单"的动作。 3. 文字换到新渲染路径上时，内容判据（读回）+ 渲染判据（像素）缺一不可。 4. 重定向出来的文件先验编码再读；"过于整齐"或"完全读不懂"的结果都先怀疑量具。 5. **验证脚本（归档守卫）也是量具**：它里面的每一个期望值（坐标、集合大小、缺省值语义） 都要从被测源现推或现读，凡是手抄的常数都要有一条"它等于哪个源里的哪个字面量"的线索 —— 本轮它自己红出三条抄错的期望，正好说明守卫也要先被守卫一次。

记录 63　一条判据钉在"页面自己也会画"的像素上，就是抛硬币

这一轮做的事：`docs/DEV_STATUS.md` 的「仍未做」清单里，从 U-37.8 一路挂到 U-40.8 第 3 条写着"危险区按钮没有按下/长按视觉反馈（只有进询问态后按钮底色变浅）"。 读代码先发现**这条描述不是根因**：旧行为是 `wire tap_dz_clr = hit_stb && 命中`， **按下那一拍就进询问态**；而询问态的合成把对话框之外整屏压成 `C_DIM`，危险区按钮 （y 520..583）整个落在对话框（y 196..403）之外 ⇒ 那一刻给按钮上什么颜色都看不见 —— 缺的不是"一个颜色"，是**反馈的时机**（被自己的模态吃掉）、**编码**（spec §5.2 "颜色绝不作为唯一的状态编码"，要靠形状）与**范围**（模态让步只该让步一个按钮那么大）。 本版三件事一起做：按下与发声分家（`dz_prs_lvl` 拉电平 / `dz_latch` + `hit_rel` 才产 `dz_click`、按住滑出再抬不算 = spec §6.5 的"防误触但不弹窗"）、反馈用按钮里侧 10 px 处 一道 **2 px 内嵌框**（形状编码）、 `dz_keep = dlg_on && dz_btn` 把模态让步的半径压到一个 按钮。写寄存器路径一字未动。

坑一（本轮最值钱）：判据钉在"页面自己也会画"的像素上，就是抛硬币。 [9] 段有一条 旧判据 `expect_not_px(192,496, C_TEXT, "panel off: 没有 [-] 符号")`，用的是面板的签名色； 可 `(192,496)` 落在 P1 卡片 [4] 的**数值格**上，而 tb 的激励里 `t_angle` **每帧 +8192**、 数值格每帧被 `val_format` 重写 ⇒ 那一点是不是 `C_TEXT` **取决于仿真相位**。证据链： `_disp_work/_run_full40.txt`（U-40 全量回归）里 `tb_disp_engine PASS errors=0` ⇒ 当时绿； 此后**只有 tb 两个文件变过**（`git diff` 确认 RTL 一字未改）； ㊲ 在第 [7] 段插了两条 `sample_at`（它是 `wait`，跨帧边界会把后续段相位整体推一格）就翻红。 ⇒ 判据改成读**层信号**：与合成同拍的输入坐标（`u_dut.px2/py2`）上断言 `u_dut.pnl_any === 0`（关）/ `=== 1`（开，证伪点）。 `pnl_any` 自带 `pnl_on` 门控、 与页面内容无关 —— 这是**加强**判据，不是把它改松。

坑二：加端口要 grep 例化点，漏一处就是 X 漏进合成。 `disp_engine` 有 **8 处**例化 （`disp/enc_top_disp.v` + 7 个 tb），用 `grep '\.ui_dlg'` 一次找全（它与 `ui_blink_en` 全部同现），逐个补 `.ui_dz_press`：真实链路接 `dz_prs_lvl`，tb 里除引擎台接 `dz_prs_tb` 外都接 `1'b0`。

坑三：日志取证的编码坑有两层，而且会吞掉 ASCII。 `disp/logs/*.log` 里的中文在生成 时就按 GBK 解码成了 U+FFFD，**GBK 双字节还会吞掉相邻的 ASCII 字符** —— 实测 `[FAIL] dz btn visible…` 在日志里是 `z btn visible…`、 `panel off:` 是 `off:`； 行结束是 `\r\r\n`（Python 文本模式把孤立 `\r` 当换行 ⇒ 文件行数与编辑器/ripgrep 不一致，**不能跨工具对行号**）。⇒ 取证只按坐标/编号这类**保证 ASCII** 的键过滤 （本轮 `_disp_work/_ok41.py` 就是这么抽的）。另外 runner 解析不出错误数时会打 `errors=?`，那不是 0。

坑四：沙箱里带中文输出的 runner 会崩在 `print` 上，别把 print 的崩溃当成仿真失败。 `python tools\run_smoke_disp.py` 第一次 exit 1 是 runner 末尾 `print(out)` 抛 `UnicodeEncodeError: 'gbk' codec can't encode character '\ufffd'` （xsim 的 `$finish` 早就打完了）。⇒ 走 `cmd /c "set PYTHONIOENCODING=utf-8&& python … > 文件"`；注意 PowerShell 的 `>` 会把输出写成 **UTF-16LE**（记录 62 的坑四）。

坑五：归档守卫（验证脚本）自己的期望值也会抄错，而且沙箱里的 `EXIT=0` 可能是假的。 `_disp_work/verify_u41.py` 第一次跑报 **9 条 FAIL**，逐条现读源码后发现**9 条全是我的期望 写错**：端口连接是对齐过的 `.ui_dz_press (dz_prs_lvl)`（4 个空格）、 `dz_keep = dlg_on && dz_btn`（多个空格）、FSM 是 `if (dz_click)` 独占一行再 `dlg <= 2'd1;`、 `reg dz_prs_tb = 1'b0;` 同样对齐；而 `tap_dz_clr` **确实**不再是 RTL 信号名，但注释里还留着它 当旧名做对照（`disp/enc_top_disp.v:481`、 `:564`、 `disp/tb_enc_disp_panel.v:439`，三行全是 `//`）—— "某字符串不存在"这类断言必须区分**代码**与**注释**（本轮

写法：`'//'` 在关键词 之前才算注释；对齐空格一律用正则 `\s*` 容忍，别抄字面）。 同一个坑还有第二层：`cmd /c "… & echo EXIT=%ERRORLEVEL%"` 里的 `%ERRORLEVEL%` 在**整行 解析时就展开**（`&` 与 `&&` 都一样），打出来的是**这条命令行执行之前**的退出码 —— 那 9 条 FAIL 我第一眼看到的恰恰是 `EXIT=0`。⇒ 判成功要看**输出内容**（`ALL CHECKS PASS` / `TEST PASSED` / `PASS n / m`），要看退出码就用 `if errorlevel 1 …` 或 `call echo %%ERRORLEVEL%%`（延迟展开）。

注一（承袭）：xsim 把 `$display` 里的中文写成 U+FFFD。 所以引日志只引坐标 / 计数 / `TEST PASSED/FAILED`（记录 59/60/61/62 的注一）。

承袭的两条纪律：① **新判据写完先自问"改回上一版它会不会红"**（记录 59）—— 本轮内嵌框八点用**同一点两态**：不按时必须是脸、按住时必须是红框，只测按住态的话 "把整个按钮刷红"也能过。② 判据里的几何从 RTL 常量**现算**（内嵌框四条边 = `DZ_BTN_*` ± 10/12）而不是抄屏幕上的目测值（记录 62 的坑一）。

教训六条： 1. 判据的观测对象要挑"只有被测层会画"的信号（层使能 / RAM 码值），不要挑合成后的 颜色 —— 页面、面板、文字层会共用同一个颜色值。 2. 加端口先 grep 例化点（含测试台）。 3. 一条"永不红"的旧判据可能只是因为它的输入恰好没变；要问一句"它依赖的量是谁在写"。 4. 日志取证按纯 ASCII（坐标 / 编号）过滤：中文已丢、行号不可比、还可能吞掉相邻 ASCII。 5. 沙箱里带中文输出的 runner 会崩在 print 上，用 `PYTHONIOENCODING=utf-8` + `cmd` 重定向。 6. 归档守卫也是量具：期望值现读源码（对齐空格用正则），"不存在"要分清代码与注释； 成功与否看输出内容，不看 `%ERRORLEVEL%` 那种解析期就展开的变量。

记录 64　用 PowerShell 改中文文件，等于把文件扔进碎纸机（附"从会话记录里捞回原文"的救法）

这一轮做的事：给 `docs/16_host_ui_refactor_R16_R20.md` 补两处计数（156 → 159）。 本该用 `edit` 工具，却顺手写了 `(Get-Content $f -Raw) -replace '…156…','…159…' | Set-Content $f -NoNewline`。 本机 `pwsh` 的 `Get-Content` / `Set-Content` 走**系统 ANSI 代码页（cp936）**： 读的时候把 UTF-8 字节按 GBK 成对解码、遇到非法序列就替换，写回时再按 GBK 编码 —— 结果是**文件几乎原样保留，但每隔几处丢一个字节。** `read` 工具随后报 `invalid UTF-8 text`，18,458 字节的文件里有 456 处不可解码位置（例：`草案` 的 `案` 只剩 `E6 A1 3F`）。**中文越多的文件，坏得越密**（本文件正文几乎全中文）。

救法（可复用）：DSH 把每次 `write` / `edit` 的工具调用参数**原样存进会话记录**（`~/.dsh/sessions/<工作区>/session-*/session.v4.jsonl.zstd`，zstd 压缩；本机 Python 有 `zstandard` 模块）。于是可以**按顺序重放**：取该文件的第一个 `write` 的 `content` 作基线，再逐条 `str.replace(old, new, 1)` 应用后续所有 `edit`，得到与损坏前一致的内容。 本轮重放 14 条调用还原出 10,144 字符 / 247 行，与原文件逐节核对无误。 两个必须注意的点：①只重放**file_path 属于该文档**的调用，否则别的 `write` 会把基线覆盖掉 （第一版就是这么把脚本源码当成文档写出去的）；② `replace_all` 要按参数处理，未命中的 `edit` 要停下来看，不能静默跳过（本轮就有一条是改另一个文档的）。

教训三条： 1. **改仓库里的文本文件一律用 `edit` / `write` 工具**；PowerShell 只用来跑命令、读数值输出。 非要用不可时显式 `-Encoding utf8`（且本机 PS 版本对参数名挑剔，报错也别硬凑）。 2. 破坏性写操作**先备份**（本轮那份 `.bak` 是事后才补的，等于没有）；改完立刻用 `read` 工具读一遍——它按 UTF-8 严格解码，是**最便宜的编码体检。** 3. 会话记录是被低估的"版本库"：**没有 git 兜底的未跟踪文件**（本文件当时 `git status` 是 `??`）被写坏时，重放 transcript 里的工具调用是唯一能拿回**逐字原文**的路子；先搜再动手重写。

记录 65　拆卡片墙时最危险的，是"删掉的东西还留在别人的代码里"

这一轮做的事：界面重构阶段 2 —— 把主窗口从"一根长页面上摞 11 张卡片"改成"左侧窄侧栏 + 顶栏 + 六页"， 并**取消 ⛶ 单卡放大**（用户 `m12177` 授权，决策表见 `docs/16_host_ui_refactor_R16_R20.md` §2/§3）。结果：离线 `selftest_ui.py` **151/151 PASS**、真机 `--real-port COM3 --seconds 6` **PASS**、 交付包第八次重打（见 `docs/DEV_STATUS.md` §1.86）。

为什么先讲手法：这次几乎没动面板代码，靠的是两条小决定—— ①**控件原地创建、最后 `addWidget` 搬进页面**（Qt 会自动改父），于是 ⑥⑧⑨ 的创建代码一行不改； ②把 `self.scroll` 从"那一个滚动区"改义成"**当前页的**滚动区"，老自检里满地的 `w.scroll.verticalScrollBar()` 只有日志那一段需要先 `show_page("diag")`。**接口语义对齐比重写便宜得多。**

坑一（最值得记）：删除重构的盲区是"被 `except Exception` 包住的调用点"。 `[ ]` 退场时删了 `_restore_card_max()`，但 `_save_prefs()` 里还留着一行 `if self._max_card is not None: self._restore_card_max()`。关窗时它抛 `AttributeError`， 被同函数的 `except Exception` 吞掉 ⇒ **`up.save()` 从没执行**，而界面看起来一切正常， 是自检 ⑯「参数记忆」4 项失败（`wave.x_span_ms=None window=None`）才把它揪出来。 ⇒ 教训：**删掉一个名字之后，要把这个名字当搜索词在全仓搜一遍（含字符串与注释）**； 并且"静默兜底"是善意的设计，却会替坏代码保密。

坑二：新方法默认老对象已经有某个属性。 概览页要 `AlarmPanel.summary()` 的一句话结论， 而 `_state` 只在 `_refresh()` 里赋值 ⇒ 自检第一轮就 `AttributeError: 'AlarmPanel' object has no attribute '_state'`。 改成 `(getattr(self, "_state", None) or {}).get(...)`。**"对象还没被刷新过"是常态，不是异常。**

坑三：界面自检的用例是"有状态"的，失败先怀疑前置条件。 ⑱ 测 `F` 全屏时红了一次， 根因是前一条用例把焦点留在了数字框里，而 `app.focusChanged` 会把快捷键**全部让给输入框** （这个设计本身是对的）。修法是测之前 `w.wave.canvas.setFocus()`。同类还有：测全屏前必须先 `show_page("live")`， 因为默认页已经换成概览了。**迁移期最贵的错误，是把"测试没准备好"当成"功能坏了"。**

坑四：中文引号写进 Python 双引号字符串 ⇒ `SyntaxError`。 一句界面文案里用了 `"…"`， 编译器把它当成字符串结束。文案也是要被解析的代码 ⇒ 统一用「」。这与记录 64（改中文文本用 `edit`） 是同一族教训：**中文和代码混在一起的边界，是最容易出事的地方。**

一条方法论收获（当时写的，阶段 3 已更正）：这轮用离屏截图逐页走查，我记成"离屏**能验结构、不能验配色** —— `#Card`、边界色这些 QSS 令牌在 offscreen 平台下会退化成灰阶"。**阶段 3 抓像素证明这是错的**： 离屏平台其实保留颜色（背景 `(245,245,247)`、卡片 `(255,255,255)`、强调 `(0,113,227)` 一字不差）， 当时看着灰是**那套令牌本身就是灰蓝的。**教训比结论更值钱：**"工具不行"和"我的输入本来就这样"要先分清**， 否则会用一个假限制把后面一整阶段的方法论带偏（我差点以"离屏不能验配色"为由跳过截图验色）。

顺手修掉的真缺陷：走查截图里卡片⑤ 的提示末行「依据。」被切掉一半——`ui_kit.fix_lines()` 用 `setFixedHeight(lineSpacing()*lines+2)` 把高度钉死，1600 宽下字体度量与实际换行算不到一块。 改成 `setMinimumHeight` + `setMaximumHeight`（留一行余量），并加注释"别再改回 `setFixedHeight`"。 **几何"钉死"在多分辨率下迟早会切字**：能给区间就不要给点值。

最后一步反过来查，才把死代码揪出来（这轮最值钱的教训）：打完第八次包做 PYZ 复核时， 正向那一半（"新东西进没进包"）一次全绿；我顺手写了**反向那一半**（"退场串还在不在包里"）， **当场抓到六处"删了功能、没删代码"的残留**： `host/theme.py` 的 `QToolButton#CardMax` 三段 QSS、`host/ui_kit.py::ScopeNav.set_plain_wheel_zoom()`（连同 `_plain_zoom` / `FINE_STEP` / `TIP_MAX` 与 `eventFilter` 里的裸滚轮缩放分支）、`ui_kit.field()` 里的 `v._row_lay`、 `host/panel_wave.py` / `host/panel_abz.py` 的 `_fill_ctrl(vertical)` 竖排分支与 `_reflow_readouts()` / `READOUT_CELLS`、 `host/main_window.py` 的三个空壳方法、以及全屏提示里"Esc 只负责还原卡片放大"的过期文案。 ⇒ **删除重构必须"逆向验收"**： `grep` 只找得到源码里的调用点，找不到"没人调用但仍在包里"的常量与 QSS； 这些东西在离线自检里全是绿的。删净后**重打第二遍**才是真正交付的包（8,014,886 B / 19:10:22）。

数字：界面自检 159 → **151 项**（退场的是放大态用例，新增的是分页/侧栏/概览用例）； 终跑真机（清理后的源码）一键使能 UVW `0x0109 = 0x0040 → 0x00C0 → 0x0040`、六状态 合法 20375 / 非法 0、 方波可看 256→1024 箱 / inter 256→64、界面取帧 60659 = 记录器 60659 行、文件行数 103857、丢帧 913、队列丢弃 0； 交付包第八次（两遍）**8,014,886 B @19:10:22**、SHA256 `C6E8F280…FA2562`、426 文件 / 583,895,025 B、PYZ 681 模块。

记录 66：一次"用错工具写文件"的事故，和一次被自己证伪的限制（2026-10-16，阶段 3）

事故：为了把 `host/README.md` 里两处自检条数改掉，我在 pwsh 里敲了 `(Get-Content host\README.md -Raw) -replace … | Set-Content host\README.md -NoNewline`。结果是**文件被写坏**： 控制台按本地代码页解码再重编码，**受损汉字的续字节连同紧随的一个空白字节一起变成 `?`**（连换行都被吞掉）， 文件不再合法 UTF-8（`read` 工具直接报 `invalid UTF-8 text`），而且**行被合并**（664 行变成 464 行）。 抢救用的是"碎片-间隙法"：把坏行按 `?` 切开，在 `git show HEAD:host/README.md` 里定位碎片， 两片之间的 HEAD 文本就是被吃掉的内容——**也只补回了一部分**，因为阶段 1/2 改写过的那几百行在 HEAD 里 根本找不到对应。最后只能**回到 HEAD 底稿，把阶段 1/2/3 的改动重新用 `edit` 工具贴一遍。**

教训按价值排序： 1. **中文文本文件一律用 `read` / `edit` / `write` 工具（显式 UTF-8）；要批量就写 Python 并显式 `encoding="utf-8"`。 永远不要用 pwsh 的管道 + `Set-Content` / `>` 碰它们。** 这不是"小心一点"能解决的，是工具选错了。 2. 出事后**唯一可靠的底稿是 `git show HEAD:<file>`；没提交的改动救不回来。**所以"先提交再大改"不是洁癖， 是把可恢复性买下来。 3. 诊断这类事故时，控制台是 GBK ⇒ `print()` 带 `\ufffd` 会抛 `UnicodeEncodeError`，**结果要写文件再读。**

被自己证伪的限制：阶段 2 我把"离屏截图看着一片灰"归因成"离屏平台丢颜色 ⇒ 不能用截图判配色"， 还把它写进了两份文档和本条日志的方法论里。阶段 3 抓像素一看：背景 `(245,245,247)`、卡片 `(255,255,255)`、强调 `(0,113,227)`，**颜色一字不差** —— 灰是因为**当时那套令牌本身就是灰蓝的。 ⇒ 把"工具不行"写进方法论之前，先用一个最小探针验证它。** 假限制比没结论更贵：它会让你主动跳过一整类验证。

这轮真正奏效的两条： * **配色不靠眼睛靠像素**：自检第 ⑳ 节 `grab()` 整窗后按步长采样找 `#F5F5F7` / `#FFFFFF` / `#0071E3`， 字号阶梯直接读 `pixelSize()`（22/17/28）⇒ 视觉契约变成回归项。**能断言的不要靠走查。** * **字体"随包"要验到"从包里装载"**：源码里跑通不等于冻结后跑通（路径从源码目录变成 `sys._MEIPASS`）。 证据是包内烟测运行日志里那句「界面字体：Inter 已装载（4 个文件）」。

顺手又踩到一个"侧面重置"坑：`setStyleSheet("color:…")` 会把手写 `setFont()` 里的 font feature（`tnum` 表格数字） 一起重置 ⇒ **上色和字体特性有先后顺序**（先上色、后开 tnum）。这类坑眼看不出来，靠的是"47/47 个读数标签 都必须是 Inter + tnum"这种全量断言。

数字：界面自检 151 → **164 项**；真机 PASS（界面取 60879 = 记录器 60879 行、文件行数 100694 = rows、 丢帧 780、队列丢弃 0）；交付包第九次 **8,021,631 B @19:25:53**、SHA256 `CDEAB910…4C89C7`、439 文件 / 585,288,730 B、PYZ 682 模块；走查做了**离屏 1950×1080 + 最小 1600×900 + 真实平台 150% DPI** 三档， 真机那档抓到可编辑端口框文本溢出（`setCursorPosition(0)` + 最小宽度 240→344）。

记录 67：重打同一份源码，哈希为什么会变；以及"退出码 0"不等于"我的验证跑过了"（2026-10-16，阶段 4）

哈希为什么会变：阶段 4 只改文档、没动一行源码，但按"交付包必须由最终源码构建"的规矩又重打了一遍。两次构建的结果：exe 都是 **8,021,631 B**、目录都是 **439 文件 / 585,288,730 B**、包内资源同一批 13 个文件 —— **内容完全等价**；但 SHA256 从 `CDEAB910…4C89C7` 变成了 `2CBEA58C…5782C7`，原因是 **PE 头里的构建时间戳。** ⇒ 教训：**交付包的身份要连"大小 + 文件数 + 时间戳 + 哈希"一起报。** 只报哈希，会把"同一份源码重打"误报成"内容变了"；只报大小，又挡不住篡改。

退出码陷阱（差一点把假证据写进文档）：包内烟测我原本这样跑 —— `& .\host\dist_onedir\host_app\host_app.exe --smoke … --seconds 4`，然后读 `$LASTEXITCODE` 得到 **0**， 差点据此写下"烟测通过"。实际上它是**窗口化（GUI 子系统）进程**，PowerShell 不会等它结束， `$LASTEXITCODE` 里残留的是**上一条命令（python）的 0**；那次运行**连截图文件都没生成。** 改成把输出重定向/管道捕获（`2>&1 | Out-File …`）后 PowerShell 才会等它退出，于是拿到了真结果： exit 0、 `截图成功：…1413x824`、运行日志「界面字体：Inter 已装载（4 个文件）」。 （`Start-Process -Wait` 在沙箱里被拒——那是另一条边界。） ⇒ 教训：**"命令返回 0"不是证据，进程自己产出的东西才是**（这次是截图文件 + 运行日志里的那一行）。 凡是我报告的"通过"，都要能指到"它写下的物证"；指不到，就说明我测的是别的东西。

这一轮照跑的验收（文档改完也不免测）：`compileall` exit 0；离线 `selftest_ui.py` **164/164 PASS**； 真机 `--real-port COM3 --seconds 6` **PASS**（界面取 60907 帧 = 记录器 60907 行、文件行数 112180 = rows、丢帧 944、关窗队列丢弃 0）。

记录 68　预览件是另一个实现："图上没有"不等于"屏上没有"

这一轮做的事：`docs/DEV_STATUS.md` 的「仍未做」清单里，spec §4.4（`docs/13_display_ui_requirements.md:177`）的「项高 88 px」从 U-23.7 一路挂到 U-41.8 第 1 条 —— 4 x 88 + 96 = 448 px 而内容区有 536 px，照字面做会在危险区上方空出 88 px。 用户拍板「加第五行把 88 px 的空档填掉」。**88 卡在哪**：它与 P6 用的两套全局栅格都不可 通约（文字层 16 px：88 = 5.5 x 16；数值层 64 px：88 = 1.375 x 64），v19..v41 的 128 px 档距是"标题行与数值格行都必须落在栅格上"逼出来的唯一解。⇒ 本轮照 v30 的 P5 五列表框 那条路线给 P6 加**虚拟格网**：一串比较器算出"项号 k"与"带内 y = v"（`p6_vy` / `p6_k` / `p6_ktop` / `p6_v`），拿它们当 RAM 行号与字形 ROM 的"格内 y"（标题 = lab 行 4 + k、格内 y = v；数值 = val 行 1 + k、格内 y = v - 13），两层各带一个可见性门；第 5 行放三个 v15 修正参数（增益S / 增益C / 相位修正，Q1.15，`m*125>>12` 得千分数）；字库 +3 字 （增益修，正 早在"正常"里）。

坑一（本轮最值钱）：预览件镜像是另一份实现，"图上没有"不能当"屏上没有"的证据。 P6 标题原先写 lab RAM 行 3..7，而虚拟区从 y 64 开始 —— 行 3 的**自然**位置是 y 48..63， 落在虚拟区外 ⇒ RTL 的自然映射会把「倍频」**多画一行**到第一行带上方当残留文字。 而 `tools/preview_pages.py` 的 `draw_p6_planes()` 只按虚拟行画、不画自然行，所以 `_disp_work/page_p6.png` 上**干干净净**。修法（行号 3..8 -> 4..8 的四处契约同步）之后 那张图与上一版**字节完全相同** —— 这正是"镜像看不出"的证明。⇒ 版面类改动必须靠 tb 的 **证伪判据**（"旧行区间残墨必须 0 px"），不能只看预览图。

坑二：判据自己的窗口会压到别的东西，红的可能不是被测件。 上面那条残墨判据第一次跑 就报 **22 px**，逐条查下来是标题带那颗「闪烁」复选框（`CHK_X0/Y0 = 668/36`、16x16， 实心时是 `C_TEXT`，占 y 36..51）被窗口下沿 **49** 数了进去；窗口改成 y 52..62 后为 0 px （旧标签墨迹在 y 49..62，取 52 起仍覆盖 11/14 行）。与上一轮的坑五（归档守卫自己的 期望值也会抄错）同类：**判据是量具，窗口与期望值都要现推/现读。**本轮的归档守卫自己也 抄错了 6 条期望（章节里 `@@NPASS@@` 只出现 1 次却断言 2 次、把 `p6_lrow = P6_LAB_R0 + k` 记成字面 `4 + k`），预检当场把它们拦下来。

坑三：改一处"数据层"要同时改四处"契约"。 `P6_LAB_R0` 一个常量牵动 `disp/disp_engine.v`（常量与虚拟格网）、`tools/gen_text_ram.py`（版面行号 —— 改完必须 重跑生成器， `disp/text_ram.v` 不能手改）、`tools/preview_pages.py`（镜像行号）、 `disp/tb_disp_engine.v`（回读地址 `{6'd4, 6'd6}`）。四处不同步时，"仿真绿而屏上错"与 "屏上对而仿真红"都可能出现。

坑四：新字段种类要先看状态机的分流条件，否则是静默错。 新增的 `K_Q15` 必须走 BCD， 而 `disp/p6_view.v` 的 `S_PREP` 原先是拿"种类编号 >= `K_STR`"当"不做 BCD"的判据 —— 新种类编号排在 `K_STR` 之后，照旧写法会**跳过 BCD**（屏上是空白，不报错）。已改成显式的 `wire cur_bcd = (cur_kind == K_NUM) || (cur_kind == K_PFX) || (cur_kind == K_Q15);` 再把 `if (cur_kind >= K_STR)` 换成 `if (!cur_bcd)`。

坑五：复制链脚本的"count == 1"断言把"我记错了"变成"当场停下"。 `_disp_work/_mk_u42.py` 从 `u41_numbers.py` 整块替换生成 `u42_numbers.py`，第一版多写了 一条**根本不存在**的替换串 ⇒ `AssertionError: count 0 for: 本文件是 u40_numbers.py 的 复制链`，脚本死在写文件之前（没有产生半成品）。这与逐处替换的 `_edit15_u4x.py` 是同一套 纪律：**宁可当场停，也不要写出一份数字半新半旧的归档。**本轮的 `_edit15_u42.py` 也当场 停下过一次：一句注记的锚点被我记成"那里是两处对不上…"，真文件是"那是两处对不上…" （错一个字就命中 0 次），干跑先报出来，改对一个字即 29 处全中。

坑六：记录编号是共享资源；编号有第二种写法，计数句还有第二处抄写。 本轮的协作记录脚本先按"当前 62 条 ⇒ 插记录 63"写、再按"已有 66 条 ⇒ 插记录 67"改， **两次都撞车**：落盘前现扫才发现文件里已经有 67 条，而且 66/67 是 `**记录 NN：…**`（加粗行）不是 `### 记录 NN` —— **只按 `### 记录` 统计会少算两条。**

更要紧的是 §5 第 1 条那句"本记录 63 条里有 60 条是自我纠错"：别的会话线两次追加只改了 §2 的标题计数，把这句留在了 63/60。⇒ 追加归档前先**现读**：① 两种写法一起扫出最大编号； ② 找齐所有"条数"字面量；③ 别拿上一轮脚本里的数字照抄。

坑七：构建日志是 UTF-16LE，按 UTF-8 读会静默取不到任何东西。 `tools/run_vivado.ps1 -LogFile _disp_work\impl_disp_u42.log` 产出的文件以 `FF FE` 开头 （387,890 B 里有 193,944 个 NUL 字节）；`_edit15_u42.py` 原本用 `io.open(..., encoding="utf-8", errors="replace")` 读它，拿到的字符串里全是 NUL，正则 `compression saved (\d+) bits` 必然找不到。**这次是那条** `SystemExit("找不到压缩节省比特数")` **救了归档** —— 如果当初写成"取不到就跳过这一句"，docs/15 里就会悄悄少掉"未压缩位流"这一条 而没人发现。修法是先看 BOM 再 `decode("utf-16")`。同源坑：仓库里早有"PowerShell `>` 会写 UTF-16LE"的记录（记录 64），这次是从**给外部程序传文件参数**那一路进来的。

注一（承袭）：xsim 把 `$display` 里的中文写成 U+FFFD。 所以引日志只引坐标 / 计数 / `TEST PASSED/FAILED`（记录 59/60/61/62/63 的注一）。

承袭的两条纪律：① **新判据写完先自问"改回上一版它会不会红"**（记录 59）—— 本轮第 5 行的两条判据（标题墨迹 280 px、数值墨迹 1188 px）在旧版面上必然红：第 5 行 那三项在 3 x 4 的版面里**根本不存在。** ② 判据里的几何从 RTL 常量**现算**（带内 v 24..65 = `P6_VPH 13` + ROM 的 `y_off 11`，不是照图上的目测值）。

教训七条： 1. 预览件 / 镜像 / 生成器都是"另一个实现"：它们与 RTL 的差异本身就是坑，版面改动要靠 证伪判据（旧位置必须"什么都没有"），不要只靠"新位置有东西"。 2. 判据的窗口要一起审：窗口压到别的控件（复选框 / 导航 / 危险区）就会红成假象； 写窗口时先问"这个 y 区间里还有谁在画"。 3. 一个版面常量牵动四处抄写（RTL 常量 / 版面生成器 / 预览镜像 / tb 回读地址）， 改完要逐处找（`grep` 常量名与 RAM 行号）。 4. 新增字段种类 / 状态机分支时，先看分流条件是"枚举比较"还是"编号大小"：后者会在 新编号上静默走错分支。 5. 文字层与数值层要一起设计相位：标题墨迹 v 1..14 与数值墨迹 v 24..65 之间的 9 px 空隙、以及数值距带底 22 px，都不是随手取的数（它们决定"带与带之间是否粘连"）。 6. 归档脚本一律"整块替换 + count 断言 + 失败不写文件"；编号与计数这类**共享字面量** 要现读文件（且要考虑多种写法），不要从上一轮的脚本里继承。 7. 外部工具写出来的日志/报告，读之前先验编码（BOM / 前 16 字节）：命令行 `>` 与 `-LogFile` 都可能写 UTF-16LE；而"取不到内容就跳过"的写法会把缺陷留进归档。

记录 69　命令间隔、虚拟行带与"只有被写脏的那一项会报错"

这一轮做的事：spec §4.6 的下钻路径（`docs/13_display_ui_requirements.md:202`）写着 `—危险区:「清除锁存故障」「恢复默认」⟶ 确认对话框`，而 ⑲ 那轮只做了前一半。本轮补上 后一半：危险区一颗按钮变**两颗对称按钮**，右那颗按下 → 同款二次确认 → 确认后**屏侧自己发 13 条写寄存器命令**（16 个寄存器）把 P6 的 15 项 + 两个标定偏置恢复成复位默认值，再等 2 ms **回读抽头的 17 项**逐项确认（spec §6.2 只许显示回读值），不一致就在那一项的格位画 **红 ✕、不回滚。**屏侧没有 UART 读通路，所以回读只能走 `disp_tap`：抽头里没有的 （`0x0104` 迟滞(UVW) / 方向位 / H2_K）本轮**不写也不承诺。** `rtl/` 与冻结契约一个字没动 —— 屏侧复用的就是上位机那条写通路（`disp/ui_edit.v` 的 4 字节命令打到 `enc_top.uart_rx`）， 而桥的一条命令写**两个连续寄存器**，13 条正好覆盖 16 个。

坑一（本轮最值钱）：命令之间的间隔要按"对端最坏处理时延"算，不能按本端的字节时间算。 `disp/ui_restore.v` 第一版 `GAP_CLK = 4000`（80 µs）：13 条命令之间只空 80 µs。而 `rtl/uart_spi_bridge.v` 收到 4 字节后要走 C_ARM → C_SPI1（内部 SPI 帧 10.56 µs）→ C_REPLY（抢遥测行空闲，最坏 ~100 µs）→ C_TX（6 字节应答 30 µs），全程约 **160 µs**， **这期间它只在 C_IDLE/C_BYTE 里消费 rx** ⇒ 第 2 条及以后的命令**整条被丢掉。** 症状极其隐蔽：只有**被写脏过的那一项**会报不一致 —— 其它寄存器本来就是默认值，写不写 看不出；引擎台/端到端台看到的 `bad=15'h0010` 长得像"限幅那一项写坏了"，其实是"第 4 条 命令根本没被接收"。**修法**：`GAP_CLK = 25000`（500 µs，3 倍余量），序列从 ~3.3 ms

变 ~8.8 ms。**回归钉**（端到端台新加两条）：`cmd_gap_min >= 300 µs` 与"每条命令首字节到达时 桥都在 `C_IDLE`" —— 后者在 80 µs 时会报 12 次，`cs_bad=0` 就是根因的直证。

坑二：虚拟行带号 ≠ 自然格行号，"第 k 项"要用虚拟坐标选行列。 失败 ✕ 的第一版按**自然格行**选行带（`bad_row_k = lab_cy - 6'd4`）。P6 的五个行带是 **虚拟**的：第 k 带的标题墨迹落在自然 y = 64+88k..+15，**横跨 1~2 个自然格行**（第 1 带 = y 152..167 ⇒ 自然格行 9/10）⇒ 只有第 0 带能中，其余四带的 ✕ 永远画不出来。 改成按行带号 `p6_k` + 行带内标题窗口 `p6_lvis`， `bad_bit_i = p6_k*3 + bad_col3` 之后 两条轴才对。

坑三：画框与命中区是两个常量，改一个必须在同一个提交里改另一个。 按钮画框从 `192..640` 挪到 `180..591`，而命中矩形还留在 `184..648`（旧画框 ±8）。 后果不是"边缘差 8 px"：`x 608..647` 是**第 2 颗按钮的左起 40 px**，它会落进第 1 颗的命中 区，而模态 FSM 是 `if (dz_click) … else if (dz_click2 …)` ⇒ 按「恢复默认」弹出 「清除锁存故障」。端到端台为此加了一条**回归钉**：点 (620,552) 必须仍然弹"恢复默认"。

坑四：下降沿数不是字节数。 端到端台第一版用 `always @(negedge enc_uart_rx)` 数 "发了几个字节"，在 52 字节的流里数出 **99**（每个字节内部的数据位也给出下降沿）。 而这套协议里"按线空闲切起始位"也走不通：`disp/ui_restore.v` 把同一条命令的 4 个字节 **连着发**（只隔一个停止位），只有命令之间才有 500 µs。⇒ 字节数由**逐字节比对黄金流** 来钉（`seq_ok == 13`），线上计数器只负责"有没有活动"（`wait_dlg_off` 之后计数必须不再增长）。

坑五：写脏之后要等桥跑完，否则"查早了"会变成"查错了"，还会让后面的判据假绿。 物理 UART 的写命令要经过 `uart_spi_bridge` 的"4 字节（20 µs）+ 内部 SPI 帧（10.56 µs）" 才落到 `reg_file`；端到端台第一版只等 40 拍（800 ns）就查 `tap_ablimit`，于是报 `got 0 want 85`。**危险之处**：这一条如果在写脏那一步"看起来是对的"，后面"恢复后 `tap_ablimit == 0`"会因为本来就 0 而**假绿** —— 判据的失败被另一个失败掩盖。 改成等 2000 拍（40 µs）。

坑六：预览件是另一个实现（承袭记录 68），本轮它两条都犯了。 ① `draw_p6_planes` 把 ✕ 画成 `C_TEXT`（不红）—— RTL 是"只把该格墨迹涂红"； ② **两颗按钮的标签从来没画过**：标签在格行 34，既不在 `draw_plane(c0=0, c1=P6_X0//LAB_PX)`（只画导航竖条）里，也不在 `draw_p6_planes`（只画行 4..8 与值行 1..5）里 ⇒ 静态 P6 图上 是**空框**，而引擎台的 `sc_dz_lbl` / `sc_dz_lbl2` 一直在判它有墨。新增 `_disp_work/_p6chk.py`（把引擎台 ⑥⑦ 段的像素判据在预览图上重跑，约 50 条）就是防这类 "预览件看不见的反馈"；它第一次跑还反过来抓出**我的四条判据写错了**（`(80,599)` 是危险区 下沿的 C_FAULT 不是 C_CARD、`(190,542)` 就在内嵌框竖带上、询问态那 16 px 缝是 C_DIM 不是 C_CARD、`(400,300)` 落在对话框里）。

坑七：错位 72 px 只有人眼看得出来。 第二套对话框的动作对象行 `(19, 29, "REG 0X0100-051A")` 是 15 字 x 16 px = 240 px，从格列 29 起落在 x 464..703（中心 583.5），而对话框中心是 x 512 —— 判据（"这一带有墨"）全绿，人眼一眼就看出偏右。改成从格列 24 起（中心 503.5），并把 判据从"有墨"升级成"有墨 **且** 右侧 x 624..703 必须是空的"。**口诀：字符串长度不同， 起始格列必须跟着变**（kind=0 的 `REG 0X0405` 只有 10 字，中心 544，属 ㊳ 已验收版面， 本轮不动它 —— 不为一处观感去改已验收的图）。

注一（承袭）：xsim 把 `$display` 里的中文写成 U+FFFD（3 个字节的乱码）。 所以引日志只引坐标 / 计数 / `TEST PASSED/FAILED`；本轮解码 xsim 日志用 `_disp_work/_p11_log.py`（依次试 BOM/utf-16/gbk/utf-8，把命中的行写成 UTF-8）与 `_disp_work/_p11_hex.py`（把命中行的十六进制与四种候选解码并排打出来）。

注二：这台机器上没有 `pwsh`。 `pwsh` 报 `CommandNotFoundException`，所有脚本都要用 `powershell -NoProfile -ExecutionPolicy Bypass -File …`；harness 前置的 `[System.Text.UTF8Encoding]::new($false)` 会在 stderr 报 `CannotCreateTypeConstrainedLanguage`（ConstrainedLanguage 限制），**无害**，不是脚本失败。

承袭的两条纪律：① **新判据写完先自问"改回上一版它会不会红"**（记录 59）—— 本轮 `cs_bad == 0` 与 `cmd_gap_min >= 300 µs` 两条在 `GAP_CLK=4000` 上必然红（实测 12 次与 80 µs）； ② 判据里的几何从 RTL 常

量**现算**（✕ 的判据点按"行带号 x 88 + 带内偏移"算， 不照图上的目测值）。

教训七条： 1. 往有状态机的链路上连发命令，间隔按**对端最坏处理时延**算（本轮 160 µs ⇒ 取 500 µs），并且要给"整条命令被丢掉"配一条**能看见它**的判据（到达时对端在不在 IDLE）。 2. 失败编码要按**虚拟坐标**（行带号 / 项号）算，别按屏幕的自然格行 —— 虚拟格网是本项目 反复出现的结构（P5 五列表框、P6 五行都用了），两者混用会"只有第 0 项能中"。 3. 画框、命中区、内嵌框、标签列是**四个常量**：改一个就 grep 一遍其余三个，并让台判 "每颗按钮自己的边缘点"与"按右那颗不许弹左那颗的对话框"。 4. 线上计数（下降沿 / 字节数）不可当协议判据：要么按起始位同步采样，要么逐字节比对； 计数只能用来判"有没有活动 / 活动有没有停"。 5. 写外部器件之后必须等"它真的收下"再查（桥的 20 µs 字节 + 10.56 µs 帧 + 抢行窗口）；"查早了"不只是红，还会让后续判据**假绿**。 6. 预览件 / 生成器 / 判据脚本都要问一遍"它画的东西与 RTL 是不是同源"：本轮预览件少了 一整行按钮标签、✕ 不上色，而 RTL 早就画了 —— 反向自检脚本（`_p6chk.py`）值得写。 7. 像素判据只能证明"有墨 / 没墨"，**居中 / 错位只能靠人眼**：把"有墨"升级成"有墨且 指定区间必须空"是把人眼结论固化成判据的办法。

记录 70 "注释里写清了理由"不等于"欠账被记住"：判据静默少跑，和两个只读审计

这一轮做的事：目标（第 18 轮）要求"把屏幕侧 spec 缺口按四个批次收口"，四批次交付后 我派了**两个只读审计子代理**逐条比 spec 与实现：一个查 §4（信息架构/导航）+ §7（标注）， 一个查 §5（文字与状态）+ §6（交互契约）+ §8（反模式）+ §12（不做清单）。审计一共点出 **20 + 8 条缺口**。其中有一条**纯沙箱就能闭掉**：spec `docs/13_display_ui_requirements.md:173` 写 P2 的入口是「导航「波形」→ Tab「**模拟**」」、`:176` 写 P5 的入口是「P4 表格「**展开**」」， 而屏上一直是 ASCII `SIM` 与 `EXPAND` —— 原因是那四个汉字**不在字库里**。本轮把 **模/拟/展/开** 补进字库（字库 +4 字：51 ASCII + 132 汉字 = 183 码、144,768 bit = 17.67 KB）， 把 spec 的字面写回屏；顺带查出一处**从上一轮起就陈旧的测试台常量**。

坑一（本轮最值钱）：测试台里手写的"规模常量"会让判据静默少跑，而台照样 PASS。
`disp/tb_font_rom.v` 里是 `localparam integer NC = 125;` / `NCH = NA + NC; // 176`。上一轮 （U-43）加了 恢/默/认 三个字，却**忘了改这两个常量** ⇒ 台只遍历前 176 个码位， "最近 3 个字的点阵非空 / 两两不重复"这两条判据**一直没有跑**。台仍然 PASS，日志上 `PASS errors=0` 与真跑过一模一样 —— **"判据少跑"和"判据通过"在日志里长得一样。** 本轮改成 `NC = 132` / `NCH = 183`，并在注释里写明"这两个常量跟着字库走"。**能写成"从数据源数出来"就别写死。**

坑二：spec 与实现的偏差有两种，第二种会变成看不见的欠账。 第一种是"实现做不到"（要改协议 / 碰冻结契约）；第二种是"当年不值得做" —— 比如这里： 两个字不在字库，补字库要动 ROM 与位流。第二种偏差的特点恰恰是**注释里写得清清楚楚** （`tools/gen_text_ram.py` 就写着"这两个字都不在字库里"），于是谁读代码都觉得"事出有因"， 但 **spec 的字面一直没有落实。教训：把偏差的理由写进注释是对的，但要在同一处留下 "补上它的代价"**（本轮量出来是 +1024 bit、3.90 → 3.93 RAMB36），否则下一轮无法判断 "值不值得补"。本轮量完发现代价极小 ⇒ 立刻补。

坑三：审计的证据会过期，引用它要看取证时间。 IA 审计抓证据时 P2 Tab 还是 `SIM`、 P4 按钮还是 `EXPAND`（那时我还没改）； 它的报告里这几行现在读起来像"实现错了"， 其实是"审计快照早于修复"。**审计输出要连时间/版本一起记，否则下一个人会把已修的问题 再修一遍。**

坑四：加字库是"只追加"的操作，所以它的风险全在"别人写死的常量"上。 新字**追加在 `CN_ORDER` 末尾**（`"模拟展开"`）⇒ 已有 128 个汉字的码位一个都不动 （码 = 51 + 序号）⇒ `p2_lab_ram` / `p4_lab_ram` / `p6_lab_ram` / `undo_lab` 里所有既有标签 **一处都不用改**，位流只变大一点点。反过来说：加字之前必须先"量缺口" （`tools/cn_glyphs.py` 的 `C.CHARS` 里有没有这两个字），加完要数一遍三个数 （汉字数 / 总码数 / 总 bit），并 grep 全仓"还写着旧规模"的地方 —— 本轮就是靠这一步 抓到 `tb_font_rom.v` 的旧常量。

坑五：预览件只画文字，页签底色 / 按钮边框是 RTL 画的（承袭记录 68/69）。 本轮改的是"屏上哪两个字"，预览图 `_disp_work/page_p2.png` / `page_p4.png` 一眼可核对； 但"页签矩形在哪、有没有底色"这类信息

预览件里根本没有 ⇒ 人眼核对要分清 "预览看不到"与"屏上没有"，像素判据仍以 RTL 台为准。

承袭的两条纪律：① **新判据写完先自问"改回上一版它会不会红"**（记录 59）—— 本轮 `NCH = 183` 的判据在上一版（`NCH = 176`）上确实**不会红**，这正是它当年漏掉的原因， 所以本轮把"遍历到第几个码"这件事本身也变成判据；② 判据里的数从数据源**现算** （字库数从 `tools/cn_glyphs.py` 的 `CHARS` 数，码位号从 `CN_ORDER` 的序号 + 51 算， 不手抄）。

教训五条： 1. 测试台里的**规模常量/枚举长度**是"第二份真相"：加项时必须与数据源同改，能现算就别写死；更要紧的是——**给"判据有没有跑"本身留一条证据**（本轮改成遍历 183 个码位，少跑就会红）。 2. 记录 spec 偏差时，**同时记下"闭掉它的代价"**；只写理由会让欠账变成"注释里的事"。 3. 只读审计适合"逐条比 spec"这类穷举活；但审计结果要**带时间戳/版本**，并且要先筛 "纯沙箱能闭掉的"（本轮：字面标签）再谈"要裁决的"（spec 内部冲突、已验收版面的取舍）。 4. 加表（字库/枚举/常量表）优先"只追加"：不动已有码位就能保住所有既有版面与判据； 加之前先量缺口、加之后数三个规模数并 grep 旧规模。 5. 目标级验收（`_disp_work/verify_goal18.py`）与轮次级验收（`verify_u44.py`）分开写： 前者判"四批次交付物还在不在"，后者判"本轮改动是否落地"，两者都不许手抄数字。

记录 71　"全局副作用要全局收口"，以及三次"我的验证在骗我"

这一轮做的事：用户接上板子实际用了一轮，提了 14 条意见（原文见 `docs/12_host_ui_requirements.md` 的 R20， 附了 7 张实机截图）。我先做代码定位 —— 其中第 1 条"概览总显示型号未知"是个**真 bug**： `"0x0C" in dev.upper()`， `upper()` 把 `0x0c` 变成 `0X0C`，模式永远不匹配。其余多是"看着别扭"， 但第 8 条（波形一直左右跳）和第 12 条（把平均出来的阶梯消掉、变成标准方波）是**要动机制**的， 所以先算物理约束、再给用户 A/B/C 三选一，他回「C，按此执行」，然后按 A（小改）/ B（字号自适应）/ C（波形）/ D（记录 + 方波）四批落地，每批过两道硬门槛（离线自检 + 真机）。

坑一：全局副作用只修了自己那一份。 字号自适应要重套样式表（`app.setStyleSheet(th.app_qss(scale))`）—— 而样式表和字体是**应用级**的。 我最初把"重开 tabular figures"写在 `MainWindow._apply_scale()` 里，只处理 `self`；结果自检报 `47 个读数标签，其中 46 个 Inter+tnum`。根因是自检里"为量默认版面"新建了**第二个** `MainWindow`， 它重套样式表把先建好的窗口里标签的字体特性全洗掉了。修法：遍历 `app.topLevelWidgets()` 里所有 `MainWindow` 逐个刷新。⇒ 教训：**改应用级状态时，收口也要写在应用级**；"我改的是我这个窗口" 在一个可能开多窗的程序里是错觉。

坑二：离屏平台上，开过第二个窗口之后所有快捷键都失灵。 自检 ⑱ 的四条键盘用例突然全红，我第一反应是"我动了快捷键表"。写了个最小探针才发现： `_kbd_keys` 全是 enabled=True，但 `app.activeWindow()` 是 **None** —— 离屏平台在第二个 `MainWindow` close 之后没有把 active 交还给第一个窗口， `WindowShortcut` 于是全不响应。 测试侧补 `w.activateWindow(); w.raise_(); app.processEvents()` 即恢复。 ⇒ 教训：**先写最小探针，再决定是改代码还是改测试**；"功能坏了"和"平台没给焦点"是两件事。

坑三：用合成数据自测得到"100% 一致"，真机一跑就知道差在哪。 方波判决写完，我用合成帧喂 `AbzAnalyzer(bins=1024, inter=64)`，与理想方波**逐箱一致 100%** —— 差点据此收工。真机（`inter=256`，每周期只有 4 箱）跑出来：平均值 21 档"阶梯"照旧， 判决虽然能在已填箱上到 98.6%，但**覆盖率只有 75/1024**；把 `inter` 降到 64（每周期 16 箱）后 覆盖率 41%、与理想一致 437/439 = 99.5%。⇒ 教训：合成数据只能验证**算法逻辑**， 验证不了**采样密度/覆盖率**这类由物理条件决定的事实；自测的全绿要标清"在什么前提下达成的"。

坑四：约束要先算再承诺，宁给建议不画假图。 一箱 = `inter/bins` 个 A/B 周期 ⇒ 每周期至少 8 箱才看得出方波形状，即 `bins ≥ 8×inter`； 1024 箱时 `inter ≤ 128`。 `inter=256` 时**任何算法都画不出标准方波**（每周期 4 箱）， 所以程序此时给的是"请把 inter 降到 ≤128"的可执行建议，而不是硬拟合一个好看的方波出来。 ⇒ 教训：**先把物理边界算清楚，再决定"能不能做"**；把做不到的原因做成一句可执行的话， 比悄悄画一条假曲线有价值。

坑五：用户说"打不开目录"，而实测那个目录能写。 记录页报 `[Errno 13] Permission denied`，但用户用 PowerShell 往同一目录写文件是**成功**的。 说明不能只用 `os.access()`（Windows「受控文件夹访问」是运行时拦截，ACL 上看不出来）。 改成**真写一个探针文件**再删 + 不可写时自动改存 + 把 PermissionError 翻成人话 （放行 exe / CSV 被 Excel 占用 / 目录只读）。⇒ 教训：**权限问题要用"真做一次"来测**， 而且报错要报到"下一步该干什么"，不能只说"permission denied"。

坑六：用户说的名词未必等于界面上的名词。 他写「"**输出**信号波形"这一界面可以单独放大」，但界面上只有 ⑧「**输入**信号波形」这一张时域波形卡。 我按最接近的那张卡实现（并做了独立大窗口），同时在交付说明里**显式标注这处待确认**， 留一句"如果你指的是编码器页的等效采样图，我再补一个同款弹窗"。 ⇒ 教训：需求里的名词与界面不一致时，**先按最接近的做 + 明写假设**，比停下来等一轮更有效率， 但**假设必须写在用户看得到的地方**。

这一轮照跑的验收：`compileall` exit 0；离线 `selftest_ui.py` **222/222 PASS** （A 173 → B 182 → C 200 → D 222，四批各加一节断言）；真机 `--real-port COM3 --seconds 6` **PASS** （界面取 63,696 帧 = 记录器 63,696 行、文件行数 120,843 = rows、丢帧 1,011、关窗队列丢弃 0、 方波可看 1024 箱 / `inter` 64）。交付包第十一次重打：`8,040,142 B @2026-10-16 22:21:30`、 SHA256 `FB03757F…C841`、439 文件 / 585,307,241 B；PYZ 683 模块（正向 60 串全命中、反向 12 串全无命中）； 包内烟测 exit 0 且日志写着「界面字体：Inter 已装载（4 个文件）」。

3.1 三层结构

```
第 1 层  主智能体（人机对话）
         持有需求、做取舍、写文档、对用户负责
            |
第 2 层  子代理（subagent）— 上下文隔离的专项工作
         例：审计一个文件、挖一段历史、核对一批锚点
         产出"结论 + 证据锚点"，不产出"我觉得"
            |
第 3 层  确定性工具（脚本）— 唯一可作结论的判定来源
         sim/logs/SUMMARY.md、impl_timing.rpt、*_check.py 的退出码、
         真机 selftest 的 PASS n/m、产物时间戳与 SHA256
```

为什么要有第 2 层：一次"审计 23 个文件"的任务如果全在主线上下文里做， 到第 10 个文件时我已经在敷衍自己的检查清单了。分片给独立子代理后， 每片的上下文都足够小，评审质量显著上升（记录 9 的 26 处真缺陷就是这么找出来的）。

为什么必须有第 3 层：模型（包括我）会自我确认。把"结论权"交给退出码与报告文件， 是这套工作流里最重要的一个设计决定。

3.2 子代理的四个使用约定

1. **任务自包含**：子代理看不到主对话，所以提示词里必须给全路径、判据与产出格式。
2. **只做一件事**：一个子代理 = 一个文件或一个主题；不要让它"顺便"做别的。
3. **必须带锚点**：要求它给出 `文件:行号`，并要求它**用 read 逐条复核行号命中**， 不允许只凭 grep 结果就下结论。
4. **审计类子代理禁止改代码**：只出报告，改由主智能体逐条复核后执行（记录 9）。

3.3 上下文管理（长会话的工程化做法）

本项目是长周期开发（30+ 天、多次上板），单次会话上下文必然不够。做法：

- **结论外置**：所有结论写进仓库文件（`docs/DEV_STATUS.md` 是主线流水账，`rtl/SPEC_interface_contract.md` 是接口基准），而不是留在对话里。 ⇒ 会话可以断，工程不断。
- **消费过的长输出压缩成摘要**：长的工具输出（仿真日志、PDF 提取文本）在被消费后 压成"结论 + 锚点 + 文件路径"，需要时再回原始文件查。
- **每个动作留时间戳与哈希**：位流、报告、日志都带时间戳，`git log` 提供另一条时间轴。

3.4 本工作流的两个真实代价（诚实记录）

1. **子代理会误报**：记录 9 里子代理报的疑点**多于**26 条，多出来的是误报。 ⇒ 复核环节不可省，不能用"子代理说有问题"直接改代码。
2. **AI 会写出看起来对、实际上是错的文档**：例如把"总输入延迟 19.5…24 ns"当成 AD9238 手册的 `tPD`（真实 `tPD` 是 2…6 ns）写进 5 处文档。 发现方式不是再审一遍文档，而是**回到一手资料**（手册）与**一手实测**（示波器）。 ⇒ 所以本项目有一条硬规定：**报告里的每个数字旁必须标 `[实测]` / `[仿真]` / `[手册]`。**

记录 72　构建环境的痕迹会把交付物降权：一个「软件坏了」的现场，真凶在工作区 ACL 上

现场：用户两轮反馈「记录页记录不进去、打不开文件夹」，报 `[Errno 13] Permission denied`；而他用（管理员）PowerShell 手动往**同一个目录**写文件是成功的。第一轮我按"权限猜谜"修：探针写测试、自动改存、人话原因、`打开文件夹` 三级兜底、导出前探写 —— 都对，但**都没碰到真因**，用户第二轮回传「还是不行」。

差点写进文档的错误结论：我给诊断报告加了"沙箱指纹 = 有无 `DSH_*` 环境变量"，用户那份报告显示"无 DSH_*环境变量" ⇒ 我当时准备据此写"确认是普通双击运行、问题在系统防护"。但同一份报告里"备选目录"五条全是拒绝访问 —— 一个普通双击的进程不该连自己桌面的新建文件夹都被拒。环境变量只是启动器给的指纹，不是运行机制的证据*（启动器完全可以不传）。

换成机制证据，一次就定性：用 ctypes 读进程令牌（`TokenHasRestrictions` / `TokenRestrictedSids` / `TokenIntegrityLevel` / `TokenElevation` / `TokenSessionId`）+ `QueryFullProcessImageNameW` 取父进程名；再用 `Win32_Process` 查父进程。结果：父进程**确实是 `explorer.exe`**（真的双击了），但*运行中的* `host_app.exe` 是 **Low 完整性 + 受限令牌**（`S-1-16-4096`、`has_restrictions=1`）。

真凶：工作区根目录带着 `Mandatory Label\Low Mandatory Level:(OI)(CI)(NW)` 和一个 `S-1-4-…` 沙箱 SID 的 ACE —— 这是**构建环境**（受限令牌 + 工作区 ACL 标记）留下的痕迹；PyInstaller 生成包目录与 exe 时把这些**继承**了下去，而 Windows 会按镜像文件上的低标签给进程降权 ⇒ **交付的 exe 一启动就是低完整性进程**，只能写标签允许的位置（程序旁边那个目录行，我的文档/桌面/AppData 一律拒绝）。用户"管理员能写、程序不能写"也就顺理成章。

修法：对工作区做 `icacls <root> /reset /T`（32,799 个对象）+ `icacls <root> /setintegritylevel (OI)(CI)M /T`；复核 exe 标签回到 `Medium`、整树 Low 标签与沙箱 ACE 计数**归零**。产品侧另加两道与环境无关的防线：`env_warning()` 在检测到低完整性/受限令牌时**直接在记录卡上说明并给出"从资源管理器双击运行"的建议**（不让用户对着一串"拒绝访问"猜），`write_diagnosis()` 增加「进程身份」段。

三条教训：1. **交付物的属性要量、不要推断。**我当时已经差一步把"确认普通双击运行"写进报告——如果写了，下一轮就会朝系统防护方向白查一天。凡是"环境正常/用户操作正常"的判断，都要落到一个可测量的量上（令牌、标签、ACL、父进程）。 2. **工具的指纹 ≠ 工具的机制。**环境变量能看出"是谁拉起来的"，看不出"以什么权限跑"；后者要读令牌。排查"权限类"问题，优先拿机制证据。 3. **构建/运行环境会在产物上留下可继承的痕迹**（ACL、强制标签）。交付前应检查**镜像文件自身的标签与 ACL**，这条已加进打包清单（`icacls <exe>` 必须是默认标签、不含可疑 SID）。

追记（2026-10-17）：用户问「怎么样才能不让问题重演？」—— 我意识到上面第 3 条写的是"加进打包清单"，而**清单靠人记得，正是这次翻车的前提**（我就是那个"记得"的人，却在受限令牌下打了包、直接把产物交了出去）。于是把它改成**两道可执行门禁**：`tools/build_host_onedir.ps1` 在打包**前**读当前终端令牌，见 `Mandatory Label\Low` 或 `S-1-4-…` 就**拒绝打包**（要硬来必须显式 `-Force`）；`tools/check_package.ps1` 在打包后量四处标签（exe / 交付目录 / 包内树 / 工作区根）+ 文件数 + SHA256 + PYZ 复核 + 包内烟测，FAIL 即 exit 1。教训落成一句可执行的规矩：**"记得"不是防线，"跑不过就交不出去"才是。**（门禁自己的坑也记进了 `docs/DEV_STATUS.md` §1.91.3：PowerShell 5.1 上 `Start-Process -PassThru` 把成功退出读成空串；`.ps1` 丢 BOM 会被 5.1 按 GBK 解码并报"指向正常行"的假语法错；★ 还有一条最像本轮教训的——我给烟测输出指定 UTF-8，结果把 exe 正常写的 `截图成功：…` 解成了 `��ɕ�ɹ���`，量了字节才发现那个 exe 写的是 **ANSI 码页。别替别人的输出指定编码，先量它的字节。**）

记录 73　"全面自查"里最值钱的不是修了多少 bug，而是我请了一个专门证伪的验证员、以及发现门禁自己坏了

现场：用户要求"全面自查上位机软件，检查功能实现程度，稳定性，检查并修复可能存在的 bug"。这是一个"开放性正确"的任务：没有具体现象，很容易变成"读一遍代码写一份好评"。我把它拆成四路只读审计（链路/协议、算法/面板、生命周期·记录·导出、需求追溯）+ 我自己认领的两个角度（`host` 寄存器语义与帧布局 ↔ RTL 交叉核对、自检断言强度）。

第一件对的事：把"审计"变成"可判定的指控"。我要求每个子代理只读、禁写工作区、每条结论必须给 `文件:行号` 并标注【确定】/【需验证】。结果是 40 余条指控 —— 但它们**不能直接采信**：上一轮（记录 71）才刚吃过"子代理报告里混着假指控"的亏。

第二件对的事：专设一名"反方验证员"。它的任务不是复述、不是补充，而是**证伪**：自己写探针跑，然后给我"哪些指控是真的、哪些是报告说错了"。它交了 12 条判定，其中 **6 条被证伪**：解析器"每轮只前进 1 字节的活锁"（实测一次 `resync` 吃掉整段垃圾）、"读线程异常会静默死亡"（`try/except/finally` 明明兜住了）、"先 stop 再 drain 丢尾帧"（现有顺序是有意为之，旧顺序实测才丢 1~2 批）、"告警判定被稳定值放宽"（`r` 只进文案，判定来自板上 `io_flags`）。**★ 这 6 条要是照单全改，就是 6 次把好代码改坏。**教训：**"多个独立审计者都提了"不等于"真"；要给证伪留一个正式席位，而不是让它发生在最后一刻的直觉里。**

第三件对的事：审计"审计工具"本身。四路都在读产品代码，没人问"那 255 项 PASS 到底在断言什么"。我去核了自检的判据，当场发现**两个离线自检长期恒假失败**：`export_png.py` 的像素判据里写死着一份**旧调色板**（`primary=(46,125,209)`），而画图侧早就改成从 `theme.C` 取色（`#0071E3`）；`recorder.py` 的 [6] 条还在断言"目录不可写 ⇒ 报 error"，而阶段 5 D 批起记录器**先探写再自动改存**。两个断言都"过期"了 —— README 里挂着七条 `--selftest` 命令，历轮只跑 `selftest_ui`，**从来没人一条条跑。**所以我给它们补上真判据，并把清单变成门禁：新增 `tools/check_offline.ps1`（`compileall` + 七模块自检 + 界面级 277 项，任一项失败 `exit 1`）。

最有价值的三个真发现（都不是靠"读代码"读出来的，是量出来的）：
* ⑨ 独立窗口**整条链路是坏的**：`main_window` 建完窗就 `p.notice.connect(...)`，而 `panel_abz.AbzPanel` **没有 `notice` 信号** ⇒ `AttributeError` 在 Qt 槽里被打印到 stderr 后吞掉：五个信号一个都没接上；更隐蔽的是窗口**已经建好**，第二次点击只会 `show()` 一个空壳。**"自检点过这个按钮、而且是绿的"** —— 因为断言只看了"没抛异常"（异常被 Qt 吞了），没看"窗口真的可见、信号真的接上"。
* **提示文案停在 v8 语义**（最像"会害用户改错硬件"的一条）：诊断页写着 AB 迟滞"0 = 用默认档"、Z 脉宽"0 = 不出 Z"，而 v9 起分别变成"0 = 不启用"和"0 也出 1 步"（`rtl/abz_gen.v:120`、`:130`、`:519-520`）。用户照着提示调，会去动本来正确的硬件 —— **文案也是交付物的一部分，得跟固件一起审。**
* **两条记忆项"写了等于没写"**：`ui_prefs.DEFAULTS` 里没有 `ui` 段和 `wave.trig` 键，而合并只遍历默认键 ⇒ 缩放字号与波形触发源每次启动回默认值。旧自检抓不到，是因为它

在**同一个会话里** `put` 完立刻 `get`（内存往返永远成功）。修法不只是补键，还加了一条**静态交叉核对**：正则扫 `host/*.py` 里所有 `up.put/number/choice` 的段/键，不在 `DEFAULTS` 里就报出来 —— 把"记住"变成"跑不过就报"。

四条教训： 1. **开放性任务要自己造边界**：把"全面自查"翻译成"四路分工 + 每条可复核 + 有没有人负责证伪"，否则它就是一次自我表扬。 2. **给证伪一个正式席位**：反方验证员不是"再读一遍"，而是"专门找错"。本轮它删掉 6 条假指控 —— 价值不低于修掉 6 个真 bug。 3. **审计要往上盖一层**：产品代码之外，还要审"我用来判定产品没问题的那些工具"（自检、门禁、脚本）。**门禁自己坏了，比产品坏了更危险**：它会让后续每一轮都得出"没问题"的结论。 4. **同源坑会以不同面貌出现**：上一轮是"给烟测输出指定 UTF-8，把 ANSI 码页的输出解成乱码"（记录 72 追记），这一轮是"自检在同会话内存里往返"，本质都是**用想当然的等价代替真实路径。**凡"验证"都要问一句：它和我担心的那个真实场景，差在哪一层?

数字：修掉 19 类真问题、证伪 6 类；门槛 `compileall` exit 0 + 七模块自检 + `selftest_ui` **277/277 PASS**；需求追溯 R1–R20 = 已实现 61 / 部分 5 / 未实现 0 / 已退场 4（**R20 十八项全落地**），6 处 `docs/12` 状态列过期已更正；交付包**已重打（第十四次，2026-10-17 08:13）**： exe `8,067,586 B`、SHA256 `72825284…8ADB3`、目录 439 文件 / 585,334,685 B， `tools\check_package.ps1` 门禁 PASS。详见 `docs/DEV_STATUS.md` §1.92 与 `docs/16_host_ui_refactor_R16_R20.md` §13.10。

线	状态	说明
7 寸屏本机界面	**需求已收集，未实现**	原话在 `_synth_work/prompt_disp.md`；用户明确要求"先沟通确认，再做下一步执行"；需求文档 `docs/13_display_ui_requirements.md` 已成文
千兆以太网上位机通道	**可行性分析完成，未实现**	`docs/dual-channel-host-feasibility.md`（v2）；结论：技术上可行（复用官方 RGMII 层，3–5 天），但**不解决 10 kSa/s 这个根问题**，且只落在"性能与资源优化效果（20 分）"那格

记录 74　"端口表少一个名字"不会报错：隐式线、`force` 过的内部状态、撑不住一个周期的新组合路，与两条只在真机上看不见的判据

这一轮做的事：目标（第 18 轮）收口四批次之后，用户选的路是"继续按审计缺口往下收"， 先做 §6.3 的**分频八档步进**（㊶）：面板此前**只改倍频**，分频从头到尾原样带回读值写回 （`disp/ui_panel.v` 的文件头把这条偏差写得很清楚）。本轮给面板加了一颗 **64×64 的字段切换 按钮**（×倍频 ⇄ ÷分频），切过去之后**同一套控件**（[−]/[+]、8 档滑条、长按加速、数字键盘） 全部作用于分频； `disp/ui_edit.v` 的回读确认链路其实早就比 `tap_inter` + `tap_div` 两个字， 缺的只是 UI 与 `req_div_code` 的来源。判据分五层: `disp/tb_ui_panel.v`（46 条交互）、 `disp/tb_undo_lab.v`（9 格标签）、 `disp/tb_disp_engine.v` §[9b]（像素、×与÷图标、值窗墨量）、 `disp/tb_enc_disp_panel.v` 第 12 节（端到端 ÷ 走一档 → APPLY → BACK 回原值）、 `disp/tb_val_format.v`。

坑一（本轮最值钱，也是真缺陷）：端口连接里写一个不存在**的名字，Verilog 只给隐式线警告。** 我给 `disp/undo_lab.v` 加了新端口 `div`，也在 `disp/disp_engine.v` 的例化里写了 `.div(pnl_undo_div)` —— 但**忘了往端口表里加** `pnl_undo_div`。于是它成了一根恒 0 的 1 bit 隐式线，**撤销标签永远写** `÷1`，而所有台（包括像素判据）都 PASS：因为判据只查前 7 格， 那个 `÷M` 是第 8/9 格。**教训：新端口是"端口表 + 例化 + 测试台"三处同时出现，改完要 grep 的是**声明（`input wire [2:0] pnl_undo_div`），**不是例化点；"编译通过"对隐式线毫无约束力。**

坑二：`force` 过的内部状态，`release` 之后不会自己回去。 `page` 是 `enc_top_disp` 的内部 状态，只在翻页事件里被赋值； `disp/tb_enc_disp_panel.v` 第 11 段 `force u_dut.page = 3'd5` （去 P6）之后 `release`，第 12 段起点**其实还在 P6** ⇒ 面板根本开不了，第一条 FAIL `12 起点仍在 P1 : got 5 want 0` 后面挂了 15 条连锁。修法是第 12 段自己 `force u_dut.page = 3'd0; repeat (4) @(posedge sys_clk);` 并在段末 `release`。 **教训：**

台里 force 内层状态必须成对（段首 force 到本段需要的值、段末 release）；红了先读 第一条 FAIL，别从后面十五条反推。

坑三：一条写命令同时写两个寄存器时，"只改一个字段"是设计责任，不是默认行为。 `disp/ui_edit.v` 的 `pack_cmd` 用一条 4 字节命令同时写 0x0101（倍频）与 0x0102（分频）； 面板旧写法是"打开时把待应用倍频**吸附**到不大于回读值的最大档"，于是**只改分频**时会把 回读的 100 顺手写成 64。本轮把开面板改成 `pnl_inter <= tap_inter`（回读真值，不吸附）， 滑条游标另算一个"显示用的档"。**教训：当写通路的原子性是"两个字一起写"时，UI 必须能表达 "其中一个保持不变"** —— 这一条也成了本轮最核心的判据（APPLY 时 `req_inter` 必须还是 100）。

坑四：复用是最省的改法，但得先证明"两边语义真的一样"。 面板的待应用值窗（×256 / ×64 / ÷7）直接复用了数字键盘那 3 个 48×80 七段大字段（`disp/seg7.v`）与整块 `KPV_*` 窗口， 只加一层 mux：省掉 3 个大字段各自的笔画比较，也**没有引入乘法器**（八档仍靠移位/查表）。 代价写在判据上：值窗墨量只能判**相对关系**（×256 > ×64 > ÷7 且 ÷7 > 0），不能写死像素数。

坑五：版面"加一位"（7 格 → 9 格）是跨层改动。 `disp/undo_lab.v` 的 `LAB_N` 由 `4'd6` 改成 `4'd8`、`cell_at` 加 k7/k8、文件头表格重写； `disp/tb_disp_engine.v` 的 `exp_lab` 由 7 格 扩成 9 格、`check_lab` 的循环上限 `7 → 9`，并新增第 4 个用例（×64 ÷8）。**判据不跟着扩就等于 没判** —— 与记录 70 的坑一同源（"判据少跑"和"判据通过"在日志里长得一样）。**教训：加一位要 顺着"模块 → 期望数组 → 循环上限 → 打印文字"四步一起改。**

坑六：往既有像素链上挂一个新值，面积看不出问题、时序一眼见血；而且这条路上连栽三版。 把面板待应用值接进数值窗之后， `u_ui_panel/pnl_inter_reg → pv_iv(256 特例 mux) → 减法拆位链 → win_d*/win_b* → 3 个 48×80 seg7 → pnl_col 颜色优先级链 → 调色板 → lcd_rgb` 成了一条组合路： `sys_clk` **WNS = −2.923 ns**、TNS −51.301、**19 条失败端点**（全部是 `u_ui_panel/pnl_inter_reg[2]/C → u_disp/lcd_rgb_reg[*]/D`），而资源只涨 296 LUT（13,681 → 13,977）。 ① 把拆位结果**打一拍**再进 seg7 ⇒ 仍 **−0.570 ns**（最差路径变成 `u_ui_panel/pnl_fld_reg/C → u_disp/pvd_t_reg[0]/D`，20.578 ns、37 级，其中 12.014 ns 是布线）； ② 再把待显示的数也锁一拍（`pv_num`）⇒ 仍 **−0.459 ns**（`u_disp/pv_num_reg[1]/C → u_disp/pvd_r_reg[3]/D`，20.472 ns、**32 级、11 个 CARRY4**）—— 这时才看清：**串行减法链 （3 次减 100 + 10 次减 10）自己就有 32 级，跟跨模块无关**；③ 最终把拆位改成**并行比较** （百位靠两个比较、十位靠 9 个并行比较、个位用 `10·t = t<<3 + t<<1`），没有串行链、没有 `*` 与 `/`， 算法先在 Python 里全量核对（`v ∈ 0..256 all ok`，`v ≥ 300` 才错，符合面板值 ≤ 256 的前提）。 **教训："只有面板打开时才走"的逻辑照样会被综合成关键路径；新值接进像素链前先数它到 `lcd_rgb` 有几级组合；写位拆分别用 `for` 减法链 —— 那是 32 级。**

承袭的两条纪律：① 新判据写完先自问"改回上一版它会不会红"（记录 59）； ② 判据里的数从 数据源现算（分频码 = 寄存器值，`÷1` ⇒ 0，从 `disp/ui_edit.v` 的 `pack_cmd` 口径来）。

教训五条： 1. 新端口 = **端口表 + 例化 + 测试台**三处；改完 grep 的是声明，隐式线不会被编译拦住。 2. 台里 `force` 内层状态要成对（段首 force、段末 release 或 force 回原值）；红了先看第一条 FAIL。 3. 写通路"一次写两个字"时，UI 要能表达"另一个不变"；这条要变成判据（本轮是 `req_inter`）。 4. 复用既有通道先证语义一致；判据用相对关系（比大小）替代写死的像素数。 5. 新值接进像素链前先数组合级数；本轮连栽三版才明白：打一拍不够、锁输入也不够，**位拆分本身要改成并行比较**（`for` 减法链是 32 级）。

记录 75　"抽头里有信号"不等于"显示侧消费了它"：一块屏自己和自己矛盾了四十多轮

这一轮做的事：接着做审计缺口里的第二条 —— spec §8-8 的**空态**（㊷，v45）。spec 只有一句话（`docs/13_display_ui_requirements.md:351`「没信号时屏幕一片空白 / 一片 `0`」），口径由本轮定： 链路不可信时，四个**测量**字段整格显示 `-`（小数点保留、骨架还在）， 配置回读（字段 1 倍频分频） 与逻辑电平（字段 5 ABZ/UVW）**不动** —— 它们不是测量，链路坏了它们照样是真值； 四个测量卡的迷你状态条同时转 ✕ 故障。口径最后落在 `sig_bad` 里的链路锁存位（**只认链路完整性**，不含信号质量位 —— 见坑二）。

坑一（本轮最值钱）：抽头里一直有链路校验位，显示侧却从没接过它。 `disp/enc_disp_tap.v` 把 375 位抽头拆成几十根线，其中链路校验位（抽头 [126]， 编码器侧的 `link_err_seen`，契约写作"链路校验错"，RTL 的置位条件是 "帧尾之后主机仍在打时钟"的**协议违例**；**sticky，只有复位能清**）从我接手之前就在那儿；`disp/enc_top_disp.v:184-186` 也早有 `wire tap_link_err;`。但它从来没有接进 `disp_engine`。 于是出现一块**自己和自己矛盾**的屏：顶栏告警灯用的 `alert_fault = q_stat[0] | q_io_flags[7] | q_io_flags[6]` 里早就含 amp_bad/cor_sat，链路一直违例时**顶栏报故障**；而六张卡的状态条只查 lag 两条判据，所以**卡片仍 全是 ● 正常**、P1 还在显示 `0.00 Hz`。这个矛盾在屏上摆了四十多轮，没有任何测试台会抱怨它 —— 因为没有任何判据同时看"顶栏"和"卡片"。 **教训：加"数据有效"这类信号之前，先 grep 目标模块的端口表和例化点。抽头（tap）是接口， 不等于被消费；`wire` 声明在那里躺多久都不会有人提醒你。 顺带露出来的老账**：这个位是 sticky 且**没有 W1C 清位路径**（编码器侧 只有复位分支清、置位靠编码器侧；`rtl/reg_file.v:528` 的 `0x053A` 那位是直读） ⇒ 真机上链路违例**一次**，屏上就一直 `-`，只能复位/重新配置编码器。 接上显示侧之后这条老账才显形（记进 U-46.8 第 4 条）。

坑二（首跑 44 条红换来的）：层级不同的判据不能合成一条。 我一开始把口径写成 `sig_bad = link_err_seen | q_io_flags[6] | q_io_flags[7]`（链路校验 | 相关器饱和 | 幅值坏），理由听着很顺："与顶栏告警灯同一口径，屏上不会自相矛盾"。结果既有的 §[11] 轮 B 判据 （`chip_st = 12'h9A5`，要求"三种状态同屏"：卡 3 故障于 `amp_bad`、卡 4 故障于 `cor_sat`、 卡 1/卡 5 只是警示）实测成了 `12'hAA6` —— 四张测量卡**一起**变故障，屏上信息反而变少。 `amp_bad` / `cor_sat` 是**信号质量**，本来就各有归属（`st_rpm` 认 `amp_bad`、 `st_angle` 认 `cor_sat`）； 链路校验位是**链路完整性**（整帧没过校验），跟"是哪张卡的问题"无关。⇒ 口径收窄成 `sig_bad` 只认链路锁存位，并在 §[11b] 加**轮 B2**（只拉 amp_bad+cor_sat、链路好 ⇒ 读数**不许**空态，只有卡 3/卡 4 故障）把这条区分钉死。 **教训：新判据第一次跑必须单台跑。** 我直接把 §[11b] 交给全量回归，第一次执行就是 794 s 的整轮； 而且我引用的"引擎台 PASS"是上一轮（那时还没有 §[11b]）的证据。

坑三：同一个文件里两块 RAM 的格地址布局不同。 §[11b] 的值层判据要读 `u_dut.u_p1_val_ram.mem[...]`，我照着 [10]/[11] 读 lab 层的写法写了 `mem[{vrow, vcol}]`（两个 6 bit）。但 `disp/text_ram.v` 里： `p1_val_ram` 是 `mem[0:319]`、 地址 = **行×32 + 列**（`disp/disp_engine.v:930-932` 的 `{val_cy, val_cx}`，行 4 bit、列 5 bit）； `p1_lab_ram` 是 `mem[0:2431]`、地址 = 行×64 + 列。⇒ 行 2 读到**别的格**（`got 0`）、 行 7 直接**越界成 X**（`got X`），一共 32 条红。**教训：读别人的存储器之前先看它的 声明（数组长度 + 地址拼法），别照抄邻近代码。**

坑四：空态判据必须双向，否则"恒空"也能全绿。 `disp/tb_disp_engine.v` 新段 §[11b] 分四轮：轮 A 先记基线（还把 P1 输入频率字段的 **数字个数与 `-` 个数**都数出来，要求"0 个 `-` 且至少 3 个数字"，并把 9 格字符码打印进日志 —— 这一条专门防"val_format 恒输出 `-`"把空态判据骗绿）； 轮 B 拉高 `t_crc` 再验"四个测量字段全 `-`、小数点还在、字段 1/5 逐格与基线相同、状态条 **✕**、卡 2/卡 6 仍 ●"； 轮 B2 验口径区分；轮 C 撤销 `t_crc`，验**回到数字**。只做轮 B 的话，把 val_format 改成 "永远输出 `-`"也能过。`disp/tb_val_format.v` 同样加了"面板开着 + nodata ⇒ 字段 1 仍显示待应用值 `*128 d1`"这一条。

坑四之二（第三次跑剩 5 条红换来的）：门槛要按"默认工况的实测显示"定，活数据字段不能 "逐格快照相等"。 第三次跑从 44 条降到 5 条，剩下的**全是判据自己的毛病**： ① 我把基线写成"数字字符 ≥4"，可 tb 默认的频率遥测是 0，屏上是 `0.00`（**3** 个数字 + 1 个点） ⇒ 判据自己红；② 字段 5（ABZ/UVW 逻辑电平）是**活数据**：A/B/Z 三个电平位跟着编码器模型的 边沿走，我拿轮 A 的快照逐格比，电平位从 0 翻到 1 就变成"空态动了字段 5"两条红。 改法：基线判据换成"**一个 `-` 都没有**"（`-` 才是空态的唯一硬标志；数字个数只是"像读数"的 软证据，门槛按实测 3 个数字定）； 字段 5 改成**分层判** —— 基线是数字位就要求"现在**仍是** 数字"（值可变），基线是字母/空格就要求**一字不动。 教训：判据的阈值要拿默认工况的实测值来定；快照比较只适用于常量字段（配置回读）， 对活数据要判"性质"而不是判"数值"。**

坑五：顺手勘误也算改文档。 `disp/stat_chip.v` 文件头写着"反转/闪烁本版明确不做" —— 那是 v31/v32 之前的注释，而 `scbar_inv`（v31 反转）/ `scbar_blk`（v32 闪烁）**恰恰就落在本模块输出的状态条上**

（`disp/disp_engine.v:2354-2362` 读 `sc_chip_st`）。本轮加 `sig_bad` 时把这句改成勘误说明： 过时注释会误导下一轮的人（我这次就差点以为"反转/闪烁"还要自己实现）。

顺手记的一条边界：`disp/p5_view.v:199-203` 的 `99999` **不是**空态哨兵，而是进位计数在 16 位 BCD 通路（double-dabble 16 次迭代）上的饱和点。P5/P2 的"空态"因此得另定口径 （记进 U-46.8，下一轮做）。

教训六条： 1. "抽头里有"≠"被消费了"：加端口前 grep 目标模块的**端口表 + 例化点**，别只看抽头。 2. **层级不同的判据不合成一条**：链路完整性（CRC）与信号质量（cor_sat/amp_bad）各有归属， 合并会把"每张卡各自的口径"抹平（首跑 44 条红就是这么来的）。 3. **新判据第一次跑必须单台跑**：别把从没执行过的段落直接交给几百秒的全量回归。 4. 空态判据要**双向 + 基线**：进入判 `-`、退出判数字、基线里先要求"原本一个 `-` 都没有"； 阈值按**默认工况的实测显示**定（tb 默认频率是 0 ⇒ 屏上 `0.00` 只有 3 个数字）。 5. 读别人的存储器先看**声明**（数组长度 + 地址拼法）：同一个文件里值层（行×32+列）与 lab 层（行×64+列）就是两种布局。 6. 一块屏自相矛盾就是缺陷信号（顶栏报故障 / 卡片报正常）⇒ 先找判据表的口径分歧； 顺手把过时注释改成勘误也算交付。 7. 活数据字段（A/B/Z 电平、计数器）不能用"逐格快照相等"判：分成"结构不变"（字母/空格一字不动）与"性质不变"（数字位仍是数字）两层。

记录 76　"没有那一位"和"那一位没接"是两种缺口：停摆空态只能用已有量的动态当代理

这一轮做的事：接着做 spec §8-8 空态的**另一半** —— **编码器停摆**（㊸，v46 起）。 ㊷ 只覆盖"帧来了但没过校验"（链路校验）：`sig_bad` 只认链路锁存位。可审计原话是 "编码器停摆时 P1 仍显示 ● 正常 + `0.00 Hz`" —— 这一半链路判据覆盖不了：**一帧都没来**时 链路校验位是干净的（它是"帧尾之后主机还在打时钟"的协议违例标志，与"根本没帧"无关）。 本轮口径：`sig_bad = link_err_seen | amp_gone`，其中 `amp_gone = (tap_amp_sq < tap_amp_low_sq)`（载波幅值掉到下限以下），四张测量卡同时 ✕ 故障。 `rtl/` **一字未动、没有新端口、没有改契约。**

坑一（本轮最值钱）：抽头里"没有那一位"和"那一位没接"是两种完全不同的缺口。 ㊷ 的教训是"抽头里有、显示侧没接"（grep 端口表 + 例化点就能发现）。这一轮反过来： `disp/enc_disp_tap.v` 的 375 位里**根本没有**"数据有效 / 心跳 / 帧计数"这一类位 （编码器侧的 `frame_cnt` 是"本次帧已发送了多少位"，`TIMEOUT_CYCLES = 10` 只复位 内部帧状态机，都与链路活动无关）。而 `disp/enc_disp_tap.v:11-13` 明令这个文件里不许出现任何 `<=` 或组合运算（否则算接口变更），契约 `rtl/SPEC_interface_contract.md:691-700` 又规定抽头总线 是"只读快照、只许放已在寄存器里的既有信号"。⇒ 想在显示侧判"编码器还在不在说话"，**不能用 心跳位，只能用已有量的动态当代理**：本板的角度由载波解调而来，载波消失时 `0x0530` 的幅值会掉到 `0x0520` 下限以下 —— 这是抽头里唯一现成的"停摆证据"。 **教训：缺口分两层 —— "位没接"是接线问题，"位不存在"是设计问题；后者要么改契约（本轮明确 不做），要么用已有量的动态把同一件事推出来，并把推不出来的一部分（纯链路静默）写进"仍未做"。**

坑二：判据要用"值"，不能用"标志"。 最省事的写法是直接用 `io_flags[7]`（`amp_bad`）。但契约 `rtl/SPEC_interface_contract.md:705-744` 写的是 `[128] amp_bad = amp_low | amp_high` —— 它把"幅值**偏高**"也算坏信号（`rtl/amp_monitor.v` 低侧高侧各一个门限），而且 ㊷ 的 §[11b] **轮 B2** 已经把"链路好、只有 amp_bad/cor_sat 标志为真 ⇒ 读数照常显示、只有卡 3/卡 4 故障"钉死了。若用标志当判据，这条既有判据当场打红。 ⇒ 判据落在**值**上：`tap_amp_sq < tap_amp_low_sq`，只吃**低侧**，与 `rtl/amp_monitor.v:95` 的 `w_low = (amp32 < amp_low_sq)` 同口径（**严格小于**；门限写 0 等于关掉判据，也是既有约定）。 **教训：同一件事在抽头里往往既有"值"又有"标志"，标志是给人看的聚合结论（可能带高侧）， 判据用值才能控制到"只认哪一种坏"。**

坑三：三条边界必须各占一轮，否则判据会被"看起来对"骗过去。 `disp/tb_disp_engine.v` §[11b] 这轮加了**三轮**，每一轮都对应一种"能骗绿"的错法： ① **轮 D2（门限边界）**：幅值**恰好等于**下限时不算没信号（判据是严格小于）—— 写成 `<=` 就红； ② **轮 E（静止 ≠ 停摆）**：编码器停住不动、载波还在、幅值在窗口内 ⇒ 读数**照常显示** —— 用"值不变"当停摆判据就红（这是最容易顺手写错的一条：屏上看不出"卡住"和"没有"的

区别， 只能看载波）； ③ **轮 C 的回程**（㊷ 已有）：撤销刺激后必须**自己回到数字**，防"一旦空态就锁死"。加上 ㊷ 轮 A 的基线（"原本一个 `-` 都没有"）与轮 B2 的口径区分，§[11b] 现在一共 6 轮。 **教训：新判据每想到一种"错法"，就补一轮专治它的刺激；判据的行数比刺激的轮数更能说明测试的 成熟度。**

坑四：tb 里那个刺激量原本是"例化处写死的常量"。 `disp/tb_disp_engine.v:191` 原来把 `.tap_amp_sq (32'h2000_0000)`（= 0.5 Q2.30）直接写死在 例化端口上（`:195/**196` 的上下限分别是 `0x170A_3D71`(0.6²) 与 `0x5C28_F5C3`(1.2²)）。 刺激必须在运行中变，所以先把它提成 `reg [31:0] t_amp_sq = 32'h2000_0000;` 再接线。 **默认值本身就是判据的一部分**：0.5 落在 (0.36, 1.44) 里 ⇒ 新判据在既有 25 台的默认态 **恒不触发**（否则一改就是全量回归一片红）。**教训：加"值层"刺激前先确认它的默认值在合法窗口 内 —— 默认值写错的表现不是"这一段红"，而是"所有旧段落一起红"。**

坑五：归档脚本的机械替换要带断言，序列锚点不能按"值相同"写。 `_mk_u47_numbers.py` 逐条替换并断言"命中恰好 1 次"（13 处）。两个真踩到的坑： ① `_edit15_u46.py` 的 LUT 序列锚点上一轮写成"末尾两版同值"（`14,064 / 14,064`），那是 U-43/U-44 恰好同为 13,681 的巧合，换一轮就 0 命中 ⇒ 锚点改成"**按收尾值等于上一轮**挑那一处"； ② 空态分支在 `disp/val_format.v:375-378` 写的是 `if/else` 两行（不是三元式）， `verify_u46.py` 一开始按三元式判，判据自己红。 **教训：文档里的"序列"要按结构锚定（收尾值/结尾形状），不要按"某两版数值恰好相同"锚定； 验收脚本读源码形状时先看它是 if/else 还是三元式。**

教训六条： 1. 缺口分两层：**"位没接"**（grep 端口表 + 例化点）与 **"位不存在"**（要么改契约、要么用已有量的 动态推代理，并把推不出来的部分写进"仍未做"）。 2. 判据优先用**值**而不是**标志**：标志是聚合结论（`amp_bad = amp_low | amp_high` 还带高侧）， 用它能当场打穿上一条口径（轮 B2）。 3. **静止 ≠ 停摆**：判"有没有信号"要看载波/心跳，不能看"值变不变"。 4. 每想到一种"骗绿"的错法就补一轮刺激：边界（等于门限）、口径（标志 vs 值）、回程（撤销）、 基线（原本没有 `-`）各一轮。 5. 加"值层"刺激前先确认**默认值在合法窗口内**，否则既有全量台会一起红。 6. 文档序列按**结构**锚定（收尾值 / 结尾形状），归档脚本的每次替换都要断言"命中恰好 1 次"。

记录 77　屏上的数字也有保质期：手抄常量、生成物与"验证链的真空"

这一轮做的事：把 P8「系统信息 / 自检」上两处"说了旧话"的字段改成单点来源（㊹，v47）： ① **固件版本停在** `V35` —— 它从 v35（P8 首次落地那轮）一直挂到 v46，十一轮没人发现； ② **资源占用写死** `65 / 41 / 23` —— 而 U-47 的 `impl_util.rpt` 实测 **LUT 67.51%**， 屏上少报了 2.5 个百分点。两处根因相同：**手抄常量，没有任何判据把它与当前位流绑起来。**

坑一：写死在"当轮是对的"位置上的常量，最容易被所有人忽略。 `disp/touch_view.v` 里 fld1 的渲染是 `case (p) 0:'V' 1:'3' default:'5'` —— U-35 那轮它是对的，之后每轮重建位流它就旧一分，而**每一轮的回归都是绿的**： 没有任何判据看"屏上的版本号是否等于这一轮的版本号"。 **教训：凡是"描述本次交付"的数字/字符串（版本、资源、日期），写死的那一刻就开始过期； 要么让它运行期算出来，要么给它配一条**每轮都会跑**的核对判据。**

坑二：资源百分比是"参与综合的快照"，流程必须是"先出位流再改常量"。 它不能像温度那样实时测 —— FPGA 读不到自己的 utilization report。 所以它只能是 `localparam`，改一次就换一版位流。于是流程被钉成 **"先出位流 → 读 `impl_util.rpt` → 若取整值变了再改常量并重出"**， 而不是"先写一个希望值"。这条和本轮以前那批"文档数字不许手抄"是同一族纪律， 只是对象从 `.md` 换成了 RTL 常量。

坑三：生成物要用"生成器输出"校对，不能只靠"我改了三格"。 `disp/text_ram.v` 是 `tools/gen_text_ram.py` 的产物。本轮把生成器输出导到临时目录， 与工作区版本逐行 diff：先看到"只差三格"，改完再 diff 到 `(identical)`。 **教训：对生成物，"改对了"的唯一证据是"与生成器输出一致"，不是"我改了"； 把这条做成一条可重跑的命令（`_gen_text_ram_tmp.py`），比在提交信息里解释强。**

坑四（本轮最意外）：沙箱策略会让"代码改完"与"已验证"之间出现一段真空。 会话的文件策略中途从 `danger-full-access` 切成 `workspace-write`，于是 `run_sim_disp.py` 写 `disp/logs/*.log`、lint 写

`_disp_work/lint_all*.log`、 Vivado 写 `_disp_work/impl_*` 全部 `PermissionError` + `[sandbox: file access denied]` —— **工具链本身依赖"能写工作区"**，而策略不在代码里。 处理方式：不在此之前声称验证通过，先把代码改完、把能做的证据（生成器 diff）留下， 等放行后再跑整套。**教训：报告"改完了"与"验证了"必须分开说； 把"因为环境限制而不能验证"当成一条要写进归档的事实，而不是沉默。**

顺带清掉的一笔：`CH_THREE` ('3') 在版本串改成两位十进制后已无人使用 —— 死常量就是下一个"看着在用、其实没人读"的坑，与本轮修的 `V35` 同类，直接删。

教训四条： 1. 描述"本次交付"的常量（版本/资源/日期）必须有**每轮都跑**的核对判据，否则写死即过期。 2. 参与综合的"快照数字"要写清流程（先出位流 → 读报告 → 必要时改常量重出），不能先写希望值。 3. 生成物只认"与生成器输出逐行一致"这条证据；把这条做成可重跑的命令。 4. "改完了"与"验证了"分开说；环境限制导致的验证真空要写进归档，不要用沉默掩盖。

记录 78　spec 的绝对坐标、字模码与"自由计数的刺激"：三处一抄就错的细节

这一轮做的事：补 spec 的两处字面缺口（㊺，v48）—— A1 卡内 1 px 分隔线 （`docs/13:162`）、A2 卡 4 次级 `pos_full 0x……` （`:188`），顺手修了一处串位 （`方向 CCW` 原本写在卡 4，按 `:187` 属于转速卡）。

坑一：spec 的绝对坐标不能照抄到不同尺寸的卡片上。 spec 写 `卡内 x 16..284` 是按 **300 宽**的卡片算的；本工程卡片是 **288 宽** （`CARD_W = 288`，另一笔历史偏差）。照抄 `16..284` 会让线右端跑到 `card_x+284`， 在 288 宽的卡里**压到右边框**——像素判据会绿得莫名其妙（那一条本来也是 `C_GRID`）。 按"左右各内缩 16 px"的**相对口径**落，并且把四个端点各判一次。 **教训：spec 里凡是 `x 16..284` 这种绝对坐标，先换算成"相对卡片哪条边、内缩多少"； 与实现尺寸不一致时用相对口径，并在归档里写清与 spec 的偏差在哪。**

坑二：判据里的字模码必须从生成物里抄，不能按段推算。 `方向` 两个字的字符码是 **73/74** （`tools/cn_glyphs.py` 的顺序），我第一版按 "汉字段从 148 起"写成 148/149 —— 必然红。**教训：字符码是生成物的第 N 项， 只认 `disp/text_ram.v` 里那一行 `// (17,46) '方'`，不认"看起来像"。**

坑三：判"运行期写进 RAM 的快照"，刺激量必须先冻住。 A2 的十六进制是每帧从 `q_pos_full` 写到 8 格；而台上 `t_pos_full` 是**自由计数器** （每时钟 +8192）。不冻住的话，"读回来的值"与"写进去的值"不是同一拍，判据自己红。 本轮用 `force t_pos_full = 32'h003A1B2C`（把 spec 的例子当刺激）再逐格比，读完 release。 **教训：快照类判据的刺激量要冻在一个已知值上；"让它自己跑"只适合时序类判据。**

坑四：串位只有"逐格对照 spec"才能发现。 `方向 CCW` 一直写在卡 4 的次级行（`(35,6)`），而 spec §4.5 把它归在**转速卡（卡 3）**。 它不违反任何既有判据（那格原本没人断言内容），所以能一直挂着。本轮把它迁到 `(17,46)`，卡 4 的次级行让给 `POS 0X` + 8 位十六进制。 **教训：新增一条次级信息时，顺手把同一行/同一张卡上的旧字面与 spec 逐格核一遍 —— "这格本来有字"不等于"这字放对了卡"。**

顺带记的一条结构事实：P1 的标签层写口现在有**三个**写者（`undo_lab` 行 34、 `stat_chip` 行 15/36、`pos_hex` 行 35），仲裁仍是"组合 `we` + `hold` 停拍"，靠**行号不冲突** 把优先级简化成三级 mux。这是 `stat_chip` 第 3 节那条教训的第三次复用： **`we` 不能寄存，否则"发出去了没人收"会每帧丢一格。**

教训四条： 1. spec 的绝对坐标先换算成相对口径，尺寸不一致时按相对口径落并记录偏差。 2. 判据里的字模码只从生成物抄，不按段推算。 3. 快照类判据要把刺激量冻在已知值上。 4. 补新信息时顺手逐格核同卡的旧字面 —— "有字"不等于"放对了卡"。

记录 79　"放不下"要给数，批量改名会要命：六通道条与一次 `dlg_kind` 血案

这一轮做的事：A3 的第一半（㊻，v49）—— 把 spec §4.5 :190 的"卡 6 底部迷你 6 通道 逻辑条"落到屏上：六道 = `io_state` 的 a/b/z/u/v/w，高电平整条、低电平只剩左端短桩。

坑一：spec 的"每通道 16 px"在本工程放不下，要拿数说，不要硬塞。 卡片 256 高（spec 264），val 行 7 占到 y 511，底部只剩 64 px；`6 × 16 = 96 px` 进不去。 把三个数（可用 64、要吃 96、按 8 px 落成 48）

写进归档，并指认它是 "卡片几何 288×256 vs 300×264"那一整笔偏差的一部分。 **教训：遇到"规格尺寸 != 实现尺寸"，先算"差多少、能不能塞、塞进去挤掉谁"， 再把偏差记成一条账；只说"差不多"等于给下一轮埋雷。**

坑二（本轮最贵的）：批量改名用短子串，把接口改了。 为了绕开符号撞车（`LG_*`、`MN_*` 都与 `disp_engine.v` 里 P4 的既有族冲突），我做了 整文件替换 `lg_k → mn_k`。结果：`ui_dlg_kind` 端口里含 `lg_k` ⇒ 端口名被改成 `ui_dmn_kind`，tb 立刻报 `cannot find port 'ui_dlg_kind'`；`DLG_X0`、`lg_lvl` 也一起被改。处置：`git checkout -- disp/disp_engine.v` 整文件回滚， 再用**一次** `edit` 只插入 A3 那一段，符号统一加 `C6_` 前缀。 **教训：① 批量改名只允许"整词/带边界"的替换，短子串（`lg_k`、`col`、`x0`）一律禁用；② 改名前先 grep 该子串的全部出现位置；③ 命名优先用模块专属前缀（`C6_`）避免撞车； ④ 撞车时先回滚再重做，不要在已经被污染的文件上继续打补丁（两次都是这么红的）。**

顺带记的一条结构：六通道条走的是 `bg_sel → bg_c6 → bg_grid` 这三级配色链， 位置在"卡片底"之上、"边框/卡内分隔线"之下 —— 这样它不会盖住边框与 A1 的分隔线， 也不会盖住任何文字（文字在更外层合成）。**新增图元要先问"它在哪一级配色链上"， 位置错了就是"看不见"或"盖住别人"。**

教训三条： 1. 规格尺寸冲突要给出三个数（可用 / 需要 / 实取）并记成偏差账。 2. 批量改名禁用短子串；优先用模块专属前缀，撞车先回滚再重做。 3. 新增图元先定"配色链上的位置"，再谈形状。

记录 80　"没读数"不是数字：空态做到最后一页，与一次静默没生效的替换

这一轮做的事：B3（㊼，v50）—— 把 spec §8-8 的空态补到最后两处还写着数字的地方： P5 六状态表的「次数 / 占比」与 P2 右下四枚徽标（Vpp / f / Δt / ΔV）。 判据沿用 ㊷/㊸ 的 `sig_bad`（链路不可信 ｜ 无载波），两处各只改一个 `wdata` 三目。

坑一：差点拿 `99999` 当空态哨兵。 P5 的计数走 16 bit BCD 通路，`99999` 是**饱和点**（数满了就停在那里），不是哨兵。 用它判空态，会把"数满了"和"链路断了没数"混成一件事 —— 而这正是空态要防的误读。 **教训：空态判据只能来自"信号状态"，不能来自"数值形态"。凡是溢出/钳位/饱和出来的值 （`99999`、`0xFF`、`65535`）都不是哨兵，它们本身就是合法读数。**

坑二（本轮最贵的）：替换串静默没生效，把上一轮的说明写错了一行。 U-50 的漂移表末行本该是"㊻ 卡 6 底部迷你 6 通道逻辑条"，实际写成了"㊻ 卡内 1px 分隔线 + 卡 4 次级 pos_full 十六进制"（那是 U-49 的说明）。原因：`_mk_u50_helpers.py` 里改 `hist.append(...)` 的那条替换串是按 **COMMON 替换之前**的形貌写的（`"U-49"` / `"㊺"`）， 而它真正执行时 COMMON 已经把该行改成了 `"U-50"` / `"㊻"` —— 于是 `s.replace()` 什么也没做， **而且当时那件脚本根本没校验"是否命中"**，所以连个警告都没有。 处置：① 本轮给每件派生脚本加 `CHECKS`（写完之后必须含哪些关键串），缺了就报错退出； ② "没命中"要逐条打印出来，让人看见；③ 在 `u51_numbers.py` 里加一行订正， 把 U-50 那行的说明改回真实内容，并在 U-51 的归档里记一笔勘误。 **教训：① 生成式脚本必须校验"产物里有关键串"，而不是"脚本跑完了"； ② 派生链每长一轮，上一次的"入口数字"（说明文字、编号、基准行）都可能被静默漏改， 越是"看着对"的地方越要断言；③ 静默的 no-op 比报错难查一个量级 —— 它连红都不红。**

坑三：`xsim.dir` 被占用，两次"红"都是环境不是代码。 `xsim.type` 写不进（`43-3356`）→ `xsimk.exe` 删不掉（`43-3345 + 43-3238 Failed to link`）。 处置：`Stop-Process xsimk,xelab,xvlog` → 删该台工作目录 → `--rebuild` 单跑。 **教训：同一个台不要并发跑两次；中断过的仿真要先清 `xsim.dir` 再跑。**

顺带记的一条数：本轮只加了两处三目选择（+7 个 LUT），WNS 却从 +0.438 涨到 **+0.913 ns**。 又一次说明 WNS 的绝对值不能跨版本横比 —— 要看的是"失败端点是否为 0"与"最差路径落在哪个模块" （本轮保持最差那条第一次落进 `u_disp/u_p2_badge` 的除法器）。

教训三条： 1. 空态/无效态判据只能来自信号状态，饱和值不是哨兵。 2. 生成式脚本必须校验产物关键串；"没命中"必须报错，不能静默 no-op。 3. 环境锁（`xsim.dir`）先清再跑，同一台不并发。

记录 81　十进制位权不是移位：一个分频器里的三处算术错，与"16 分钟一轮"逼出来的单元台

本版要在编辑面板里算出 **等效倍率 = inter/div** 与 **AB 边沿率 = 4 × 等效倍率 × f_in**。 两个数都只要一位小数，所以做法是：一个 32 位恢复余数分频器**复用两次**（相位 0 占 12 拍、 相位 1 占 24 拍），全在垂直消隐期跑；像素通路一行没动，也没用 DSP 除法核。位置够、代价小 （LUT +281、WNS 掉 0.605 ns），难的是**把数算对** —— 三轮仿真里屏上打出来的数与期望值 一条不差地差在算术上：

1. **`.0`**（商恒为 0）。根因：恢复余数法每拍看 `sh[30]`，而我把 12 位被除数写进了**低位** （`{19'd0, iv * 4'd10}`）⇒ 移位 12 次之后商位全落在寄存器外面。修法：左对齐 `{iv * 4'd10, 19'd0}`（12 + 19 = 31 位）。
2. **`39.3` / `69.6` / `19.4`**（应为 `33.3` / `64.0` / `17.4`）。根因是拆十进制的减法链 写成了 `m - {c_div100(m), 8'd0}` —— **`{x, 8'd0}` 是 ×256，不是 ×100**；`{x, 10'd0}` 是 ×1024，不是 ×1000。333 减 3×256 直接借位，于是高位退一格、低位乱码。修法：十进制位权 **只能用移位相加凑** —— ×1000 = 1024 − 16 − 8 = `{d,10'd0} - {d,4'd0} - {d,3'd0}`、 ×100 = 64 + 32 + 4 = `{d,6'd0} + {d,5'd0} + {d,2'd0}`、×10 = 8 + 2。
3. **`.3`**（应为 `0.3`）。根因：整数位三格只在 `val >= 10` 时写数字， `val < 10` 就留空格 —— 前导空白逻辑把"该有一个 `0`"当成了"没有整数部分"。修法：个位那格永远写 `CH_D0 + d1`。

三处都不是"设计没想清"，而是**写代码时把位运算与十进制混了**。把它们抓出来的是判据，不是 再读一遍代码 —— 但 16 分钟一轮的引擎台跑不起这个循环（`tb_disp_engine` 一轮 **927 s**，而 §[19] 只占 8 条断言）。于是另建 `_disp_work/tb_pnl_calc_unit.v`：只例化 `pnl_calc`， `always @(posedge clk) if (we) cap[waddr] <= wdata;` 抓 5 格，**10 秒一轮**，八组用例 （×100÷3 / ×64÷1 / ×256÷8 / ×2÷8 / `sig_bad` / 负频取幅值 / 1 MHz 饱和 / 面板关着不写）。 **先单元台全绿（14 条）再跑引擎台**，引擎台一次就绿（341 条 `[ OK ]`、0 条 FAIL）。

两条不是算术的坑：

4. **面板里没有"随便放"的位置。**第一版把两行数字放在行 25/27 列 22..31，看着没问题，但 引擎台 §[9b] 立刻红：`[FAIL] ÷7 拆位/前导空白与键盘不一致：213/12/852 vs 213/12/852` —— 打印出来的三格墨量一样，可它还要求"另两格 0 墨"，实得 213/12。根因：那两行落在 KPV 数值大窗（x 330..572 / y 342..442）**里面。**面板里唯一的空白带是 x 576..840 / y 360..440，最终落在行 23 列 41..45（x 656..735）与行 25 列 42..46（x 672..751）。 顺带一条：**排除窗必须同时盖住标签与数值**，只盖数值列时面板一收起，标签就留在页面上。
5. **`cell` 是 SystemVerilog 关键字**（`syntax error near 'cell'`，VRFC 10-4982），单元台里 `reg [7:0] cell [0:4095]` 必须改名 `cap`；另外本机控制台是 GBK、xsim 日志里有非法字节， `print(out[-8000:])` 会抛 `UnicodeEncodeError: 'gbk' codec can't encode character '\ufffd'` —— 量具脚本的输出要先净化（decode gbk/replace 再 encode/decode）并落盘，再挑行打印。

口径记账：spec §6.3（`docs/13_display_ui_requirements.md:284`）要"等效倍率 = inter/div 与预测的 AB 边沿率"，而卡片 2 上的静态标签是 v44（㊶）引入 `div` **之前**写的 `边沿率 4×倍频×F`。屏上的数按新口径算（`4 × (inter/div) × f_in`，单位 kHz 一位小数、 上限 999.9 饱和），这个偏离记在 `DEV_STATUS.md` 的 U-52.2。

教训四条： 1. 十进制位权只能用移位**相加**凑（×1000 = 1024−16−8）， `{x, 8'd0}` 永远不是 ×100。 2. 分频器的被除数必须**左对齐到最高有效位**，否则商位会在移位过程中被丢掉。 3. 大台的循环周期决定"能不能迭代"：16 分钟一轮的集成台后面必须垫一个 10 秒一轮的单元台， 否则算术错会被"再读一遍代码"糊过去。 4. 面板的空白带是一条**几何事实**：加字段前先算像素范围；排除窗盖住"标签 + 数值"整段。

记录 82　加一个字模加在哪里，比字模本身贵

本版要按规范 §4.5（`docs/13_display_ui_requirements.md:190`）把卡 6 补成 `A● B● Z○` / `U V W` —— 屏上缺的其实只有一样东西：**空心圈** `○`。字库里早就 有实心 `●`（码 48），没有 `○`。于是"加一个字模"看着是最小改动，实际有三层：

1. **加在 ASCII 表末尾 ⇒ 132 个汉字码整体 +1。**字库的真实结构是 `CHARS = ASCII_CHARS + CN_CHARS`，两个平面共用一张表；把 `○` 追加进 `ASCII_CHARS`， `disp/font_rom.v` 立刻位移 **+2,238/−2,158**、`disp/text_ram.v` **306/306**（每一行汉字都挪位），更要命的是 `disp/stat_chip.v` / `disp/selftest_lab.v` / `disp/undo_lab.v` 与七个 tb 里有一百多处按索引写死的 `8'd159` 之类。
2. **按"旧码 → 汉字 → 新码"批量改也救不回来。**写了 `_disp_work/fix_chcodes.py` 想按行尾注释里的汉字反查，改到一半才发现注释形态太杂：`// ●正常`、 `// 按（v38 新增）`、 `// '1'（div 码 0 = ÷1）`，按注释取字会取到「码」「新」 这类噪声； `git checkout --` 回滚。
3. **真正便宜的加法是"平面各自在字符表末尾追加"**：`VAL_EXTRA = "○"`（val 码 **51**）、 `LAB_EXTRA = "○"`（lab 码 **183**，接在 132 个汉字之后）。两个平面的字符表与 ROM 本来就分开（lab 8 bit / val 6 bit），RTL 从不跨平面传码 ⇒ **同一个字两个编号， 汉字码一个都不动。**最终改动面塌缩到两个生成器 + 一处 tb 映射 （`disp/font_rom.v` +100/−11、`disp/text_ram.v` +3/−1，而且后者只改了头部注释）。代价是 +36 个 LUT、WNS 涨 0.079 ns。

真正花时间的不是字形，而是**测试台自己会动**：引擎台里 `t_iostate` 由编码器模型每帧 旋转（`disp/tb_disp_engine.v:95-103` 的 `t_iostate <= {t_iostate[6:0], t_iostate[7] ^ t_iostate[5]}`）。§[20] 写完第一次跑，5 条 FAIL 里有 3 条是这么来的：赋 `8'h38`（三路全高）却读到 `U○ V● W●`、赋 `8'h10`（只有 V 高）读到 `U○ V○ W●` —— **赋进去的值在等待的两帧里 被移了两位。**修法：判电平之前先 `tap_freeze = 1'b1` 冻住模型（`frame_st` 帧计数与 `wait_frames()` 照常走），改完值等 3 帧再读，收尾解冻。

另外三处坑，都是"工装"不是"功能"：

4. **xsim 不允许 force 位选**：首版 §[20] 第 ③ 组写 `force u_dut.q_io_state[2:0] = 3'b101;`， xelab 报 `ERROR: [VRFC 10-3149] bit-select or part-select is not allowed in a force statement for non-net 'q_io_state'`。改从端口侧给（`t_iostate` 才是驱动源， `q_io_state` 只是打一拍）。
5. **PowerShell 的 `Get-Content | Set-Content -Encoding UTF8` 会毁掉中文源文件**：抬帧数 上限时图省事用了它， `disp/tb_disp_engine.v` 当场变成乱码（CP936 解码再写回，行也 粘连，3,102 行变 2,580 行），只能 `git checkout --` 回滚。**而回滚是整文件的** —— 同一轮里先前改好的 §[11b] 判法也跟着回了旧版（"8..17 是数字"），复跑时多红 2 条， 等于替这处回滚交了第二次学费。
6. **列号也要当场核一遍**：§[20] ③ 首版把 `Z` 判在列 30、圈判在列 31，而字段 5 是 8 格落在列 **23..30**（`f_row = 7 / f_col = 23 / f_wid = 8`）⇒ 复跑报 `got 18 48 0 19 51 0 48 0`（列 30 读到 ●、列 31 还没写）。差一格错的不是逻辑，是数 格子。

教训四条： 1. 给共用字符表加字是**全局改动**：先问"能不能只加在一个平面的末尾"，再谈字模好不好画。 2. 想批量改"按索引写死的码"，先确认**能从旧表解码**（`git show HEAD:…` 取旧 `CHARS`）， 别从注释里猜字。 3. 带自激励的测试台，判"输入 → 显示"之前必须先**冻住输入**；否则读到的是移位后的值。 4. `git checkout -- <file>` 是整文件回滚：回滚后要把**这一轮改过的所有段落**重新核对一遍 —— 而中文源文件永远不要经过 PowerShell 的文本管道。

记录 83　一条被自己删掉的入口，和一个吃掉判据前缀的 72 字节

本版按规范 §4.6（`docs/13_display_ui_requirements.md:195-204`）补两条卡片下钻： 「点卡片 3 → P7 标定、点卡片 6 → P4 逻辑时序」。写之前翻了一遍历史，发现 **卡片 3 这条十年前就已经被我们自己删过**：v20（⑳）那轮的记录里写着"§4.5 把第 3 张卡 的内容钉成『转速』，与『点它进标定页』像是矛盾，所以只保留标题带的『标定』按钮"。 这一轮把两处条款并起来读才看清：**§4.5 管卡片画什么、§4.6 管点击去哪里**，它们描述的 是同一张卡的两个侧面，根本不冲突 —— 当年的"矛盾"是自己把两条当成了同一条。

于是这版做成**双入口**：卡片 3 与标题带按钮都能进 P7，不互相替代。改动只有三处： 一个 `localparam [4:0] HIT_CARD_ABZ = 5'd13;`（第 6 张卡在 P1 网格里的编号 = `8 + 列号 + 3×行号`）、两条 `wire tap_card_cal / tap_card_abz`、以及 `page` 那个 always 块里两个带 `page == 3'd0` 守卫的赋值。-5 个 LUT，WNS 涨 0.279 ns —— 又一次 和改动量不成比例，只能记进"布局噪声"那一栏。

真正花掉两个小时的是两件**工装**的坑：

1. **`chk_true` 的形参只有 72 字节。** `disp/tb_enc_disp_panel.v` 的 `task chk_true(input [8*72-1:0] label, input cond);` —— 中文按 UTF-8 是 3 字节/字， 一条 30 字的标签就 90 字节，超长时 Verilog **截高位**，也就是**把标签的前半段 连"8b "前缀一起削掉。**日志里于是只剩后半句，grep `8b` 的人看到 10 条，以为"只跑了 10 条判据"，实际 **19 行全执行了**（18 条判据 + 1 条反骗绿）。 更糟的是这些标签在日志里根本不是给人看的：`label` 是**字节向量**，xsim 按代码页 重新编码，中文全变成替换字符（`[ OK ] 8?????3 -> P7 = 6`），只有 ASCII 那部分 （`P7 = 6` / `P1 = 0`）还能读。修法不是抬位宽（那是全局改动，且会连带所有 tb 的 label 缓冲），而是**把标签压到 24 个汉字以内**，并用一个逐条命中校验的脚本（`_disp_work/_fix8b_u54.py`）改，避免改漏。
2. **xsim 日志里的中文本身就是坏字节。**它不是 UTF-8，是按控制台代码页写出来的， `Select-String` 读它会"少 10 行"——看着像判据没执行，实际是解码把整行吃了。 所以判据的**计数**必须落在 ASCII 上（`OK` / `FAIL` 行数、自己累加的计数器）， 中文只用来给人看；读日志一律 Python `errors="replace"`。

还有一条是给"页面跳转"这类判据专门加的**反骗绿**：只断言"当前页 == 期望页"很容易 全绿——只要起点恰好等于期望值，后面一连串"点一下看看是不是没变"就自动通过。 所以 §8b 段首挂了一个 `pg_chg` 计数器（`always @(posedge sys_clk) if (u_dut.page !== pg_last) …`），段末追一条 `chk_true("8b 本段真的翻过页（>=6）", pg_chg >= 6)`； 实测这一轮翻了 **8** 次页。

教训三条： 1. 删掉一条需求之前，先确认它是**被另一条覆盖**还是**只是没实现。**这次是后者， 而当年的记录把"没实现"写成了"矛盾" —— 于是这条入口白丢了三十多轮。 2. 断言标签也是代码：**有长度上限的形参**会把中文标签截断，截掉的那一半恰好是最 有用的前缀（段号）。标签要短、要带 ASCII 段号。 3. 只判"状态等于期望"会骗绿；**同时判"这段确实发生过变化"**（计数器 / 次数）。

记录 84　"现在没有时钟"和"曾经错过一次"，是两种状态

本版补两件**遗留了两轮**的事：链路静默要进空态（C3），以及屏上「清除锁存故障」真能清掉 链路的粘滞故障位（C4）。两件事看着无关，根子是同一个：我们一直只有一个"不可信"的 信号（`link_fault_seen`），而它把两种完全不同的语义混成了一种。

1. **`link_fault_seen` 是事件**：主机在帧尾之后继续打时钟，说明帧边界错位了。它一旦发生 就不会自己消失，只能由人确认后清 —— 所以它必须是粘滞位，也必须有一条 W1C。
2. **没有时钟是状态**：主机不打时钟了，帧同步沿不再出现。它会自己回来（主机恢复就打）， 所以它必须是**非粘滞**的：帧同步沿之后的计数超过 1 ms 就置位，来了边沿立刻落下。

以前屏侧只有一个来源，于是 `sig_bad = link_fault_seen | 无载波`：主机停摆时屏上一切照旧， 显示最后一次读数。 `disp/disp_engine.v:677-685` 当时还留了一句注释："抽头里没有链路心跳 ⇒ 纯链路静默本版判不出来 —— 要么改 rtl/（用户已明确排除）"。这一轮用户把它列进了需求， 那句话就成了"该删的注释"：**注释里写"用户排除了什么"是会过期的**，比代码更容易骗人， 因为下一轮读它的人会当成设计约束。

实现比想象的小：`rtl/enc_top.v` 里一个 24 位计数器（帧同步沿清零， 到 `LINK_LOST_CYCLES = 100000` 置位）、`rtl/reg_file.v` 里一行 `clr_link_fault <= we && (addr == 16'h0405) && wdata[0];`（与 `clr_otr` 同脉冲）、 `rtl/enc_top.v` 里把 `link_lost` 拼到 `disp_tap[375]`（宽度 375 → 376），显示侧只加一个 或门输入。+29 个 LUT，WNS 掉 0.161 ns。

这一轮真正花时间的还是**测试台**，三个坑：

1. **门限取成了帧间空隙**：`LINK_C = 20` 拍 @10 ns = 200 ns，而 tb 每帧开头那段 200 ns 空闲正好也是 200 ns ⇒ "正常打时钟时不许置位"那条当场假红。测门限的 用例，参考量级必须跟被测协议的帧间空隙无关，别取"刚好相等"。
2. **异步输入的同步链会吃掉余量**：帧同步沿走 3 级同步才产生内部脉冲，边沿比 tb 的赋值晚 2~3 拍才被认出来。只多等 4 拍（`LINK_C + 4`）时计数器差一拍到不了门限，第二条假红 （`gap_cnt=60` 而 `link_lost=0`）。改成 `LINK_C + 16`。
3. **层次名写错**：`disp/tb_disp_engine.v` 的 §[21] 先写成 `u_dut.u_disp.sig_bad`， xelab 报 `ERROR: [VRFC 10-2989] 'u_disp' is not declared under prefix 'u_dut'` —— 这个测试台**直接例化** **`disp_engine`**，`u_dut` 就是引擎本身。全量回归第一轮因此 24/25（唯一一条红）。

判据上有一条是这一轮特意加的：**"落回"必须单独判。**只判"拉了 `link_lost` 就进空态" 是不够的 —— 把非粘滞实现成粘滞（比如忘了"边沿清零"那一路）时，那条照样绿。所以 §[21] 第三条反着拉一次，要求 `sig_bad` 回 0、数值行恢复。

教训三条： 1. 一个"不可信"信号里混了两种语义（状态 vs 事件）时，先拆语义再谈实现；**粘滞位一定 要配一条清位通路**，否则"故障清掉了、画面还是空态"这种状态会把用户锁死。 2. 注释里写"某方案已被排除"时，要注明**是谁、在什么需求下排除的。**需求会变，注释不会 自己更新。 3. 测门限值时，参考时间不能与被测协议的空隙取值相当；异步输入还要给同步链留余量。

记录 85　常量会对，注释会旧：一次"只改文字"的小批，和一棵被别人的工作树挡住的位流

这一版起因是一次对账：上一轮收尾时把 P8 的字段逐个对回源文件，发现屏上 `固件版本` 显示 `V47`，而位流已经是 v54。往回翻，`FW_MAJ/FW_MIN` 是 U-48 写下的（那时正好 v47），此后 七版没人碰过它。同一次对账还发现 `LINK_LOST_CYCLES = 100000` 的注释写"1 ms @100 MHz"， 可板子的 `sys_clk` 是 20 ns（50 MHz，`constr/enc_top_disp.xdc:146`）—— 100,000 拍是 **2 ms。**

两件事是同一类错误：**值是对的，话是旧的。**而且都不是靠"再读一遍代码"能发现的 —— `FW_MAJ = 4'd4` 本身完全合法，`// = 1 ms` 也完全像真的；只有把屏上的字、位流的版本号、 约束文件里的周期三样摆在一起比，矛盾才跳出来。

所以这一批的定位是**最小批**：`LINK_LOST_CYCLES` 一个数都不改。理由不是为了省事： 1. 2 ms 是当前板子**真实发生的行为**，把它改成 1 ms 是在"改版本号"的批次里偷偷改一个 安全门限； 2. 1 ms 会贴线 —— 合法链路的帧间隔可以低到 1 ms 量级，门限取 1 ms 会让 一台"合法但慢"的机器被误判成失联； 3. 旧注释的**理由**才是错的：它写"最慢的合法链路也只占 1 ms 的 1%"，而那个 1% 是按 最高帧率算的，与"最慢的合法链路"没有关系。修正一个错数字容易，修正一条 被抄了三处的错理由更值钱 —— 契约 §1.10、显示侧文档都抄了它。

顺带修掉了整段派生结论：过采样粒度 10 ns → 20 ns、3 级同步流水 30 ns → 60 ns， 外部时钟上限从"10 MHz（绝对 16.67 MHz）"变成 **6.25 MHz（保守 5 MHz，绝对 8.33 MHz）**。 契约 §1.2 的表里本来就写着"50 MHz ⇒ ≈6 MHz"，是文件头这段推导与契约表各说各话了两版。

版本号这一侧真正的难点不在改值，在**这个值散在三处**：`disp/touch_view.v` 的 `FW_MAJ/FW_MIN`、`tools/gen_text_ram.py` 的 `P8_VAL` 表、以及生成物 `disp/text_ram.v` 里 两个字形码（`mem[46]` / `mem[47]`）。只改前两处不重跑生成器，屏上会是 `V45` —— 半截版本号 比错的版本号更难查，因为前两位是对的。

这一轮最贵的一课与代码无关：**位流被别人的工作树挡住了。**工作区里另有一条在飞的工作线 （B1 抓包 / B2 千兆以太网），它已经把 `rtl/enc_top.v` 改了 +267 行并例化 `eth_top`，但 `rtl/eth_top.v` 还在**未跟踪**的 `rtl/eth/` 里，`tools/impl_disp.tcl` 只 glob `rtl/*.v`：

```
ERROR: [Synth 8-439] module 'eth_top' not found [C:/Users/zhaoy/Desktop/FPGA/rtl/enc_top.v:1474]
```

这不是本批的 bug，但它让"重出一版位流"这件小事变成了"先把别人的半成品收拾干净"。 不碰别人的文件是底线，于是走了 `git worktree`：检出 HEAD（b62eb02）得到一棵干净树， 把这批 9 个文件覆盖进去再 impl。**第一遍还是失败** —— 因为 `tools/impl_disp.tcl:25` 是

```
set ROOT   "C:/Users/zhaoy/Desktop/FPGA"
```

硬编码绝对路径，脚本"在 worktree 里"，读的却是主树的源文件。复制一份 `tools/impl_disp_wt.tcl` 把 ROOT 指到 worktree 之后才真正干净。

教训四条： 1. **"值对了"不等于"话对了"。**版本号、门限、单位这类"附在常量旁边的字"，必须有机器判据，靠人眼在两页之间对照，在有更急的事时必然失效。 2. **一个事实写在三处**（屏侧常量、生成器表、生成物），就要有一条"三处同版"的判据； 漏了生成器会得到"半截版本号"这种最难查的错。 3. **修正错结论时，先看它被抄到了几处。**这次错的是"1 ms @100 MHz"这条**理由**，它同时 躺在 RTL 注释、契约、上板文档三处 —— 只改代码注释等于没改。 4. **共享工作区里，构建脚本的"绝对路径"是定时炸弹。**要么从脚本自身位置推导 ROOT， 要么明确写入文档："这个脚本只能在 `<root>` 下跑"。

记录 86　清理也要有判据：三处"人能读错"的地方，各补一条机器检查

上板前盘点，剩下的事都有同一个形状：**同一条事实有两个版本，而两个版本都很像真的。**

其一， `docs/15` §4.42 的易混点 3 写着"面板也可由卡 6 底部迷你逻辑条打开（㊻ 加）"。 这句话我自己的说明书里也照抄过，因为那一段字面看起来像是逐条核对过的。直到把 `pnl_open_req` 的置位点全仓 grep 一遍，才发现**全仓只有一处**：`disp/enc_top_disp.v:833` `if (tap_card_md && (page == 3'd0) && (dlg == 2'd0) && !w1c_busy) pnl_open_req <= 1'b1;`， 而 `tap_card_md` 只对命中 id `9`（卡片 2）有效，迷你条在 `hit_test.v` 里根本没有命中 id。 一句话错了三版，没人被它绊倒，因为**没人真的去点那条迷你条。**

其二，P8 的三个资源百分比是手抄常量。上一轮重建位流后 LUT 从 69.69% 变成 69.68%， 取整仍是 70，于是没改 —— 但那次"没改"靠的是我记得去比，不是靠任何检查。这类常量 每隔几版就少报一点，屏上照样显示得好好的。

其三， `sim/logs/SUMMARY.md` 被跟踪，而 `disp/logs/` 整目录早在 `.gitignore` 里。 于是每跑一轮回归就多一个必脏文件，此前两轮各手工 `git checkout --` 过一次。

三件事的处理都选了"补判据"而不是"这次改对"：新增 `tools/gen_res_pct.py`（读 `_disp_work/impl_util.rpt` 与三个常量对账，不一致退出码 1， `--write` 可按报告改写）、 把 SUMMARY 两份从索引里摘掉并加 ignore、把 §4.42 那句错话连同"为什么错"一起写回文档。

这一轮最狼狈的一次是自己的手滑：改 `tools/run_sim.ps1` 时用 Python 以 `utf-8` 写回， 把它的 **UTF-8 BOM 删了。** `.gitattributes` 明明白白写着 `*.ps1 binary`，注释里还解释了 原因 —— 这类脚本带中文注释，PowerShell 5.1 读**无 BOM** 的 .ps1 会按 GBK 解码， "半个汉字"会吃掉字符串的收尾引号，直接 `ParserError: Unexpected token '}'`。 更要紧的是它是 binary：**无法按 hunk 挑**，而它同时被并发线改过（`rtl/eth/` 、 `rtl/ddr3/` 入回归）—— 一旦 `git add` 就把别人未完成的改动一起提交了。最后是整段撤回 + 还原 BOM， 本批对 `tools/run_sim.ps1` **零改动。**

教训四条： 1. **"这句话看起来核对过"不是证据**，置位点、命中 id 这类事实要 grep 到唯一出处； 文档里的错话比代码里的 bug 更长寿，因为没人会去执行它。 2. **手抄常量要被工具对账**，而且要对着"这一次"的报告，不是记忆里的百分比。 3. **构建产物不该进仓库**；加 ignore 之前先查它是否被跟踪 —— `disp/logs/` 被忽略不等于 `disp/logs/SUMMARY.md` 没被跟踪，这里是 `sim/` 那份单独被跟踪。 4. **改脚本前先看**

`.gitattributes`：`binary` 的文件既不能按 hunk 挑，也不该由"顺手改两行" 的人去碰；还有，`.ps1` 的 BOM 是功能，不是风格。

---

5. 结论：这套协作方式真正起作用的地方

1. **不是"AI 写得快"，而是"AI 愿意反复怀疑自己"。** 本记录 70 条里有 67 条是自我纠错，而每一条的修正都不是靠"再想一遍"， 而是靠**一次新的实测或一个新增的静态检查**把错误暴露出来。
2. **判定权必须交给确定性工具。**人（含 AI）的自述只解释，不下结论。
3. **失败必须入档。**被证伪的预测、被推翻的手册建议、被自己抓住的解码错误 —— 这些恰恰是"设计过程中的判断依据"（竞赛明确要求写进报告， `COMP_GUIDE_2026.txt:825`），也是最难造假的一部分。
4. **沉淀成技能包才有复用价值。**见 `skill/README.md`：7 篇方法长文全部来自本记录的失败与观察； 2026-11-03 又把这 7 篇里"只能读、不能跑"的部分做成了 19 项可执行资产（6 个校验脚本、7 个模板、 5 篇提示词工作流、18 条踩坑清单），并用 `skill/scripts/skill_selftest.py` 的 14 条用例（含 4 条 负对照）把"这份技能包本身能不能被判错"也固定下来。

---

6. 追加：v70 转台死锁的两次误判与四步定位（2026-10-31 – 11-01）

> 本节是 2026-11-01 真机调试的追补。§2 的 86 条写于 10-19 之前，v70 暴露的是**两个只在 真机跨相位边界时才出现的 RTL 死锁**，与 §2 里的测试台/文档错误不同类：仿真全绿、时序收敛、 构建签核全过，仍然整机失效。这一节只写**可核查的动作与产物**，不写"AI 猜对了"的故事。

6.1 用户当时的约束（转述，非逐字引用）

用户的要求可以概括为三点：**停止加功能**；先把 v69 位流放到转台上跑真实数据；**用真机现象决定改哪里**，而不是继续在仿真里找。这个约束决定了后面的顺序：先证伪配置假设，再动 RTL，每改一版都必须有 一个"能在 10 秒内判定生死"的判据。

6.2 第一次误判：把"计数恒 0"归因于 PPR/POLE_BITS 配置

- 现象（v69，BUILD `0x0011`）： 10 s / 101,718 帧，`cycle_cnt` 恒 0；RPM 显示 0；Z 不出现； UVW 不换相；多圈计数恒 0。四个现象同源，指向"角度增量没有进累加器"。
- 误判：先怀疑转台齿数/极对数配置（PPR 32 倍、`POLE_BITS`）与 v68 不一致，准备改配置重综合。
- 证伪（A/B 换位）： 把同一转台换回已签核的 v68 位流，**症状完全相同** ⇒ 不是 v69 新引入的配置， 也不是"转台换了一台"。这一步把搜索空间从"配置/硬件"压到"RTL 的增量路径"。
- 代价：半天。教训：**能用一次 A/B 换位排除的假设，不要用读代码排除。**

6.3 第二次误判：只修前向，漏掉反向 UVW 的独立缺陷

修完 `cycle_cnt`（见 6.4）后真机 RPM/Z/多圈恢复，但反向转动时 UVW 仍只有 3 个扇区。 第一反应是"前向修法的边界常数没对齐"，准备把 `BND` 表重算一遍；实测 `tb_uvw_rev` 显示 反向 7 次里 82% 停在扇区 110，且 `r_sec==0` 时 `off_bwd = BND[r_sec] - eang` 的基准为 0， 角度绕回 65535 侧后偏移恒负 ⇒ 0→5 环绕边界被吞。**这是与 `cycle_cnt` 无关的第二个缺陷**， 只是被第一个缺陷掩盖了。修法：新增 `bwd_wide`，在 `r_sec==0 && eang_x>17'd54614` 时用 `17'd65536 - eang_x` 作偏移，与前向 `eang_x < 10923` 的写法同构。修后反向 50 次六态均匀 （6,055–6,246 次/态）。

6.4 四步定位（每一步都有一个独立判据）

步	动作	判据（不依赖模型自述）	结论
1	真机读 v69 的 4 个观测量	10 s 内 RPM/Z/UVW/多圈**同时死**	故障在"角度→增量→累加"这条公共路径
2	加探针位流（BUILD `0x0013`，`wrap_p_raw = pair_valid`）	探针 `hi distinct = 2049`（角度在动），而 `cycle_cnt` 仍是 0	角度有效、**增量计算错**，排除 ADC/前端/传感器
3	ModelSim 定向台 `tb_acc` / `tb_dbg6` 复现	`BADSTEP i=246 s0=32851 s1=32718 step=-65403 acc=32585`；`END cnt=0 acc=-12469 nbad=1`	跨 32768 时 `step` 变成 −65403（应为 +133）
4	读 RTL 定位表达式	`ang_s0/ang_s1` 是 0..65535 无符号相位，却被按补码做符号扩展再相减	根因：符号扩展把"绕回"解释成"倒退 65403"

修法（`rtl/pole_counter.v`）：

```
wire [AW-1:0] diff_u = ang_s0 - ang_s1;              // 先做无符号差（模 2^AW）
wire signed [AW+1:0] step_s = {{2{diff_u[AW-1]}}, diff_u};  // 再零扩展成有符号
```

`tb_acc2` 用 3000 个 +133 的增量驱动：修前 `cycle_cnt=0`；修后 `cycle_cnt=6`（理论 6.09），再来 3000 个 ⇒ 12。这个"6.09 → 6"的截断是**预期的**，因为 3000×133/65536=6.09 圈，整数计数器只应记 6 圈——它同时验证了"不重复计数、不丢圈"。

6.5 修复后的五条验收判据（2026-11-01 真机，全部通过）

1. Z 频率 **12.00 Hz**（转台 720 rpm × 1 齿/圈）；
2. 位置计数跨度 **−2047..0**、`distinct = 2048`（2048 PPR 一圈，正好不重不漏）；
3. 转速 **719–731 rpm**（与转台设定一致，误差 <2%）；
4. UVW **六态均匀**、换向 **293 次/s**（720 rpm × 4 极对 / 60 ≈ 48 电周期/s × 6 态 ≈ 288）；
5. 60 s 长跑 **598,115 帧 / 9,968 fps**，`r` 漂移 <0.02%，无丢帧累积。

6.6 这一轮真正的教训

1. **"计数器恒 0"优先怀疑位宽/符号，不要先怀疑配置**：跨 2^(N−1) 的相位差是最典型的静默错值，而配置错误通常表现为"倍数错"而不是"恒 0"。
2. **A/B 换位是最便宜的二分**：换回上一版签核位流只要几分钟，能直接排除"配置/硬件/转台"三类假设。
3. **探针要打在"被怀疑的输入"上，不要打在输出上**：`wrap_p_raw=pair_valid` 证明"角度在动"，这一步把"ADC 坏了"永久排除，比读一万行 RTL 有效。
4. **修完一个缺陷要重跑反向**：第二个缺陷（UVW 反向）本来就在，只是被第一个掩盖；单方向测试会漏。
5. **仿真全绿不等于真机正确**：这两个缺陷都不在 33/33 回归的覆盖里，因为回归用的是"小角度、不跨 32768"的激励。技能包 `testbench-blindspot.md` 因此新增了"边界跨越"检查项。